AlgoLang

50 Sample Programs

Input · AlgoLang code · Output in
Command line · **Graphical UI (Studio)** · **Prompt UI**

AlgoLang v0.2 Enterprise
AlgoProfessor AI R&D Solutions, Hyderabad
AlgoLang Innovator: Dr. S. Satyanarayana, Ph.D., PDF (AI)

Document prepared 30 October 2026

About this document

AlgoLang has *no functions*. Every unit of work is an `agent` (role, goal, tools, model, policy) and a `flow` only calls agents, runs them in parallel, branches on a score and emits results. This document gives **50 small programs** across 20 sectors. Each one shows its **input**, the **AlgoLang code**, and the **output** in all three modes of the toolchain.

The five program patterns

Pattern	Programs	How it works
single	4	One agent, one flow.
pipeline	15	Each agent's output feeds the next agent.
fan-out	9	`par { ... }` runs several agents together; one agent merges the results.
committee	4	Several scoring agents run in parallel; a chair agent decides. Protected attributes can be forbidden by policy.
router	13	A score decides which agent handles the case (`if score >= t`).
par for	5	`par for x in list` fans one agent out over a list; another summarises.

The three output modes

Mode	Start it with	What you see
Command line	`algo run file.algo --arg "k=v"`	A text trace: every agent call with its confidence, then the emitted values, token count and audit file.
Graphical UI	`algo studio`	Browser IDE: editor, agent graph, and a run panel listing each agent call. Shown here as the agent graph and run table.
Prompt UI	`algo ui`	Chat-style page: you describe a problem in plain English, AlgoLang picks the best domain pack, runs it, and shows every agent's work.

How the outputs in this document were produced

All 50 programs were compiled (`algo check`, no errors) and executed with AlgoLang 0.2.0 using its **offline mock backend**. Please read the outputs with these points in mind.

- **Mock backend.** It is deterministic and needs no API key. Agent text is a placeholder that echoes the agent's goal, and scores and confidences are generated. The outputs therefore show *what flows where* (calls, order, branches, policy gates), not the quality of a real model's answer. Run with `--live` and an `ANTHROPIC_API_KEY` for real Claude output.
- **Command line.** Pasted exactly as printed by `algo run`.
- **Graphical UI.** These are *not* screenshots. They are drawn from the data returned by the Studio server (`/api/graph` and `/api/run`). The graph shows the Studio graph with redundant edges removed; dashed lines are branch conditions. A red "human" tag marks a call that policy sent to the human review queue.
- **Prompt UI.** It does not run the program listed on the page. It matches the plain-English request to the closest of the 120 built-in domain packs and runs *that pack*. Where the library has no dedicated pack (for example a greeting), the closest pack is shown and the box says so.

Install: `pip install git+https://github.com/drssnaiml1-cell/algolang`, then `algo doctor`.

Objectives of AlgoLang

Main objective

To make AI work that is built from agents safe, governed, auditable and easy to read, by giving it a language of its own in which the only unit of work is an `agent` and the only way to combine agents is a `flow`.

Basic objectives

No.	Objective	What it means in the language
1	Agent-native design	There are no functions. Every task is an agent with a `role`, a `goal`, declared tools, a `model` and a `policy`.
2	Small, readable control flow	A `flow` can only do four things: call agents, run them in parallel, branch on a score, and emit results. Anyone can read what happens and in what order.
3	Safety before running	Mistakes are found by the compiler. Calling a non-agent, using an undeclared tool or passing a forbidden attribute is a compile error, not a surprise in production.
4	Governance built in	Token budgets, retries, forbidden attributes, and human review are part of the program text, and an organisation file can add stricter rules for every program.
5	Accountability	Every agent call returns a value with its **confidence** and provenance, and is written to a hash-chained audit log that can be verified.
6	Cost and speed control	Cheap models for easy steps, strong models where needed, response cache, token caps, and parallel execution with `par` and `par for`.
7	Reuse at scale	120 ready domain packs in 20 sectors, modules with `use`, and batch processing of thousands of records.
8	Access for everyone	Three modes on one engine: command line, graphical Studio and a plain-English Prompt UI, plus a Python SDK for developers.
9	Teachable and testable	An offline mock backend runs without an API key, so classes, tests and CI work anywhere. Live mode uses real Claude models.

Design principles

- **Explicit, never ambient.** An agent has only the tools it declares. Nothing is available by default.
- **The stricter rule wins.** When the organisation policy and a program's own policy differ, the stricter one applies.
- **Scores drive decisions.** A decision is made on a `score` between 0 and 1, so the reason for a branch is visible and can be recorded.
- **People stay in the loop.** A low-confidence call goes to a human review queue instead of going out.
- **One engine, many doors.** The command line, Studio, Prompt UI and SDK run the same compiler and runtime.

Who it is for

Domain experts	Read and review flows in sectors such as banking, health, law and government, and use the Prompt UI without coding.
Application developers	Use the Python SDK, company tools, behaviour tests (`*.algotest.json`) and `algo dev ci`.
Language contributors	Extend syntax, error codes, tools and packs using the grammar and the developer guide.
Teachers and students	Learn multi-agent design with the agent graph and the mock backend.
Compliance and audit teams	Check the audit chain, the review queue and the usage reports.

AlgoLang syntax and format

This section describes the syntax as it is used in the 50 programs of this document, all of which compile with `algo check`, and as it is described in the repository README. The formal grammar is the file `grammar.lark` in the repository (Lark EBNF); the sketch below is an informal summary, not a replacement for it.

1. Shape of a program

A program is a plain text file with the extension `.algo`. By convention the parts appear in this order: comments, `model` lines, `tool` lines, imports, a program-level `policy`, the `agents`, and then one `flow main`. Every program must have a `flow main` (error E111).

```
// 1. comment: what the program does
model fast  = "claude-haiku-5-5"        // 2. model aliases
model smart = "claude-sonnet-5-5"
tool  web.search                         // 3. tools the program may use

policy {                                 // 4. rules for every agent
    budget tokens 5000
    retry 1
    require human when confidence < 0.8
}

agent Name(input: text) -> text {        // 5. one or more agents
    role  "Who the agent is"
    goal  "What to do with {input}"
    model fast
}

flow main(input: text) {                 // 6. exactly one main flow
    answer = Name(input)
    emit answer
}
```

2. Basic elements

Element	Format	Notes
Comment	`// text`	To the end of the line.
Model alias	`model fast = "claude-haiku-5-5"`	The alias is used in agents. An unknown model is error E106.
Tool	`tool web.search`	Declares a tool. Built-ins: `calc.eval`, `file.read`, `kb.search`. Your own Python tools can be registered.
Name	Agents in `PascalCase`, variables in `lowercase`	The programs follow this convention. A duplicate name is error E107.
Text value	`"double quotes"`	Used for `role`, `goal` and model ids.
Interpolation	`{input}` inside a `goal`	Inserts a parameter. A name that is not a parameter is error E109.
Block	`{ ... }`	Used by `policy`, `agent` and `flow`.

3. Types

Type	Holds	Where it appears in this document
`text`	Words and sentences	Most inputs and outputs, such as P01 and P10.
`score`	A number from 0 to 1	Scorers and routers, such as P03 and P16. Only a score can be compared (E110).
`list`	A list of items	`par for` inputs and results, such as P08 and P24.
`json`	Structured data	Data passed between agents and tools.
`bool`	true or false	Yes-or-no values.

An unknown type is error E105. Every agent call returns its value together with a **confidence** and provenance.

4. The agent

```
agent FraudScorer(transaction: text) -> score {
    role  "Fraud analyst"                                  // who the agent is
    goal  "Score the fraud likelihood of {transaction} from 0 to 1"   // the task
    uses  web.search                                       // optional: a declared tool
    model fast                                             // which model alias
    policy {                                               // optional: agent-level rules
        budget tokens 6000
        retry 2
        require human when confidence < 0.70
        forbid gender, caste, religion
    }
}
```

Part	Format	Rule
Signature	`agent Name(p: type, ...) -> type`	Parameters have names and types. The result type follows `->`. A wrong argument count or name is error E102.
role	`role "text"`	The agent's identity.
goal	`goal "text with {p}"`	Required. No goal is error E112.
uses	`uses tool.name`	Only declared tools. An undeclared tool is error E104. A tool the organisation does not allow is E115.
model	`model alias`	Chooses a declared model alias.
policy	`policy { ... }`	Optional rules for this agent. See the next table.

5. Policy rules

A `policy` block can sit at program level (applies to all agents) or inside an agent. The organisation file `algo.config.json` is merged in, and the stricter rule wins.

Rule	Meaning
`budget tokens 5000`	Limit on tokens for the agent's calls.
`retry 2`	Number of retries if a call fails.
`require human when confidence < 0.8`	A call below this confidence goes to the human review queue.
`forbid gender, caste, religion`	Attributes the agents must not use. An agent that takes one as input is error E108.
`timeout, cache`	Further policy keywords for a time limit and for the response cache. See `DEVELOPER_GUIDE.md` for their exact arguments.

6. The flow: only four actions

```
flow main(application: text, ideas: list) {
    a = Intake(application)                      // 1. CALL an agent

    [x, y, z] = par {                            // 2. run in PARALLEL, results by position
        AnalystOne(a),
        AnalystTwo(a),
        AnalystThree(a)
    }

    results = par for it in ideas { Evaluator(it) }   // 2b. one agent over a LIST

    risk = RiskScorer(a)
    if risk >= 0.7 {                             // 3. BRANCH on a score
        plan = EscalationPath(a)
        emit risk, plan                          // 4. EMIT the results
    } else {
        note = RoutineNote(a)
        emit risk, note
    }
}
```

Action	Format	Rule
Call	`v = Agent(arg, ...)`	Only agents can be called. Calling anything else is error E101. An undefined variable is E103.
Parallel	`[a, b] = par { A(x), B(x) }`	The names on the left must match the calls inside, in number (E113). The calls run together.
Parallel over a list	`r = par for x in list { A(x) }`	The collection must be a list (E114). The result is a list.
Branch	`if score >= 0.7 { ... } else { ... }`	Compare a `score` with a number, using `>=` or `<`. Comparing a non-score is E110. Each branch can end with its own `emit`.
Emit	`emit a, b`	Returns the named values as the output of the flow.

7. Modules and packs

```
use "common.algo" as common              // import another program file
use pack insurance.claims_triage as claims   // import a ready domain pack

flow main(claim: text) {
    r = claims.Intake(claim)             // call an imported agent with its prefix
    emit r
}
```

Errors inside an imported file are reported with the file name, and a circular `use` is detected. The organisation policy is enforced at compile time, so a forbidden attribute or an ungranted tool fails before anything runs.

8. Compile-time error codes

Code	Rule	Code	Rule
E100	Syntax error	E108	Agent takes a forbidden attribute as input
E101	Calling something that is not an agent	E109	Goal uses a {name} that is not a parameter
E102	Wrong number or name of arguments	E110	Comparing a non-score value
E103	Undefined variable	E111	No `flow main`
E104	Tool used but never declared	E112	Agent without a goal
E105	Unknown type	E113	`[a, b] =` does not match the `par` block
E106	Unknown model	E114	`par for` over a non-list
E107	Duplicate definition	E115	Tool not allowed by the organisation policy

9. Informal grammar sketch

```
program = { model_decl | tool_decl | use_decl | policy } { agent } flow_main
model_decl= "model" NAME "=" STRING
tool_decl = "tool" DOTTED_NAME
use_decl = "use" ( STRING | "pack" DOTTED_NAME ) "as" NAME
policy = "policy" "{" { rule } "}"
rule = "budget tokens" INT | "retry" INT | "timeout" ... | "cache"
       | "require human when confidence <" NUMBER | "forbid" NAME { "," NAME }
agent = "agent" NAME "(" [ param { "," param } ] ")" "->" TYPE
        "{" "role" STRING "goal" STRING [ "uses" DOTTED_NAME ] "model" NAME [ policy ] "}"
param = NAME ":" TYPE
TYPE = "text" | "score" | "list" | "json" | "bool"
flow_main = "flow main" "(" [ param { "," param } ] ")" "{" { statement } "}"
statement = NAME "=" call | "[" NAME { "," NAME } "]" "=" "par" "{" call { "," call } "}"
       | NAME "=" "par for" NAME "in" NAME "{" call "}"
       | "if" NAME COMPARE NUMBER "{" { statement } "}" [ "else" "{" { statement } "}" ]
       | "emit" NAME { "," NAME }
call = [ NAME "." ] NAME "(" [ arg { "," arg } ] ")"
```

10. Running a program

Command	What it does
`algo check file.algo`	Compiles and reports errors E100 to E115.
`algo run file.algo --arg "k=v"`	Runs a program and prints the trace. Add `--live` for real Claude models.
`algo run pack:insurance.claims_triage`	Runs a ready domain pack.
`algo batch file.algo --input data.csv`	Runs many records in parallel.
`algo studio`	Opens the graphical IDE.
`algo ui`	Opens the Prompt UI (plain English).
`algo review / algo audit verify / algo report`	Human review queue, audit-chain check, and usage and cost summary.
`algo doctor`	Checks the installation.

11. Common mistakes and what the compiler says

Mistake	Code	Fix
Writing a helper that is not an agent and calling it	E101	Turn it into an agent with a role and a goal.

Mistake	Code	Fix
Using `web.search` without declaring it	E104	Add `tool web.search` at the top, then `uses web.search` in the agent.
Using `{topic}` in a goal but naming the parameter `subject`	E109	Make the goal and the parameter names match.
Branching on a text value	E110	Make an agent return a `score`, then compare that.
Two agents on the left of `par` but three calls inside	E113	Make the number of names equal the number of calls.
Running `par for` over a single text	E114	Pass a `list`.
No `flow main`	E111	Add one; it is the entry point.

Why AlgoLang matters

Most AI applications are built by wiring prompts into ordinary code. The result works in a demo, but it is hard to audit, hard to cost, and easy to break. AlgoLang changes the unit of work: there are *no functions*, only governed `agent`s, and a `flow` that can call them, run them in parallel, branch on a score and emit results. That small surface is the main source of its value.

Main importance, by aspect

Aspect	Common problem	How AlgoLang helps	Seen in
Safety by construction	An LLM call can use any tool, or be applied to the wrong thing, and nobody notices until it runs.	The compiler rejects the program first. E101 stops calls to anything that is not an agent. E104 means an agent can use only the tools it declares. E115 enforces the organisation's allowed tools.	P09, P41
Fairness	Protected attributes leak into scoring and no one can prove they did not.	`forbid gender, caste, religion` is part of the policy. E108 is a compile error if an agent takes a forbidden attribute as input.	P14, P22, P30
Human oversight	Low-confidence answers go straight to the customer.	`require human when confidence < 0.8` sends the call to a human review queue with approve and reject, recorded with the reviewer's name.	P09, P16
Auditability	Logs can be edited, and regulators ask "who decided what, and why".	Every agent call is written to a SHA-256 hash-chained audit log (model, input hash, confidence, tokens, status, policy, tool calls). `algo audit verify` detects any edit or deletion.	all
Cost control	Token bills grow silently, and one big model does every job.	`budget tokens`, a run-wide cap in `algo.config.json`, a SQLite response cache (identical calls cost zero tokens), and cheap models for scoring with stronger models only on the branch that needs them.	P16, P34
Speed and scale	Sequential calls are slow, and batch jobs need custom plumbing.	`par` and `par for` run on a thread pool. `algo batch` and `run_batch` process thousands of records under one global concurrency limit.	P11, P24
Simplicity	Prompt plumbing is hidden in application code that only programmers can read.	Four flow actions only. A domain expert can read a flow and see who does what, in what order, and under which rule.	P01–P08
Reuse	Every team rebuilds the same triage and review patterns.	120 ready domain packs in 20 sectors, plus `use "file.algo" as x` and `use pack ...` to compose larger systems.	all 50
Organisation-wide control	Each project sets its own rules, and the rules drift.	One org policy file applies to every program, and **the stricter rule wins**.	config
Access for everyone	A new language is hard to try.	Three modes on one engine: command line, Studio (graphical), and Prompt UI (plain English). A browser playground and a Python SDK (`import algolang`) are also available.	all 50
Teaching and research	Students cannot see how agents coordinate.	The agent graph, run table and offline mock backend show *what flows where* with no API key, which suits classes and CI. The named patterns give a shared vocabulary for it.	P01–P08

Where AlgoLang fits best

AlgoLang suits work where *a decision must be made, explained and recorded*: triage, screening, review, routing and summarising. These are the patterns used throughout this document: single, pipeline, fan-out, committee, router and `par for`. It is less suited to work that needs hard real-time guarantees or heavy numerical computation; see the limits below.

Real-time applications and solutions

"Real-time" here means *near real-time*: a case arrives, the agents run, and a decision or draft comes back while someone is still waiting, usually seconds. The speed is set mainly by the model service that answers each agent call. AlgoLang's job is to keep that fast path cheap, safe and recorded.

Twelve real-time scenarios

Application	Real-time need	AlgoLang solution	Programs
Card fraud alerts	Decide before the payment settles.	Router: a fast model scores the transaction. If the score is `>= 0.8`, a stronger agent writes the hold notice. Otherwise a fast agent writes an approval note. Low-confidence calls go to a human.	P16
ER triage	A nurse needs a priority at once.	Router on an urgency score. High urgency produces an immediate handoff for the clinician to confirm. This is decision support only, with a human gate.	P09, P11
Security operations	Alerts arrive faster than analysts can read them.	Pipeline enriches each alert, and a router scores a suspicious email. `algo batch` drains a burst of alerts in parallel.	P41, P42
IT outages	Customers are affected while the team investigates.	Incident triage as a pipeline. After the fix, a postmortem writer turns the timeline into a report.	P39, P40
Power grid	Abnormal readings must reach the right team quickly.	Router on an anomaly score, with a different agent path for each severity.	P43, P44
Maintenance	Act before a machine fails.	Router on sensor-log risk. High risk produces a work-order plan. Low risk is logged.	P34, P33
Logistics exceptions	A delayed shipment needs a recovery plan today.	Pipeline: understand the exception, then check tracking and stock cover, then write a recovery plan.	P35
Customer replies	Reviews and complaints should get a prompt, on-tone answer.	Router on sentiment and severity. Angry cases get a careful reply, simple cases a quick one.	P38, P50, P03
Citizen services	Grievances have service-level deadlines.	Router that sends each grievance to the right department, with a human gate for sensitive cases.	P26, P27
Compliance and legal	Trades and contracts need a check before they proceed.	Router for trades, and `par for` to review every clause of a contract in parallel.	P20, P28, P29
Crop and pest alerts	A pest sighting is time-critical for the farmer.	Router on pest severity, with advice by risk level.	P45, P46
Live fact-checking	Claims in a story need checking before publication.	`par for` checks each claim at once, and one agent summarises.	P47, P48

Solutions to common real-time problems

Problem	Solution in AlgoLang	Example
Too slow for the user	Run independent work together with `par`, or one agent over a list with `par for`.	`[a, b] = par { A(x), B(x) }`
Strong model is too slow or costly	Score with a cheap model, and call the strong model only on the branch that needs it.	P16 uses `haiku` to score and `sonnet` to write the hold notice.
Same question repeated	The response cache answers identical calls from SQLite at zero tokens.	`cache` policy, or `"cache": true` in the config.
A call fails or hangs	Policy gives retries and a timeout.	`retry 2`, `timeout` in the agent policy.
Cost runs away under a burst	Token budget per agent and a run-wide cap.	`budget tokens 5000`
Wrong answer reaches a person	Low confidence is escalated to the human review queue.	`require human when confidence < 0.8`
Unfair or unsafe decision	Forbidden attributes are blocked, and compile errors catch misuse before launch.	`forbid gender, caste, religion`
Need proof afterwards	Hash-chained audit log, checked on demand.	`algo audit verify`
A burst of thousands of events	Batch run with a global concurrency limit.	`algo batch programs/x.algo --input data.csv --workers 16`
Needs company data	Register your own Python functions as tools, which an agent can use only if it declares them.	`@al.tool("company.directory", ...)`

Worked example: a live fraud-alert service

The flow follows program 16 (score, then hold or approve). The Python below is a *sketch* that uses the documented SDK calls (`al.configure`, `al.domains.search`, `al.pack`, `prog.run`, `run_batch`, `al.review`, `al.audit`) shown in the repository README. Check the pack's flow argument names before use. The event source, `next_event()` and `notify()`, belong to your own system.

```python
import algolang as al

# 1. Org policy: applies to every program, the stricter rule wins
al.configure(min_human_below=0.80, forbid=["gender", "caste", "religion"])

# 2. Find a ready pack by describing the problem (as in the README)
best = al.domains.search("flag suspicious card transactions")[0]
prog = al.pack(best["id"])

# 3. Live path: one event at a time
while True:
   tx = next_event() # your payment stream
   r = prog.run(transaction=tx) # use the pack's own flow argument name
   notify(r.values, r.status) # your alerting system

# 4. Burst path: a queue that built up during an outage
results = prog.run_batch([{"transaction": t} for t in backlog], workers=16)

# 5. Human in the loop and proof
al.review.pending() # items below confidence 0.80
al.audit.verify("runs/<program>.audit.jsonl")
```

Why this design works. The cheap scorer keeps the common case fast. The strong agent is paid for only when the score is `>= 0.8`. A human sees the doubtful cases, and the audit log can show a regulator exactly what happened.

Honest limits

- **Near real-time, not hard real-time.** Latency is dominated by the model service and the network. Do not use AlgoLang where a deadline must be guaranteed in milliseconds, such as flight control or a machine safety interlock. Use it for the decision-support layer around such systems.
- **Decision support.** In healthcare, finance, legal and similar fields the agents advise, and a qualified person decides. The human review gate is there for this reason.
- **The sample outputs in this document are from the mock backend.** They show the flow, order and policy gates, not real answer quality or real latency. Measure speed and cost with `--live` on your own data.
- **Roadmap.** The repository lists `uncertain<T>` types, `eval` blocks, MCP tool bindings, `cost` and `energy` budgets and an LSP for v0.3, and a native runtime for v1.0. These are plans, not features of v0.2.

Start now

Install	`pip install git+https://github.com/drssnaiml1-cell/algolang`, then `algo doctor`
Try a pack	`algo domains "triage insurance claims"`, then `algo run pack:insurance.claims_triage --arg "claim=..."`
Start a project	`algo init acme`
Scale and govern	`algo batch`, `algo review`, `algo audit verify`, `algo report`
Repository	https://github.com/drssnaiml1-cell/algolang (Apache-2.0)
Website	https://www.algoprofessor.com

Ethical AI and Trustworthy AI with AlgoLang

By AlgoLang Innovator: Dr. S. Satyanarayana, Ph.D., PDF (AI)
AlgoProfessor AI R&D Solutions, Hyderabad

Why this matters

When an AI system helps decide a loan, a job interview, an admission, a triage priority or a welfare check, the question is no longer only "does it work?" but also "is it fair, can we explain it, and who is responsible?" Most tools leave these answers to separate documents and good intentions. AlgoLang puts them *inside the program*: fairness rules, human review, confidence, and a tamper-evident record are part of the language and the runtime. The idea is **ethics by design**, so that responsible behaviour is the default and not an extra step.

Two ideas, one goal

Ethical AI	AI that respects people: it is fair, avoids harm, protects privacy and dignity, and keeps humans responsible for important decisions.
Trustworthy AI	AI that people and organisations have good reason to rely on: it is lawful, ethical and robust, and it can show how it behaved. Trust is earned by evidence, not by claims.

Ethical AI is about *what we should do*. Trustworthy AI is about *being able to show that we did it*. AlgoLang supports both.

Trustworthy AI requirements and AlgoLang mechanisms

The seven requirements below are the ones set out by the European Commission's High-Level Expert Group on AI for trustworthy AI. They are a widely used checklist, and each can be matched to a feature of AlgoLang v0.2.

Requirement	Risk it addresses	AlgoLang mechanism	Seen in
Human agency and oversight	Machines decide and nobody can step in.	`require human when confidence < 0.8` sends doubtful calls to a human review queue. Reviewers approve or reject, and the reviewer's name is recorded. Decision-support wording keeps the person as the decider.	P09, P12, P16
Technical robustness and safety	Wrong tools, runaway cost, failed calls.	Compile-time checks (E101, E104, E115), `retry`, `timeout`, token budgets, and a run-wide token cap in the organisation config.	P41, P42
Privacy and data governance	Agents see or use data they should not.	Agents may use only the tools they declare. Allowed tools are controlled by the organisation. `forbid` stops protected attributes from being used. Company data is reached only through registered tools.	P14, P30
Transparency	No one can see how a result was reached.	Every call returns a value with its **confidence** and provenance. The Studio agent graph and run table show each step. The audit log records the model, input hash, tokens, status and policy.	all
Diversity, non-discrimination and fairness	Decisions that depend on gender, caste, religion or similar attributes.	`forbid gender, caste, religion.` An agent that takes a forbidden attribute as input is a compile error (E108). Committees score on separate criteria and a chair decides.	P14, P22, P30, P36

Requirement	Risk it addresses	AlgoLang mechanism	Seen in
Societal and environmental well-being	Wasteful use and harm to groups of people.	Cheap models for easy steps, response cache at zero tokens, and budgets reduce waste. The roadmap lists `cost` and `energy` budgets for v0.3.	P16, P34
Accountability	No one can prove who did what.	A SHA-256 hash-chained audit log, checked with `algo audit verify`. Usage and cost reports with `algo report`. A named reviewer on every human decision.	all

Ethical principles for AlgoLang programs

Fairness	Do not use protected attributes to decide. Declare them with `forbid`, and let the compiler enforce it.
Human dignity	A person, not the machine, makes final decisions that affect rights or welfare. Use `require human` and decision-support goals.
Transparency	Make the reason visible: use scores, keep the agent graph readable, and keep confidence and provenance with every result.
Accountability	Keep the audit chain, name the reviewers, and report usage. Responsibility stays with people and the organisation.
Privacy	Give agents the least data and the fewest tools they need. Nothing is available by default.
Safety and non-harm	Stop and escalate when unsure. Limit tokens, retries and tools. Do not use AI as the only safeguard in a safety-critical system.
Beneficence	Use AI to help: faster service, fewer errors, wider access, and clear explanations for people affected.

The trust cycle in AlgoLang

Stage	Tool	What it gives
1. Design	`algo check`	The compiler rejects unsafe programs before anything runs: non-agent calls, undeclared or disallowed tools, forbidden inputs.
2. Govern	Policy and `algo.config.json`	One organisation file applies to every program. The stricter rule wins.
3. Run	Runtime	Budgets, retries, timeouts and blocking of forbidden attributes are enforced on every call.
4. Review	`algo review`	Doubtful calls wait for a human, who approves or rejects them by name.
5. Prove	`algo audit verify`	Any edit or deletion in the audit record is detected.
6. Improve	`*.algotest.json`, `algo report`	Behaviour tests and usage and cost reports show how the system behaves over time.

Worked example: a fair credit committee (program 14)

Program 14 asks three analysts to score a loan application in parallel, and a chair recommends approve, refer or decline. Its policy is where the ethics is written down:

```
policy {
    budget tokens 4000                      // cost is capped
    retry 1                                 // a failed call is retried once
    forbid gender, caste, religion          // protected attributes must not be used
    require human when confidence < 0.8     // doubtful calls go to a person
}
```

- **Fairness.** If an agent were written to take `gender` as input, the program would not compile (E108).
- **Explainability.** The chair sees three separate scores (bureau, debt-to-income, collateral), and all three are emitted with the decision.
- **Human oversight.** A low-confidence call is queued for a named reviewer, and the chair's output is a *recommendation*.
- **Accountability.** Each agent call is recorded in the audit chain, so the decision can be reconstructed and checked later.

Higher-risk sectors in this document

Decisions about credit, employment, education, essential public services, health and insurance affect people's lives, and laws and regulations in many countries treat such uses with extra care. The programs below are the ones where the controls above matter most.

Sector	Programs	Controls to use
Banking	P14, P15, P16	`forbid` for protected attributes, a human gate on low confidence, and a record of every score behind a decision.
Hiring and HR	P30, P31, P32	Fair screening committee that forbids protected attributes. Candidates are never rejected by the machine alone.
Education	P22, P23	Admissions panel with separate scores and a chair. Decisions are open to review.
Healthcare and pharma	P09, P10, P11, P12, P13	Decision support only. A clinician or pharmacist confirms, and low confidence goes to a human.
Government	P26, P27	Grievance routing and welfare pre-checks that a human officer confirms. Sensitive cases are escalated.
Insurance and legal	P17, P18, P28, P29	Claims, renewals and contracts reviewed by people. Clause-level results are visible.

Responsible-use checklist before deployment

- [ ] The purpose is clear and a person is named as responsible for each decision.
- [ ] Protected attributes are listed in `forbid` and in the organisation config, and `algo check` passes.
- [ ] Each agent declares only the tools it needs, and the organisation's allowed tools are set.
- [ ] A human review threshold is set (for example 0.8), and reviewers are trained and named.
- [ ] Token budgets, retries and a run-wide cap are set.
- [ ] Behaviour tests (`*.algotest.json`) cover normal, edge and unfair-looking cases, with a live run on real data.
- [ ] `algo audit verify` is part of regular operations, and `algo report` is reviewed.
- [ ] People affected are told that AI is used, and they have a way to ask for human review.
- [ ] Local laws and sector rules have been checked by a qualified person.

Honest limits: what AlgoLang does not guarantee

- **Trust is not automatic.** AlgoLang gives the controls. Ethical results still depend on good goals, good data and good people.
- **Forbidding an attribute is not the whole of fairness.** Other information, such as a postcode or a school name, can stand in for a protected attribute. Test outcomes across groups, not only the inputs.
- **Confidence is a signal, not proof.** A high confidence does not mean the answer is right. The human gate and tests are still needed.
- **An audit chain shows what happened, not that it was right.** It detects tampering and supports accountability. It does not judge the decision.
- **Mock outputs in this document are not evidence of quality.** Fairness and accuracy must be measured with live models on your own data.
- **Final responsibility stays with people.** In health, finance, law and public services, AlgoLang is decision support, and a qualified person decides.

Vision

AlgoLang aims for a future in which AI agents are not black boxes but *accountable colleagues*: they work inside clear rules, show their work, ask for help when unsure, and leave a record that anyone can check. Building such systems is a shared responsibility of language designers, developers, organisations and regulators.

By AlgoLang Innovator
Dr. S. Satyanarayana, Ph.D., PDF (AI)
AlgoProfessor AI R&D Solutions, Hyderabad

Index of programs

No.	Program	Sector	Pattern	Input	Output type
1	Hello Greeter	Basics	single	`name`	`text`
2	Article Summariser	Basics	single	`article`	`text`
3	Sentiment Scorer	Basics	single	`review`	`score`
4	Translator	Basics	single	`passage, language`	`text`
5	Study Planner	Basics	pipeline	`topic, days`	`text`
6	Blog Post Pipeline	Basics	pipeline	`topic`	`text`
7	Pros and Cons Judge	Basics	fan-out	`decision`	`text`
8	Idea Ranker	Basics	par for	`ideas`	`text, list`
9	ER Triage Router	Healthcare	router	`complaint`	`score, text`
10	Discharge Summary	Healthcare	pipeline	`notes`	`text`
11	Care Team Review	Healthcare	fan-out	`case`	`text`
12	Drug Interaction Check	Pharma	fan-out	`regimen`	`text`
13	Adverse Event Router	Pharma	router	`report`	`score, text`
14	Credit Committee	Banking	committee	`application`	`text, score, score, score`
15	KYC Onboarding	Banking	pipeline	`customer`	`text`
16	Fraud Alert Router	Banking	router	`transaction`	`score, text`
17	Claims Triage	Insurance	router	`claim`	`score, text`
18	Renewal Advisor	Insurance	pipeline	`holder_profile`	`text`
19	Earnings Brief	Capital Markets	fan-out	`transcript`	`text`
20	Trade Compliance Check	Capital Markets	router	`trade`	`score, text`
21	Quiz Generator	Education	pipeline	`lesson`	`text`
22	Admissions Panel	Education	committee	`applicant`	`text, score, score, score`
23	CO-PO Mapper	Education	par for	`outcomes`	`text, list`
24	Literature Review	Research	par for	`papers`	`text, list`
25	Hypothesis Critic	Research	fan-out	`hypothesis`	`text`
26	Grievance Router	Government	router	`grievance`	`score, text`
27	Welfare Pre-check	Government	pipeline	`household`	`text`
28	NDA Triage	Legal	router	`nda`	`score, text`
29	Clause Reviewer	Legal	par for	`clauses`	`text, list`
30	Fair CV Screening	HR	committee	`cv, role_title`	`text, score, score, score`
31	Interview Builder	HR	pipeline	`job_title`	`text`
32	Exit Interview Insights	HR	fan-out	`transcript`	`text`
33	Defect Root Cause	Manufacturing	pipeline	`defect_report`	`text`
34	Maintenance Planner	Manufacturing	router	`sensor_log`	`score, text`
35	Shipment Exception	Supply Chain	pipeline	`exception`	`text`
36	Supplier Risk Panel	Supply Chain	committee	`supplier`	`text, score, score, score`
37	Product Copy Studio	Retail	fan-out	`product`	`text`
38	Complaint Reply	Retail	router	`complaint`	`score, text`
39	Incident Triage	IT Operations	pipeline	`incident`	`text`
40	Postmortem Writer	IT Operations	pipeline	`timeline`	`text`
41	SOC Alert Enrichment	Cybersecurity	pipeline	`alert`	`text`
42	Phishing Email Check	Cybersecurity	router	`email`	`score, text`
43	Grid Anomaly Router	Energy	router	`readings`	`score, text`
44	Solar Site Assessment	Energy	fan-out	`site`	`text`
45	Crop Advisory	Agriculture	pipeline	`field_report`	`text`
46	Pest Alert Triage	Agriculture	router	`observation`	`score, text`
47	Claim Fact-Check	Media	par for	`claims`	`text, list`
48	Headline Variants	Media	fan-out	`story`	`text`
49	Listing Writer	Real Estate	pipeline	`property_details`	`text`
50	Hotel Review Responder	Hospitality	router	`guest_review`	`score, text`

1 Hello Greeter

Basics single P01_hello_greeter.algo

The smallest AlgoLang program: one agent called by one flow.

Input

name text Satya

Output type: text

AlgoLang code

```
// Hello Greeter: one agent, one flow.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent Greeter(name: text) -> text {
   role "Friendly assistant"
   goal "Write a warm one-line greeting for {name}"
   model fast
}

flow main(name: text) {
   answer = Greeter(name)
   emit answer
}
```

Output 1 – Command line

Terminal

```
$ algo run P01_hello_greeter.algo --arg "name=Satya"

▶ flow main  run=f1c4abf5  backend=mock
    name = 'Satya'
  → Greeter            conf=0.66  «Greeter #6661» Write a warm one-line greeting for Satya
■ emitted:
    • «Greeter #6661» Write a warm one-line greeting for Satya
■ 1 agent calls · ~80 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P01_hello_greeter.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 1 agent calls · 80 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	Greeter	haiku-5-5	0.66	80	«Greeter #6661» Write a warm one-line greeting for Satya

Result: [«Greeter #6661» Write a warm one-line greeting for Satya]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "write a friendly greeting for a new colleague"

AlgoLang: matched **Claims triage** (insurance.claims_triage), Insurance, pattern pipe, match score 0.22. Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: Intake, Coverage, Complexity, Routing.

General-purpose task: the library has no dedicated pack, so the closest pack is shown. Other matches offered: healthcare.discharge_summary, retail.customer_service, pharma.literature_watch.

#	Agent	Model	Conf.	Tok.	Output
1	Intake	sonnet-5-5	0.76	98	«Intake #8231» Extract policy, incident and loss details from wri…
2	Coverage	sonnet-5-5	0.89	117	«Coverage #6561» Check coverage for «Intake #8231» Extract policy…
3	Complexity	sonnet-5-5	0.73	106	0.80 **human**
4	Routing	sonnet-5-5	0.71	109	«Routing #1854» Recommend a handling route for «Coverage #6561» C…

Final answer: `[«Routing #1854» Recommend a handling route for «Coverage #6561» Check coverage for «In…, 0.80]` (4 agent calls · 430 tokens · 0 retries · 1 escalated · 0 blocked)

2 Article Summariser

Basics single P02_article_summariser.algo

One agent that condenses text.

Input

`article` `text` AlgoLang replaces functions with governed agents. Each agent has a role, goal, tools and a policy.

Output type: `text`

AlgoLang code

```
// Article Summariser: one agent, one flow.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent Summariser(article: text) -> text {
   role "Editor"
   goal "Summarise {article} in three short bullet points"
   model fast
}

flow main(article: text) {
   answer = Summariser(article)
   emit answer
}
```

Output 1 – Command line

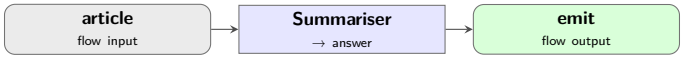
Terminal

```
$ algo run P02_article_summariser.algo --arg "article=AlgoLang replaces functions with governed agents. Each agent has a role, goal,
↪  tools and a policy."

▶ flow main  run=fb482302  backend=mock
    article = 'AlgoLang replaces functions with governed agents. Each agent has a role, goal, tools and a policy.'
  → Summariser          conf=0.85  «Summariser #1689» Summarise AlgoLang replaces functions with governed a
■ emitted:
    • «Summariser #1689» Summarise AlgoLang replaces functions with governed agents. Each agent…
■ 1 agent calls · ~102 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P02_article_summariser.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 1 agent calls · 102 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	Summariser	haiku-5-5	0.85	102	«Summariser #1689» Summarise AlgoLang replaces functions with gov…

Result: [«Summariser #1689» Summarise AlgoLang replaces functions with governed agents. Each agent…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "summarise a long article into key points"

AlgoLang: matched **Earnings call summariser** (`markets.earnings_call`), Capital Markets & Investment, pattern fan, match score 0.23. Governance: human review below confidence 0.70. Agents: Sections, Analyst, Sentiment, Brief.

General-purpose task: the library has no dedicated pack, so the closest pack is shown. Other matches offered: `realestate.lease_abstraction`, `research.literature_review`, `cyber.soc_triage`.

#	Agent	Model	Conf.	Tok.	Output
1	Sections	haiku-5-5	0.90	100	[item 1 for: Split summarise a long article into key points into …
2	Analyst	sonnet-5-5	0.91	107	«Analyst #9368» Summarise key numbers and guidance in item 1 for:…
3	Analyst	sonnet-5-5	0.71	107	«Analyst #3418» Summarise key numbers and guidance in item 3 for:…
4	Analyst	sonnet-5-5	0.82	107	«Analyst #2900» Summarise key numbers and guidance in item 2 for:…
5	Sentiment	sonnet-5-5	0.63	154	0.61 **human**
6	Brief	sonnet-5-5	0.63	155	«Brief #5305» Write an earnings brief from «Analyst #9368» Summar…

Final answer: [«Brief #5305» Write an earnings brief from «Analyst #9368» Summarise key numbers and…, 0.61] (6 agent calls · 730 tokens · 0 retries · 1 escalated · 0 blocked)

3 Sentiment Scorer

Basics single P03_sentiment_scorer.algo

An agent whose return type is score, a value between 0 and 1.

Input

review	text	The delivery was late but the support team fixed everything quickly.

Output type: `score`

AlgoLang code

```
// Sentiment Scorer: one agent, one flow.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent SentimentScorer(review: text) -> score {
   role "Review analyst"
   goal "Rate the sentiment of {review} from 0 (negative) to 1 (positive)"
   model fast
}

flow main(review: text) {
   answer = SentimentScorer(review)
   emit answer
}
```

Output 1 – Command line

Terminal

```
$ algo run P03_sentiment_scorer.algo --arg "review=The delivery was late but the support team fixed everything quickly."

▶ flow main  run=2d16399c  backend=mock
    review = 'The delivery was late but the support team fixed everything quickly.'
  → SentimentScorer    conf=0.93  0.79
```

```
■ emitted:
    • 0.79
■ 1 agent calls · ~100 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P03_sentiment_scorer.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 1 agent calls · 100 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	SentimentScorer	haiku-5-5	0.93	100	0.79

Result: [0.79]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "turn customer reviews into sentiment insights"

AlgoLang: matched **Customer review insights** (retail.review_insights), Retail & E-commerce, pattern fan, match score 0.70. Governance: human review below confidence 0.70. Agents: Splitter, Analyst, Satisfaction, Report.

General-purpose task: the library has no dedicated pack, so the closest pack is shown. Other matches offered: hospitality.guest_reviews, banking.kyc_review, retail.customer_service.

#	Agent	Model	Conf.	Tok.	Output
1	Splitter	haiku-5-5	0.65	96	[item 1 for: Split turn customer reviews into sentiment insights …
2	Analyst	sonnet-5-5	0.75	107	«Analyst #4404» Extract product issues and praise from item 1 for…
3	Analyst	sonnet-5-5	0.87	107	«Analyst #3773» Extract product issues and praise from item 2 for…
4	Analyst	sonnet-5-5	0.66	107	«Analyst #8785» Extract product issues and praise from item 3 for…
5	Satisfaction	sonnet-5-5	0.72	154	0.71
6	Report	sonnet-5-5	0.77	158	«Report #5639» Write a product insights report from «Analyst #440…

Final answer: [«Report #5639» Write a product insights report from «Analyst #4404» Extract product i…, 0.71] (6 agent calls · 729 tokens · 0 retries · 0 escalated · 0 blocked)

4 Translator

Basics single P04_translator.algo

An agent with two inputs.

Input

passage text Welcome to our workshop on agent-native programming.
language text Hindi
Output type: text

AlgoLang code

```
// Translator: one agent, one flow.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent Translator(passage: text, language: text) -> text {
   role "Professional translator"
   goal "Translate {passage} into {language}, keeping the tone and meaning"
   model fast
}

flow main(passage: text, language: text) {
   answer = Translator(passage, language)
   emit answer
}
```

Output 1 – Command line

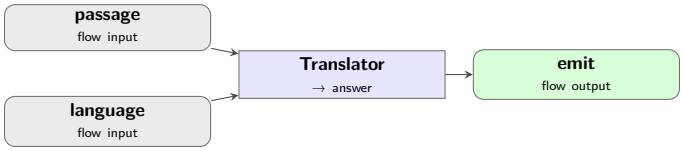
Terminal

```
$ algo run P04_translator.algo --arg "passage=Welcome to our workshop on agent-native programming." --arg "language=Hindi"

▶ flow main  run=9baba2d5  backend=mock
    passage = 'Welcome to our workshop on agent-native programming.'
    language = 'Hindi'
  → Translator         conf=0.95  «Translator #5568» Translate Welcome to our workshop on agent-native pro
■ emitted:
    • «Translator #5568» Translate Welcome to our workshop on agent-native programming. into Hi…
■ 1 agent calls · ~97 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P04_translator.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 1 agent calls · 97 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	Translator	haiku-5-5	0.95	97	«Translator #5568» Translate Welcome to our workshop on agent-nat…

Result: [«Translator #5568» Translate Welcome to our workshop on agent-native programming. into Hi…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "translate a passage into another language"

AlgoLang: matched **Crop advisory** (`agriculture.crop_advisory`), Agriculture, pattern `pipe`, match score 0.16. Governance: human review below confidence 0.70. Agents: Advisor, Translator, Safety, Final.

General-purpose task: the library has no dedicated pack, so the closest pack is shown. Other matches offered: `hr.performance_review`, `insurance.policy_compare`, `education.essay_feedback`.

#	Agent	Model	Conf.	Tok.	Output
1	`Advisor`	sonnet-5-5	0.86	97	«Advisor #0863» Give weather-aware sowing and care advice for tra…
2	`Translator`	sonnet-5-5	0.86	107	«Translator #7066» Translate «Advisor #0863» Give weather-aware s…
3	`Safety`	sonnet-5-5	0.66	109	0.81 **human**
4	`Final`	sonnet-5-5	0.88	106	«Final #1027» Finalise the advisory «Translator #7066» Translate …

Final answer: `[«Final #1027» Finalise the advisory «Translator #7066» Translate «Advisor #0863» Giv…, 0.81]` (4 agent calls · 419 tokens · 0 retries · 1 escalated · 0 blocked)

5 Study Planner

Basics pipeline P05_study_planner.algo

Each agent's output feeds the next one.

Input

topic	text	Machine learning
days	text	14

Output type: text

AlgoLang code

```
// Study Planner: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent Syllabus(topic: text, days: text) -> text {
   role "Curriculum designer"
   goal "Break {topic} into key subtopics for a {days}-day plan"
   model smart
}

agent Scheduler(syllabus: text) -> text {
   role "Study coach"
   goal "Spread {syllabus} across the days with a clear goal per day"
   model smart
}

agent Reviser(schedule: text) -> text {
   role "Study coach"
   goal "Check {schedule} for overload and add revision days"
   model fast
}

flow main(topic: text, days: text) {
   syllabus = Syllabus(topic, days)
   schedule = Scheduler(syllabus)
   plan = Reviser(schedule)
   emit plan
}
```

Output 1 – Command line

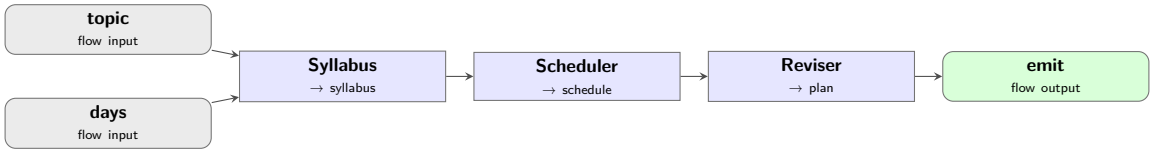
Terminal

```
$ algo run P05_study_planner.algo --arg "topic=Machine learning" --arg "days=14"

▶ flow main  run=8a5ec4d8  backend=mock
    topic = 'Machine learning'
    days = '14'
  → Syllabus           conf=0.88  «Syllabus #8700» Break Machine learning into key subtopics for a 14-day
  → Scheduler          conf=0.63  «Scheduler #4721» Spread «Syllabus #8700» Break Machine learning into ke
  → Reviser            conf=0.69  «Reviser #6323» Check «Scheduler #4721» Spread «Syllabus #8700» Break Ma
■ emitted:
    • «Reviser #6323» Check «Scheduler #4721» Spread «Syllabus #8700» Break Machine learning…
■ 3 agent calls · ~286 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P05_study_planner.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 286 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	Syllabus	sonnet-5-5	0.88	85	«Syllabus #8700» Break Machine learning into key subtopics for a …
2	Scheduler	sonnet-5-5	0.63	100	«Scheduler #4721» Spread «Syllabus #8700» Break Machine learning …
3	Reviser	haiku-5-5	0.69	101	«Reviser #6323» Check «Scheduler #4721» Spread «Syllabus #8700» B…

Result: `[«Reviser #6323» Check «Scheduler #4721» Spread «Syllabus #8700» Break Machine learning…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "explain a topic with examples and a study plan for a student"

AlgoLang: matched **AI tutor** (`education.ai_tutor`), Education, pattern `pipe`, match score 0.80. Governance: human review below confidence 0.70. Agents: Explainer, Examples, Accuracy, Lesson.

General-purpose task: the library has no dedicated pack, so the closest pack is shown. Other matches offered: `education.student_support`, `education.essay_feedback`, `education.admissions`.

#	Agent	Model	Conf.	Tok.	Output
1	Explainer	sonnet-5-5	0.94	100	«Explainer #8716» Explain explain a topic with examples and a stu…
2	Examples	sonnet-5-5	0.90	102	«Examples #7117» Add two worked examples to «Explainer #8716» Exp…
3	Accuracy	sonnet-5-5	0.77	102	0.93
4	Lesson	sonnet-5-5	0.73	105	«Lesson #9856» Finalise the lesson «Examples #7117» Add two worke…

Final answer: `[«Lesson #9856» Finalise the lesson «Examples #7117» Add two worked examples to «Expla…, 0.93]` (4 agent calls · 409 tokens · 0 retries · 0 escalated · 0 blocked)

6 Blog Post Pipeline

Basics pipeline `P06_blog_post_pipeline.algo`

Four agents in sequence: outline, draft, edit, headline.

Input

`topic` `text` Why agents beat functions for enterprise AI

Output type: `text`

AlgoLang code

```
// Blog Post Pipeline: a pipeline of 4 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent Outliner(topic: text) -> text {
   role "Content strategist"
   goal "Create a five-section outline for a post about {topic}"
   model smart
}

agent Drafter(outline: text) -> text {
   role "Writer"
   goal "Write a first draft that follows {outline}"
   model smart
}

agent Editor(draft: text) -> text {
   role "Copy editor"
   goal "Tighten {draft} and fix grammar"
   model fast
}

agent Headliner(edited: text) -> text {
   role "Headline writer"
   goal "Write one strong headline for {edited}"
```

```
   model fast
}

flow main(topic: text) {
   outline = Outliner(topic)
   draft = Drafter(outline)
   edited = Editor(draft)
   headline = Headliner(edited)
   emit headline
}
```

Output 1 – Command line

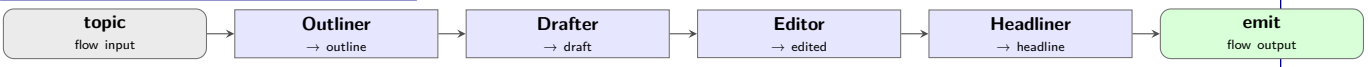
Terminal

```
$ algo run P06_blog_post_pipeline.algo --arg "topic=Why agents beat functions for enterprise AI"

▶ flow main  run=2cf69c2d  backend=mock
    topic = 'Why agents beat functions for enterprise AI'
  → Outliner           conf=0.90  «Outliner #4865» Create a five-section outline for a post about Why agen
  → Drafter            conf=0.78  «Drafter #6067» Write a first draft that follows «Outliner #4865» Create
  → Editor             conf=0.84  «Editor #2677» Tighten «Drafter #6067» Write a first draft that follows
  → Headliner          conf=0.62  «Headliner #5249» Write one strong headline for «Editor #2677» Tighten «
■ emitted:
    • «Headliner #5249» Write one strong headline for «Editor #2677» Tighten «Drafter #6067» W…
■ 4 agent calls · ~386 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P06_blog_post_pipeline.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 386 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	Outliner	sonnet-5-5	0.90	93	«Outliner #4865» Create a five-section outline for a post about W…
2	Drafter	sonnet-5-5	0.78	98	«Drafter #6067» Write a first draft that follows «Outliner #4865»…
3	Editor	haiku-5-5	0.84	96	«Editor #2677» Tighten «Drafter #6067» Write a first draft that f…
4	Headliner	haiku-5-5	0.62	99	«Headliner #5249» Write one strong headline for «Editor #2677» Ti…

Result: [«Headliner #5249» Write one strong headline for «Editor #2677» Tighten «Drafter #6067» W…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "write marketing content for a campaign"

AlgoLang: matched **Campaign brief** (media.campaign_brief), Media & Marketing, pattern pipe, match score 0.53. Governance: human review below confidence 0.70. Agents: Strategist, Creative, Fit, Brief.

General-purpose task: the library has no dedicated pack, so the closest pack is shown. Other matches offered: media.content_moderation, media.seo_audit, agriculture.market_prices.

#	Agent	Model	Conf.	Tok.	Output
1	Strategist	sonnet-5-5	0.74	93	«Strategist #4793» Define audience and message for write marketin…
2	Creative	sonnet-5-5	0.84	106	«Creative #7554» Write three campaign concepts from «Strategist #…
3	Fit	sonnet-5-5	0.75	102	0.60
4	Brief	sonnet-5-5	0.67	106	«Brief #9895» Write a campaign brief from «Creative #7554» Write …

Final answer: [«Brief #9895» Write a campaign brief from «Creative #7554» Write three campaign conc…, 0.60] (4 agent calls · 407 tokens · 0 retries · 0 escalated · 0 blocked)

7 Pros and Cons Judge

Basics fan-out P07_pros_and_cons_judge.algo

Two analysts run in parallel; a judge weighs both.

Input

`decision` `text` Move our reporting to a cloud data warehouse

Output type: `text`

AlgoLang code

```
// Pros and Cons Judge: workers run in parallel, one agent merges.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent ProsAnalyst(decision: text) -> text {
   role "Optimist analyst"
   goal "List the strongest benefits of {decision}"
   model smart
}

agent ConsAnalyst(decision: text) -> text {
   role "Sceptical analyst"
   goal "List the strongest risks of {decision}"
   model smart
}

agent Judge(pros: text, cons: text) -> text {
   role "Neutral chair"
   goal "Weigh {pros} against {cons} and give a balanced recommendation"
   model smart
}

flow main(decision: text) {
   [pros, cons] = par {
      ProsAnalyst(decision),
      ConsAnalyst(decision)
   }
   verdict = Judge(pros, cons)
   emit verdict
}
```

Output 1 – Command line

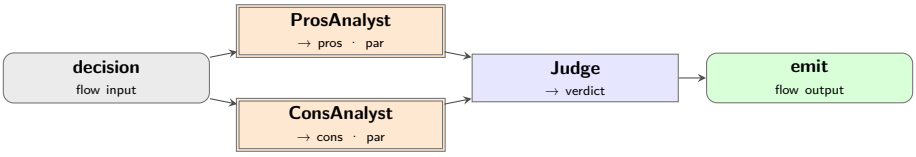
Terminal

```
$ algo run P07_pros_and_cons_judge.algo --arg "decision=Move our reporting to a cloud data warehouse"

▶ flow main  run=cefb27be  backend=mock
    decision = 'Move our reporting to a cloud data warehouse'
  → ProsAnalyst        conf=0.85  «ProsAnalyst #6732» List the strongest benefits of Move our reporting to
  → ConsAnalyst        conf=0.64  «ConsAnalyst #9354» List the strongest risks of Move our reporting to a
  → Judge              conf=0.91  «Judge #5716» Weigh «ProsAnalyst #6732» List the strongest benefits of M
■ emitted:
    • «Judge #5716» Weigh «ProsAnalyst #6732» List the strongest benefits of Move our repo…
■ 3 agent calls · ~304 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P07_pros_and_cons_judge.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 304 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	ProsAnalyst	sonnet-5-5	0.85	89	«ProsAnalyst #6732» List the strongest benefits of Move our repor…
2	ConsAnalyst	sonnet-5-5	0.64	88	«ConsAnalyst #9354» List the strongest risks of Move our reportin…
3	Judge	sonnet-5-5	0.91	127	«Judge #5716» Weigh «ProsAnalyst #6732» List the strongest benefi…

Result: [«Judge #5716» Weigh «ProsAnalyst #6732» List the strongest benefits of Move our repo…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "multi-perspective impact brief on a decision"

AlgoLang: matched **Policy impact brief** (government.policy_brief), Government & Public Services, pattern committee, match score 0.52. Governance: human review below confidence 0.80. Agents: Economist, SocialAnalyst, LegalAnalyst, Panel, Brief.

General-purpose task: the library has no dedicated pack, so the closest pack is shown. Other matches offered: markets.portfolio_risk, healthcare.drug_interaction, media.content_moderation.

#	Agent	Model	Conf.	Tok.	Output
1	Economist	sonnet-5-5	0.91	95	«Economist #0368» Assess economic impact of multi-perspective imp…
2	LegalAnalyst	sonnet-5-5	0.75	99	«LegalAnalyst #0159» Assess legal and implementation issues in mu…
3	SocialAnalyst	sonnet-5-5	0.63	97	«SocialAnalyst #7916» Assess social and equity impact of multi-pe…
4	Panel	sonnet-5-5	0.91	154	0.57
5	Brief	sonnet-5-5	0.81	155	«Brief #1913» Write a policy brief from «Economist #0368» Assess …

Final answer: [«Brief #1913» Write a policy brief from «Economist #0368» Assess economic impact of …, 0.57] (5 agent calls · 600 tokens · 0 retries · 0 escalated · 0 blocked)

8 Idea Ranker

Basics par for P08_idea_ranker.algo

par for evaluates every idea in parallel.

Input

ideas list AI tutor, smart inventory alerts, voice invoicing

Output type: text, list

AlgoLang code

```
// Idea Ranker: par for fans one agent out over a list.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent Evaluator(item: text) -> text {
   role "Product manager"
   goal "Rate the value and effort of the idea: {item}"
   model fast
}

agent Ranker(results: list) -> text {
   role "Portfolio lead"
   goal "Rank these evaluations from best to worst: {results}"
   model smart
}

flow main(ideas: list) {
   results = par for it in ideas { Evaluator(it) }
   report = Ranker(results)
   emit report, results
}
```

Output 1 – Command line

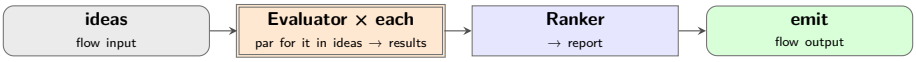
Terminal

```
$ algo run P08_idea_ranker.algo --arg "ideas=AI tutor, smart inventory alerts, voice invoicing"

▶ flow main  run=f15bb52f  backend=mock
    ideas = 'AI tutor, smart inventory alerts, voice invoicing'
  → Evaluator          conf=0.83  «Evaluator #4122» Rate the value and effort of the idea: AI tutor, smart
  → Ranker             conf=0.90  «Ranker #1816» Rank these evaluations from best to worst: «Evaluator #41
■ emitted:
    • «Ranker #1816» Rank these evaluations from best to worst: «Evaluator #4122» Rate the …
    • «Evaluator #4122» Rate the value and effort of the idea: AI tutor, smart inventory alert…
■ 2 agent calls · ~194 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P08_idea_ranker.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 2 agent calls · 194 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	Evaluator	haiku-5-5	0.83	92	«Evaluator #4122» Rate the value and effort of the idea: AI tutor…
2	Ranker	sonnet-5-5	0.90	102	«Ranker #1816» Rank these evaluations from best to worst: «Evalua…

Result: [«Ranker #1816» Rank these evaluations from best to worst: «Evaluator #4122» Rate the …, [«Evaluator #4122» Rate the value and effort of the idea: AI…

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "rank a list of product ideas"

AlgoLang: matched **Product description writer** (retail.product_description), Retail & E-commerce, pattern pipe, match score 0.53. Governance: human review below confidence 0.70. Agents: Features, Writer, Accuracy, Final.

General-purpose task: the library has no dedicated pack, so the closest pack is shown. Other matches offered: retail.review_insights, retail.pricing_brief, retail.catalog_quality.

#	Agent	Model	Conf.	Tok.	Output
1	Features	sonnet-5-5	0.94	92	«Features #0994» List features and benefits of rank a list of pro…
2	Writer	sonnet-5-5	0.70	107	«Writer #4262» Write an engaging product description from «Featur…
3	Accuracy	sonnet-5-5	0.81	108	0.61
4	Final	sonnet-5-5	0.80	109	«Final #2768» Finalise the description «Writer #4262» Write an en…

Final answer: [«Final #2768» Finalise the description «Writer #4262» Write an engaging product desc…, 0.61] (4 agent calls · 416 tokens · 0 retries · 0 escalated · 0 blocked)

9 ER Triage Router

Healthcare router P09_er_triage_router.algo

Decision support: high urgency goes to a clinician at once; a human confirms.

Input

`complaint`	`text`	58 year old with chest pain spreading to the left arm, sweating for 20 minutes

Output type: `score, text`

AlgoLang code

```
// ER Triage Router: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent UrgencyScorer(complaint: text) -> score {
   role "Triage nurse assistant"
   goal "Score the urgency of {complaint} from 0 to 1 (decision support only)"
   model fast
}

agent EmergencyPath(complaint: text) -> text {
   role "Triage assistant"
   goal "Draft an immediate-attention handoff for {complaint} for the clinician to confirm"
   model smart
}

agent RoutineQueue(complaint: text) -> text {
   role "Triage assistant"
   goal "Draft a routine queue note for {complaint} with advice on warning signs"
   model fast
}

flow main(complaint: text) {
   urgency = UrgencyScorer(complaint)

   if urgency >= 0.7 {
      handoff = EmergencyPath(complaint)
      emit urgency, handoff
   } else {
      note = RoutineQueue(complaint)
      emit urgency, note
   }
}
```

Output 1 – Command line

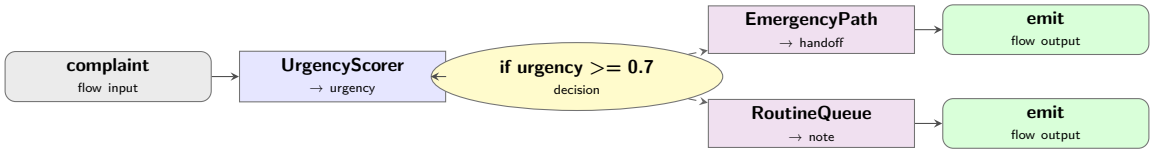
Terminal

```
$ algo run P09_er_triage_router.algo --arg "complaint=58 year old with chest pain spreading to the left arm, sweating for 20 minutes"

▶ flow main  run=684c6762  backend=mock
    complaint = '58 year old with chest pain spreading to the left arm, sweating for 20 minutes'
  → UrgencyScorer      conf=0.88  0.74
  → EmergencyPath      conf=0.93  «EmergencyPath #4761» Draft an immediate-attention handoff for 58 year o
■ emitted:
    • 0.74
    • «EmergencyPath #4761» Draft an immediate-attention handoff for 58 year old with chest pain s…
■ 2 agent calls · ~212 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P09_er_triage_router.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 212 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`UrgencyScorer`	haiku-5-5	0.88	105	0.74
2	`EmergencyPath`	sonnet-5-5	0.93	107	«EmergencyPath #4761» Draft an immediate-attention handoff for 58…

Result: `[0.74, «EmergencyPath #4761» Draft an immediate-attention handoff for 58 year old with chest pain s…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "triage patient notes by urgency for a doctor"

AlgoLang: matched **Clinical note triage** (`healthcare.clinical_triage`), Healthcare, pattern pipe, match score 0.78. Governance: human review below confidence 0.90, forbids caste, religion. Agents: Extractor, Summariser, UrgencyScorer, Referral.

Matched a pack in the same sector. Other matches offered: `healthcare.patient_feedback`, `healthcare.appointment_router`, `pharma.trial_matching`.

#	Agent	Model	Conf.	Tok.	Output
1	`Extractor`	sonnet-5-5	0.93	100	«Extractor #0485» Extract symptoms, history and medications from …
2	`Summariser`	sonnet-5-5	0.78	119	«Summariser #4086» Write a structured clinical summary of «Extrac…
3	`UrgencyScorer`	sonnet-5-5	0.74	110	0.72 **human**
4	`Referral`	sonnet-5-5	0.89	114	«Referral #5726» Draft a referral note for the doctor from «Summa…

Final answer: `[«Referral #5726» Draft a referral note for the doctor from «Summariser #4086» Write a s…, 0.72]` (4 agent calls · 443 tokens · 0 retries · 1 escalated · 0 blocked)

10 Discharge Summary

Healthcare pipeline P10_discharge_summary.algo

Notes become a clinician summary and then a patient-friendly copy.

Input

`notes` `text` Admitted with pneumonia, treated with IV antibiotics for 4 days, now afebrile.

Output type: `text`

AlgoLang code

```
// Discharge Summary: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent Extractor(notes: text) -> text {
   role "Clinical coder"
   goal "Extract diagnoses, treatments and follow-ups from {notes}"
   model smart
}

agent SummaryDrafter(facts: text) -> text {
   role "Discharge clinician"
   goal "Draft a discharge summary from {facts} for clinician review"
   model smart
}

agent PlainLanguage(summary: text) -> text {
   role "Patient educator"
   goal "Rewrite {summary} in plain language for the patient"
   model fast
}

flow main(notes: text) {
   facts = Extractor(notes)
   summary = SummaryDrafter(facts)
   patient_copy = PlainLanguage(summary)
   emit patient_copy
}
```

Output 1 – Command line

Terminal

```
$ algo run P10_discharge_summary.algo --arg "notes=Admitted with pneumonia, treated with IV antibiotics for 4 days, now afebrile."

▶ flow main  run=6814ddd1  backend=mock
    notes = 'Admitted with pneumonia, treated with IV antibiotics for 4 days, now afebrile.'
  → Extractor          conf=0.77  «Extractor #6525» Extract diagnoses, treatments and follow-ups from Admi  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → SummaryDrafter     conf=0.84  «SummaryDrafter #7271» Draft a discharge summary from «Extractor #6525»
  → PlainLanguage      conf=0.90  «PlainLanguage #5966» Rewrite «SummaryDrafter #7271» Draft a discharge s
■ emitted:
    • «PlainLanguage #5966» Rewrite «SummaryDrafter #7271» Draft a discharge summary from «Extract…
■ 3 agent calls · ~311 tokens · 0 retries · 1 escalated · 0 blocked · audit → runs/P10_discharge_summary.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 3 agent calls · 311 tokens · 0 retries · 1 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`Extractor`	sonnet-5-5	0.77	101	«Extractor #6525» Extract diagnoses, treatments and follow-ups fr… **human**
2	`SummaryDrafter`	sonnet-5-5	0.84	106	«SummaryDrafter #7271» Draft a discharge summary from «Extractor …
3	`PlainLanguage`	haiku-5-5	0.90	104	«PlainLanguage #5966» Rewrite «SummaryDrafter #7271» Draft a disc…

Result: `[«PlainLanguage #5966» Rewrite «SummaryDrafter #7271» Draft a discharge summary from «Extract…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "write a discharge summary from clinical notes"

AlgoLang: matched **Discharge summary drafting** (`healthcare.discharge_summary`), Healthcare, pattern `pipe`, match score 0.68. Governance: human review below confidence 0.90, forbids caste, religion. Agents: Collector, Writer, Checker, Finaliser.

Matched a pack in the same sector. Other matches offered: `healthcare.clinical_triage`, `healthcare.patient_feedback`, `healthcare.appointment_router`.

#	Agent	Model	Conf.	Tok.	Output
1	`Collector`	sonnet-5-5	0.93	103	«Collector #0584» Collect diagnoses, procedures and medications f…
2	`Writer`	sonnet-5-5	0.79	123	«Writer #7586» Draft a patient-friendly discharge summary from «C…
3	`Checker`	sonnet-5-5	0.94	113	0.86
4	`Finaliser`	sonnet-5-5	0.92	113	«Finaliser #9585» Finalise the discharge summary «Writer #7586» D…

Final answer: `[«Finaliser #9585» Finalise the discharge summary «Writer #7586» Draft a patient-friendly…, 0.86]` (4 agent calls · 452 tokens · 0 retries · 0 escalated · 0 blocked)

11 Care Team Review

Healthcare fan-out P11_care_team_review.algo

Three specialties review in parallel; a coordinator merges.

Input

case `text` 72 year old, diabetes, new swelling in both ankles, on three medications

Output type: `text`

AlgoLang code

```
// Care Team Review: workers run in parallel, one agent merges.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent CardiologyView(case: text) -> text {
   role "Cardiology reviewer"
   goal "List cardiac concerns for {case} for specialist follow-up"
   model smart
}

agent PharmacyView(case: text) -> text {
   role "Clinical pharmacist"
   goal "List medication concerns for {case} for review"
   model smart
}

agent NursingView(case: text) -> text {
   role "Ward nurse"
   goal "List nursing observations to monitor for {case}"
   model fast
}

agent Coordinator(cardio: text, pharmacy: text, nursing: text) -> text {
   role "Care coordinator"
   goal "Combine {cardio}, {pharmacy} and {nursing} into one action list for the care team"
   model smart
}

flow main(case: text) {
   [cardio, pharmacy, nursing] = par {
      CardiologyView(case),
      PharmacyView(case),
      NursingView(case)
   }
   plan = Coordinator(cardio, pharmacy, nursing)
   emit plan
}
```

Output 1 – Command line

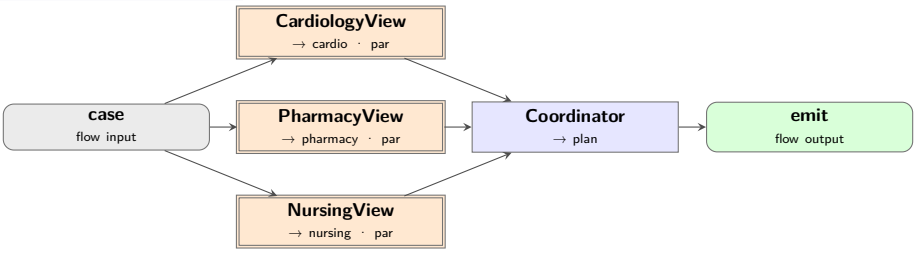
Terminal

```
$ algo run P11_care_team_review.algo --arg "case=72 year old, diabetes, new swelling in both ankles, on three medications"

▶ flow main  run=1a7baab6  backend=mock
    case = '72 year old, diabetes, new swelling in both ankles, on three medications'
  → CardiologyView     conf=0.74  «CardiologyView #4752» List cardiac concerns for 72 year old, diabetes,   ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → NursingView        conf=0.67  «NursingView #4561» List nursing observations to monitor for 72 year old  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → PharmacyView       conf=0.72  «PharmacyView #3048» List medication concerns for 72 year old, diabetes,  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → Coordinator        conf=0.83  «Coordinator #2847» Combine «CardiologyView #4752» List cardiac concerns
■ emitted:
    • «Coordinator #2847» Combine «CardiologyView #4752» List cardiac concerns for 72 year old, …
■ 4 agent calls · ~450 tokens · 0 retries · 3 escalated · 0 blocked · audit → runs/P11_care_team_review.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 450 tokens · 0 retries · 3 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	CardiologyView	sonnet-5-5	0.74	101	«CardiologyView #4752» List cardiac concerns for 72 year old, dia… **human**
2	PharmacyView	sonnet-5-5	0.72	99	«PharmacyView #3048» List medication concerns for 72 year old, di… **human**
3	NursingView	haiku-5-5	0.67	97	«NursingView #4561» List nursing observations to monitor for 72 y… **human**
4	Coordinator	sonnet-5-5	0.83	153	«Coordinator #2847» Combine «CardiologyView #4752» List cardiac c…

Result: [«Coordinator #2847» Combine «CardiologyView #4752» List cardiac concerns for 72 year old, …]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "multi-expert clinical review of a patient"

AlgoLang: matched **Patient feedback analysis** (healthcare.patient_feedback), Healthcare, pattern fan, match score 0.59. Governance: human review below confidence 0.70. Agents: Splitter, Analyst, Satisfaction, Reporter.

Matched a pack in the same sector. Other matches offered: healthcare.appointment_router, healthcare.drug_interaction, healthcare.clinical_triage.

#	Agent	Model	Conf.	Tok.	Output
1	Splitter	haiku-5-5	0.91	97	[item 1 for: Split multi-expert clinical review of a patient into…
2	Analyst	sonnet-5-5	0.87	121	«Analyst #0194» Classify the theme and sentiment of item 1 for: S…
3	Analyst	sonnet-5-5	0.78	121	«Analyst #6001» Classify the theme and sentiment of item 2 for: S…
4	Analyst	sonnet-5-5	0.68	121	«Analyst #8837» Classify the theme and sentiment of item 3 for: S…
5	Analyst	sonnet-5-5	0.70	121	«Analyst #2668» Classify the theme and sentiment of item 4 for: S…
6	Satisfaction	sonnet-5-5	0.68	179	0.77 **human**
7	Reporter	sonnet-5-5	0.85	180	«Reporter #4618» Write an improvement report from «Analyst #0194»…

Final answer: [«Reporter #4618» Write an improvement report from «Analyst #0194» Classify the theme an…, 0.77] (7 agent calls · 940 tokens · 0 retries · 1 escalated · 0 blocked)

12 Drug Interaction Check

Pharma fan-out P12_drug_interaction_check.algo

Three checks run together; the brief goes to a pharmacist.

Input

regimen	text	Warfarin 5 mg daily, ibuprofen 400 mg as needed, omeprazole 20 mg

Output type: text

AlgoLang code

```
// Drug Interaction Check: workers run in parallel, one agent merges.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent InteractionChecker(regimen: text) -> text {
   role "Pharmacology reviewer"
   goal "Flag possible interactions in {regimen}"
   model smart
}

agent ContraindicationChecker(regimen: text) -> text {
   role "Pharmacology reviewer"
   goal "Flag possible contraindications in {regimen}"
   model smart
}

agent DoseChecker(regimen: text) -> text {
   role "Pharmacology reviewer"
   goal "Check {regimen} against typical dose ranges"
   model fast
}

agent PharmacistBrief(interactions: text, contra: text, doses: text) -> text {
   role "Senior pharmacist"
   goal "Write a brief for a pharmacist from {interactions}, {contra} and {doses}; decision support only"
   model smart
}

flow main(regimen: text) {
   [interactions, contra, doses] = par {
      InteractionChecker(regimen),
      ContraindicationChecker(regimen),
      DoseChecker(regimen)
   }
   brief = PharmacistBrief(interactions, contra, doses)
   emit brief
}
```

Output 1 – Command line

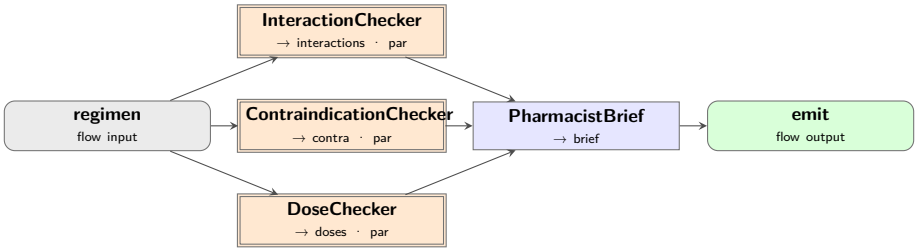
Terminal

```
$ algo run P12_drug_interaction_check.algo --arg "regimen=Warfarin 5 mg daily, ibuprofen 400 mg as needed, omeprazole 20 mg"

▶ flow main  run=d45a6310  backend=mock
    regimen = 'Warfarin 5 mg daily, ibuprofen 400 mg as needed, omeprazole 20 mg'
  → ContraindicationChecker conf=0.84  «ContraindicationChecker #1171» Flag possible contraindications in Warfa
  → InteractionChecker conf=0.65  «InteractionChecker #5120» Flag possible interactions in Warfarin 5 mg d  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → DoseChecker        conf=0.63  «DoseChecker #0339» Check Warfarin 5 mg daily, ibuprofen 400 mg as neede  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → PharmacistBrief    conf=0.80  «PharmacistBrief #6428» Write a brief for a pharmacist from «Interaction
■ emitted:
    • «PharmacistBrief #6428» Write a brief for a pharmacist from «InteractionChecker #5120» Flag po…
■ 4 agent calls · ~447 tokens · 0 retries · 2 escalated · 0 blocked · audit → runs/P12_drug_interaction_check.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 447 tokens · 0 retries · 2 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	InteractionChecker	sonnet-5-5	0.65	95	«InteractionChecker #5120» Flag possible interactions in Warfarin… **human**
2	DoseChecker	haiku-5-5	0.63	96	«DoseChecker #0339» Check Warfarin 5 mg daily, ibuprofen 400 mg a… **human**
3	ContraindicationChecker	sonnet-5-5	0.84	96	«ContraindicationChecker #1171» Flag possible contraindications i…
4	PharmacistBrief	sonnet-5-5	0.80	160	«PharmacistBrief #6428» Write a brief for a pharmacist from «Inte…

Result: [«PharmacistBrief #6428» Write a brief for a pharmacist from «InteractionChecker #5120» Flag po…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "check a medication list for interactions"

AlgoLang: matched **Drug interaction review** (healthcare.drug_interaction), Healthcare, pattern committee, match score 0.39. Governance: human review below confidence 0.90. Agents: Pharmacologist, Clinician, SafetyOfficer, RiskJudge, Advisor.

The closest pack in the library is filed under the Healthcare sector. Other matches offered: healthcare.clinical_triage, healthcare.discharge_summary, supplychain.customs_documents.

#	Agent	Model	Conf.	Tok.	Output
1	Pharmacologist	sonnet-5-5	0.70	98	«Pharmacologist #8291» Identify drug-drug interactions in check a…
2	SafetyOfficer	sonnet-5-5	0.65	100	«SafetyOfficer #7262» List contraindications and monitoring needs…
3	Clinician	sonnet-5-5	0.77	98	«Clinician #9793» Assess clinical relevance of interactions in ch…
4	RiskJudge	sonnet-5-5	0.93	169	0.74
5	Advisor	sonnet-5-5	0.82	168	«Advisor #4646» Write prescriber advice from «Pharmacologist #829…

Final answer: [«Advisor #4646» Write prescriber advice from «Pharmacologist #8291» Identify drug-drug…, 0.74] (5 agent calls · 633 tokens · 0 retries · 0 escalated · 0 blocked)

13 Adverse Event Router

Pharma router P13_adverse_event_router.algo

Serious events get an expedited report; others are logged.

Input

`report`	`text`	Patient developed severe rash and fever two days after starting the new antibiotic

Output type: `score, text`

AlgoLang code

```
// Adverse Event Router: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent SeverityScorer(report: text) -> score {
   role "Pharmacovigilance officer"
   goal "Score the seriousness of the event in {report} from 0 to 1"
   model fast
}

agent ExpeditedReport(report: text) -> text {
   role "Safety writer"
   goal "Draft an expedited safety report for {report}"
   model smart
}

agent RoutineLog(report: text) -> text {
   role "Safety writer"
   goal "Write a routine log entry for {report}"
   model fast
}

flow main(report: text) {
   severity = SeverityScorer(report)

   if severity >= 0.7 {
      expedited = ExpeditedReport(report)
      emit severity, expedited
   } else {
      entry = RoutineLog(report)
      emit severity, entry
   }
}
```

Output 1 – Command line

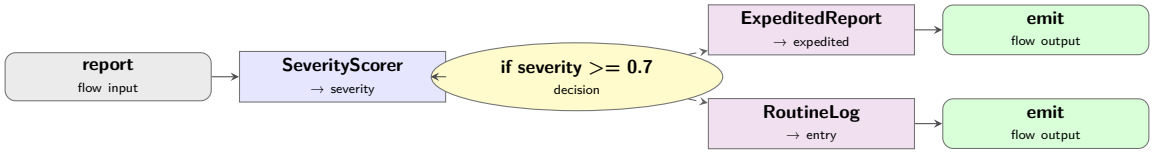
Terminal

```
$ algo run P13_adverse_event_router.algo --arg "report=Patient developed severe rash and fever two days after starting the new
↪  antibiotic"

▶ flow main  run=47392147  backend=mock
    report = 'Patient developed severe rash and fever two days after starting the new antibiotic'
  → SeverityScorer     conf=0.86  0.76
  → ExpeditedReport    conf=0.85  «ExpeditedReport #1085» Draft an expedited safety report for Patient dev
■ emitted:
    • 0.76
    • «ExpeditedReport #1085» Draft an expedited safety report for Patient developed severe rash and…
■ 2 agent calls · ~204 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P13_adverse_event_router.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 204 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`SeverityScorer`	haiku-5-5	0.86	105	0.76
2	`ExpeditedReport`	sonnet-5-5	0.85	99	«ExpeditedReport #1085» Draft an expedited safety report for Pati…

Result: `[0.76, «ExpeditedReport #1085» Draft an expedited safety report for Patient developed severe rash and…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "route an adverse drug event report"

AlgoLang: matched **Adverse event case processing** (`pharma.adverse_events`), Pharma & Life Sciences, pattern `pipe`, match score 0.52. Governance: human review below confidence 0.90, forbids caste, religion. Agents: Intake, Coder, Seriousness, Narrative.

Matched a pack in the same sector. Other matches offered: `energy.grid_fault`, `healthcare.drug_interaction`, `pharma.literature_watch`.

#	Agent	Model	Conf.	Tok.	Output
1	`Intake`	sonnet-5-5	0.71	101	«Intake #3325» Extract patient, drug, event and timing details fr…
2	`Coder`	sonnet-5-5	0.87	120	«Coder #8522» Code the event in «Intake #3325» Extract patient, d…
3	`Seriousness`	sonnet-5-5	0.82	107	0.75 **human**
4	`Narrative`	sonnet-5-5	0.88	117	«Narrative #0074» Write a pharmacovigilance case narrative from «…

Final answer: `[«Narrative #0074» Write a pharmacovigilance case narrative from «Coder #8522» Code the e…, 0.75]` (4 agent calls · 445 tokens · 0 retries · 1 escalated · 0 blocked)

14 Credit Committee

Banking committee P14_credit_committee.algo

Three scorers vote; the chair recommends. Protected attributes are forbidden.

Input

`application`	`text`	Salaried, income 85000 per month, existing EMI 12000, requests 15 lakh home loan

Output type: `text, score, score, score`

AlgoLang code

```
// Credit Committee: a scoring committee and a chair.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 4000
   retry 1
   forbid gender, caste, religion
   require human when confidence < 0.8
}

agent BureauAnalyst(application: text) -> score {
   role "Credit bureau analyst"
   goal "Score the repayment history in {application} from 0 to 1"
   model smart
}

agent DebtAnalyst(application: text) -> score {
   role "Debt-to-income analyst"
   goal "Score the debt-to-income position in {application} from 0 to 1"
   model smart
}

agent CollateralAnalyst(application: text) -> score {
   role "Collateral valuer"
   goal "Score the collateral strength in {application} from 0 to 1"
   model smart
}

agent CreditChair(application: text, bureau: score, dti: score, collateral: score) -> text {
   role "Credit committee chair"
   goal "Recommend approve, refer or decline for {application} based on the panel scores"
   model smart
}

flow main(application: text) {
   [bureau, dti, collateral] = par {
      BureauAnalyst(application),
      DebtAnalyst(application),
      CollateralAnalyst(application)
   }
   decision = CreditChair(application, bureau, dti, collateral)
   emit decision, bureau, dti, collateral
}
```

Output 1 – Command line

Terminal

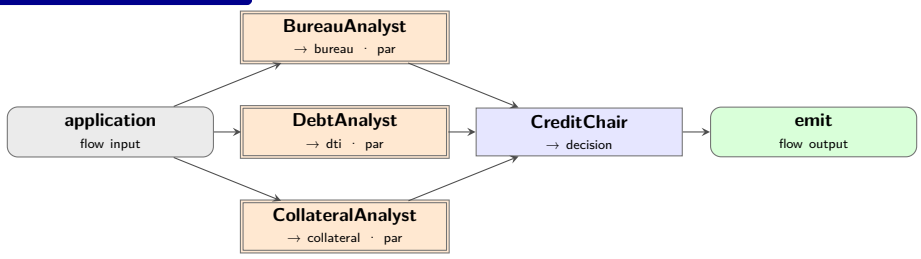
```
$ algo run P14_credit_committee.algo --arg "application=Salaried, income 85000 per month, existing EMI 12000, requests 15 lakh home
↪  loan"

▶ flow main  run=f212b203  backend=mock
    application = 'Salaried, income 85000 per month, existing EMI 12000, requests 15 lakh home loan'
  → BureauAnalyst      conf=0.72  0.75  ⚠ ESCALATED to human (conf < 0.80)
  → DebtAnalyst        conf=0.71  0.81  ⚠ ESCALATED to human (conf < 0.80)
  → CollateralAnalyst  conf=0.69  0.61  ⚠ ESCALATED to human (conf < 0.80)
  → CreditChair        conf=0.82  «CreditChair #3990» Recommend approve, refer or decline for Salaried, in
■ emitted:
    • «CreditChair #3990» Recommend approve, refer or decline for Salaried, income 85000 per mon…
    • 0.75
    • 0.81
```

```
    • 0.61
■ 4 agent calls · ~415 tokens · 0 retries · 3 escalated · 0 blocked · audit → runs/P14_credit_committee.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 415 tokens · 0 retries · 3 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	BureauAnalyst	sonnet-5-5	0.72	102	0.75 **human**
2	CollateralAnalyst	sonnet-5-5	0.69	101	0.61 **human**
3	DebtAnalyst	sonnet-5-5	0.71	104	0.81 **human**
4	CreditChair	sonnet-5-5	0.82	108	«CreditChair #3990» Recommend approve, refer or decline for Salar…

Result: [«CreditChair #3990» Recommend approve, refer or decline for Salaried, income 85000 per mon…, 0.75, 0.81, 0.61]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "run a credit committee on a loan application"

AlgoLang: matched **Loan underwriting assistant** (banking.loan_underwriting), Banking, pattern committee, match score 0.85. Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: CreditAnalyst, RiskAnalyst, PolicyChecker, CreditCommittee, Memo.

Matched a pack in the same sector. Other matches offered: banking.credit_memo, research.ethics_check, education.admissions.

#	Agent	Model	Conf.	Tok.	Output
1	CreditAnalyst	sonnet-5-5	0.90	99	«CreditAnalyst #7810» Assess repayment capacity in run a credit c…
2	RiskAnalyst	sonnet-5-5	0.94	96	«RiskAnalyst #3026» Identify credit risks in run a credit committ…
3	PolicyChecker	sonnet-5-5	0.79	98	«PolicyChecker #5360» Check run a credit committee on a loan appl…
4	CreditCommittee	sonnet-5-5	0.63	177	0.66 **human**
5	Memo	sonnet-5-5	0.66	177	«Memo #7021» Write a credit decision memo from «CreditAnalyst #78…

Final answer: [«Memo #7021» Write a credit decision memo from «CreditAnalyst #7810» Assess repayme…, 0.66] (5 agent calls · 647 tokens · 0 retries · 1 escalated · 0 blocked)

15 KYC Onboarding

Banking pipeline P15_kyc_onboarding.algo

Identity, sanctions screening, then an onboarding recommendation.

Input

`customer`	`text`	Ravi Kumar, software engineer, Pune, salary account, Aadhaar and PAN provided

Output type: `text`

AlgoLang code

```
// KYC Onboarding: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 4000
   retry 1
   forbid gender, caste, religion
   require human when confidence < 0.8
}

agent IdentityCheck(customer: text) -> text {
   role "KYC analyst"
   goal "Check that the identity documents in {customer} are complete and consistent"
   model smart
}

agent SanctionsScreen(identity: text) -> text {
   role "Compliance analyst"
   goal "Screen {identity} against sanctions and watch-list rules"
   model smart
}

agent OnboardingDecision(screening: text) -> text {
   role "Onboarding officer"
   goal "Recommend accept, hold or reject based on {screening}"
   model smart
}

flow main(customer: text) {
   identity = IdentityCheck(customer)
   screening = SanctionsScreen(identity)
   decision = OnboardingDecision(screening)
   emit decision
}
```

Output 1 – Command line

Terminal

```
$ algo run P15_kyc_onboarding.algo --arg "customer=Ravi Kumar, software engineer, Pune, salary account, Aadhaar and PAN provided"

▶ flow main  run=649271b2  backend=mock
    customer = 'Ravi Kumar, software engineer, Pune, salary account, Aadhaar and PAN provided'
  → IdentityCheck      conf=0.63  «IdentityCheck #5224» Check that the identity documents in Ravi Kumar, s  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → SanctionsScreen    conf=0.93  «SanctionsScreen #1779» Screen «IdentityCheck #5224» Check that the iden
  → OnboardingDecision conf=0.76  «OnboardingDecision #5051» Recommend accept, hold or reject based on «Sa  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
■ emitted:
    • «OnboardingDecision #5051» Recommend accept, hold or reject based on «SanctionsScreen #1779» Scre…
■ 3 agent calls · ~314 tokens · 0 retries · 2 escalated · 0 blocked · audit → runs/P15_kyc_onboarding.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 3 agent calls · 314 tokens · 0 retries · 2 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`IdentityCheck`	sonnet-5-5	0.63	104	«IdentityCheck #5224» Check that the identity documents in Ravi K… **human**
2	`SanctionsScreen`	sonnet-5-5	0.93	105	«SanctionsScreen #1779» Screen «IdentityCheck #5224» Check that t…
3	`OnboardingDecision`	sonnet-5-5	0.76	105	«OnboardingDecision #5051» Recommend accept, hold or reject based… **human**

Result: `[«OnboardingDecision #5051» Recommend accept, hold or reject based on «SanctionsScreen #1779» Scre…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "onboard a new bank customer with KYC"

AlgoLang: matched **KYC document review** (`banking.kyc_review`), Banking, pattern pipe, match score 0.84. Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: DocChecker, Verifier, RiskRating, Outcome.

Matched a pack in the same sector. Other matches offered: `banking.complaints, retail.review_insights, retail.customer_service`.

#	Agent	Model	Conf.	Tok.	Output
1	`DocChecker`	sonnet-5-5	0.74	97	«DocChecker #7192» List KYC documents present and missing in onbo…
2	`Verifier`	sonnet-5-5	0.80	119	«Verifier #9449» Check consistency of details across «DocChecker …
3	`RiskRating`	sonnet-5-5	0.79	107	0.62 **human**
4	`Outcome`	sonnet-5-5	0.92	110	«Outcome #4066» Write the KYC review outcome for «Verifier #9449»…

Final answer: `[«Outcome #4066» Write the KYC review outcome for «Verifier #9449» Check consistency of…, 0.62]` (4 agent calls · 433 tokens · 0 retries · 1 escalated · 0 blocked)

16 Fraud Alert Router

Banking router P16_fraud_alert_router.algo

High fraud scores hold the payment for human review.

Input

`transaction`	`text`	Card used in two countries within 10 minutes, 2400 USD electronics purchase

Output type: `score, text`

AlgoLang code

```
// Fraud Alert Router: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent FraudScorer(transaction: text) -> score {
   role "Fraud analyst"
   goal "Score the fraud likelihood of {transaction} from 0 to 1"
   model fast
}

agent HoldAndReview(transaction: text) -> text {
   role "Fraud operations lead"
   goal "Write a hold notice and review steps for {transaction}"
   model smart
}

agent ApproveWithNote(transaction: text) -> text {
   role "Fraud operations lead"
   goal "Write an approval note for {transaction}"
   model fast
}

flow main(transaction: text) {
   fraud = FraudScorer(transaction)

   if fraud >= 0.8 {
      hold = HoldAndReview(transaction)
      emit fraud, hold
   } else {
      approval = ApproveWithNote(transaction)
      emit fraud, approval
   }
}
```

Output 1 – Command line

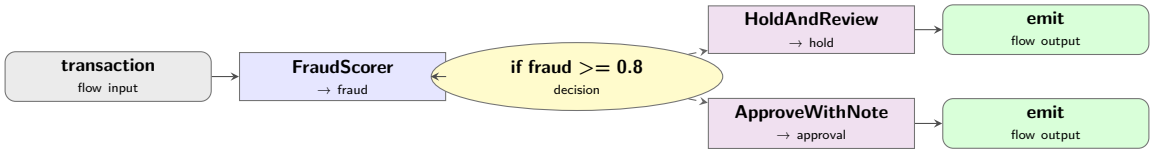
Terminal

```
$ algo run P16_fraud_alert_router.algo --arg "transaction=Card used in two countries within 10 minutes, 2400 USD electronics purchase"

▶ flow main  run=57854d57  backend=mock
    transaction = 'Card used in two countries within 10 minutes, 2400 USD electronics purchase'
  → FraudScorer        conf=0.83  0.94
  → HoldAndReview      conf=0.84  «HoldAndReview #7166» Write a hold notice and review steps for Card used
■ emitted:
    • 0.94
    • «HoldAndReview #7166» Write a hold notice and review steps for Card used in two countries wi…
■ 2 agent calls · ~198 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P16_fraud_alert_router.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 198 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	FraudScorer	haiku-5-5	0.83	98	0.94
2	HoldAndReview	sonnet-5-5	0.84	100	«HoldAndReview #7166» Write a hold notice and review steps for Ca…

Result: `[0.94, «HoldAndReview #7166» Write a hold notice and review steps for Card used in two countries wi…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "review a suspected card fraud case"

AlgoLang: matched **Card fraud case review** (`banking.fraud_cases`), Banking, pattern `fan`, match score 1.04. Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: Splitter, Analyst, FraudScore, Report.

Matched a pack in the same sector. Other matches offered: `insurance.fraud_screening, retail.return_fraud, banking.kyc_review`.

#	Agent	Model	Conf.	Tok.	Output
1	Splitter	haiku-5-5	0.64	94	[item 1 for: Split review a suspected card fraud case into indivi…
2	Analyst	sonnet-5-5	0.94	114	«Analyst #5489» Assess item 1 for: Split review a suspected card …
3	Analyst	sonnet-5-5	0.63	114	«Analyst #6180» Assess item 2 for: Split review a suspected card …
4	Analyst	sonnet-5-5	0.91	114	«Analyst #1956» Assess item 3 for: Split review a suspected card …
5	FraudScore	sonnet-5-5	0.69	154	0.78 **human**
6	Report	sonnet-5-5	0.85	156	«Report #5917» Write a fraud review report from «Analyst #5489» A…

Final answer: `[«Report #5917» Write a fraud review report from «Analyst #5489» Assess item 1 for: Sp…, 0.78]` (6 agent calls · 746 tokens · 0 retries · 1 escalated · 0 blocked)

17 Claims Triage

Insurance router P17_claims_triage.algo

Risky claims go to investigators; the rest are fast-tracked.

Input

`claim` `text` Rear-end collision, bumper damage, no injuries, claim 45000 filed same day

Output type: `score, text`

AlgoLang code

```
// Claims Triage: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent ClaimRiskScorer(claim: text) -> score {
   role "Fraud analyst"
   goal "Score the fraud risk of {claim} from 0 to 1"
   model fast
}

agent SIUReferral(claim: text) -> text {
   role "Special investigations lead"
   goal "Write a referral to the investigation unit for {claim}"
   model smart
}

agent FastTrackPayout(claim: text) -> text {
   role "Claims adjuster"
   goal "Write a fast-track payout instruction for {claim}"
   model fast
}

flow main(claim: text) {
   risk = ClaimRiskScorer(claim)

   if risk >= 0.7 {
      referral = SIUReferral(claim)
      emit risk, referral
   } else {
      payout = FastTrackPayout(claim)
      emit risk, payout
   }
}
```

Output 1 – Command line

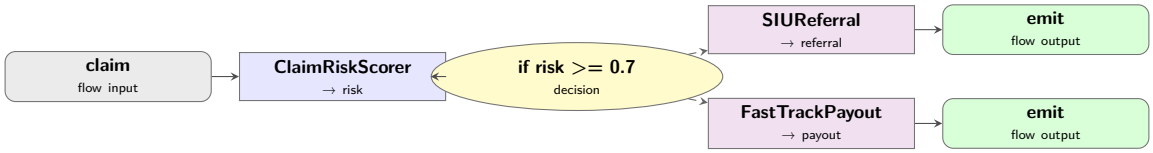
Terminal

```
$ algo run P17_claims_triage.algo --arg "claim=Rear-end collision, bumper damage, no injuries, claim 45000 filed same day"

▶ flow main  run=439339dc  backend=mock
    claim = 'Rear-end collision, bumper damage, no injuries, claim 45000 filed same day'
  → ClaimRiskScorer    conf=0.72  0.9  ⚠ ESCALATED to human (conf < 0.80)
  → SIUReferral        conf=0.89  «SIUReferral #9420» Write a referral to the investigation unit for Rear-
■ emitted:
    • 0.9
    • «SIUReferral #9420» Write a referral to the investigation unit for Rear-end collision, bum…
■ 2 agent calls · ~200 tokens · 0 retries · 1 escalated · 0 blocked · audit → runs/P17_claims_triage.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 200 tokens · 0 retries · 1 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`ClaimRiskScorer`	haiku-5-5	0.72	97	0.90 **human**
2	`SIUReferral`	sonnet-5-5	0.89	103	«SIUReferral #9420» Write a referral to the investigation unit fo…

Result: `[0.90, «SIUReferral #9420» Write a referral to the investigation unit for Rear-end collision, bum…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "triage an insurance claim for fraud"

AlgoLang: matched **Claims fraud screening** (`insurance.fraud_screening`), Insurance, pattern `committee`, match score 0.79. Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: PatternAnalyst, DocumentExaminer, Adjuster, FraudPanel, Referral.

Matched a pack in the same sector. Other matches offered: `insurance.claims_triage, banking.fraud_cases, retail.return_fraud`.

#	Agent	Model	Conf.	Tok.	Output
1	`PatternAnalyst`	sonnet-5-5	0.93	95	«PatternAnalyst #6919» Look for known fraud patterns in triage an…
2	`DocumentExaminer`	sonnet-5-5	0.90	97	«DocumentExaminer #6024» Check documents in triage an insurance c…
3	`Adjuster`	sonnet-5-5	0.82	94	«Adjuster #7430» Assess plausibility of the loss in triage an ins…
4	`FraudPanel`	sonnet-5-5	0.81	156	0.65 **human**
5	`Referral`	sonnet-5-5	0.63	159	«Referral #5615» Write an SIU referral note from «Pattern-Analyst …

Final answer: `[«Referral #5615» Write an SIU referral note from «PatternAnalyst #6919» Look for known …, 0.65]` (5 agent calls · 601 tokens · 0 retries · 1 escalated · 0 blocked)

18 Renewal Advisor

Insurance pipeline P18_renewal_advisor.algo

Risk review, pricing, then a customer letter.

Input

`holder_profile`	`text`	Two-wheeler, 4 years no claim, city commuter, policy expires next month

Output type: `text`

AlgoLang code

```
// Renewal Advisor: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent RiskReview(holder_profile: text) -> text {
   role "Underwriter"
   goal "Summarise the risk profile in {holder_profile}"
   model smart
}

agent PremiumReview(risk_view: text) -> text {
   role "Pricing analyst"
   goal "Suggest a renewal premium range and discounts from {risk_view}"
   model smart
}

agent RenewalLetter(premium_view: text) -> text {
   role "Customer writer"
   goal "Write a clear renewal letter based on {premium_view}"
   model fast
}

flow main(holder_profile: text) {
   risk_view = RiskReview(holder_profile)
   premium_view = PremiumReview(risk_view)
   letter = RenewalLetter(premium_view)
   emit letter
}
```

Output 1 – Command line

Terminal

```
$ algo run P18_renewal_advisor.algo --arg "holder_profile=Two-wheeler, 4 years no claim, city commuter, policy expires next month"

▶ flow main  run=12b03b34  backend=mock
    holder_profile = 'Two-wheeler, 4 years no claim, city commuter, policy expires next month'
  → RiskReview         conf=0.89  «RiskReview #1387» Summarise the risk profile in Two-wheeler, 4 years no
  → PremiumReview      conf=0.65  «PremiumReview #7261» Suggest a renewal premium range and discounts from  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → RenewalLetter      conf=0.87  «RenewalLetter #6389» Write a clear renewal letter based on «PremiumRevi
■ emitted:
    • «RenewalLetter #6389» Write a clear renewal letter based on «PremiumReview #7261» Suggest a …
■ 3 agent calls · ~301 tokens · 0 retries · 1 escalated · 0 blocked · audit → runs/P18_renewal_advisor.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 3 agent calls · 301 tokens · 0 retries · 1 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`RiskReview`	sonnet-5-5	0.89	94	«RiskReview #1387» Summarise the risk profile in Two-wheeler, 4 y…
2	`PremiumReview`	sonnet-5-5	0.65	105	«PremiumReview #7261» Suggest a renewal premium range and discoun… **human**
3	`RenewalLetter`	haiku-5-5	0.87	102	«RenewalLetter #6389» Write a clear renewal letter based on «Prem…

Result: `[«RenewalLetter #6389» Write a clear renewal letter based on «PremiumReview #7261» Suggest a …]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "compare insurance policies for a customer renewal"

AlgoLang: matched **Policy comparison** (`insurance.policy_compare`), Insurance, pattern fan, match score 0.76. Governance: human review below confidence 0.70. Agents: Policies, Analyst, BestFit, Advice.

Matched a pack in the same sector. Other matches offered: `insurance.claims_triage`, `retail.review_insights`, `retail.customer_service`.

#	Agent	Model	Conf.	Tok.	Output
1	`Policies`	haiku-5-5	0.85	97	[item 1 for: List the policies described in compare insurance pol…
2	`Analyst`	sonnet-5-5	0.85	106	«Analyst #2821» Summarise cover, exclusions and limits of item 1 …
3	`Analyst`	sonnet-5-5	0.93	106	«Analyst #9730» Summarise cover, exclusions and limits of item 3 …
4	`Analyst`	sonnet-5-5	0.88	106	«Analyst #3092» Summarise cover, exclusions and limits of item 2 …
5	`BestFit`	sonnet-5-5	0.79	155	0.77
6	`Advice`	sonnet-5-5	0.84	154	«Advice #8498» Write a plain-language comparison from «Analyst #2…

Final answer: `[«Advice #8498» Write a plain-language comparison from «Analyst #2821» Summarise cover…, 0.77]` (6 agent calls · 724 tokens · 0 retries · 0 escalated · 0 blocked)

19 Earnings Brief

Capital Markets fan-out P19_earnings_brief.algo

Bull, bear and risk views are written in parallel and merged.

Input

`transcript`	`text`	Revenue up 12 percent, margin pressure from input costs, guidance raised for next quarter

Output type: `text`

AlgoLang code

```
// Earnings Brief: workers run in parallel, one agent merges.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent BullCase(transcript: text) -> text {
   role "Equity analyst"
   goal "Give the strongest positive reading of {transcript}"
   model smart
}

agent BearCase(transcript: text) -> text {
   role "Equity analyst"
   goal "Give the strongest negative reading of {transcript}"
   model smart
}

agent RiskFlags(transcript: text) -> text {
   role "Risk analyst"
   goal "List risk flags in {transcript}"
   model fast
}

agent MemoWriter(bull: text, bear: text, risks: text) -> text {
   role "Research editor"
   goal "Write a balanced memo from {bull}, {bear} and {risks}; information only, not investment advice"
   model smart
}

flow main(transcript: text) {
   [bull, bear, risks] = par {
      BullCase(transcript),
      BearCase(transcript),
      RiskFlags(transcript)
   }
   memo = MemoWriter(bull, bear, risks)
   emit memo
}
```

Output 1 – Command line

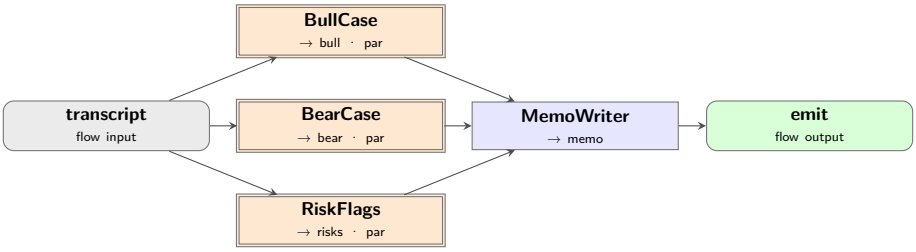
Terminal

```
$ algo run P19_earnings_brief.algo --arg "transcript=Revenue up 12 percent, margin pressure from input costs, guidance raised for next
↪  quarter"

▶ flow main  run=f38652c2  backend=mock
    transcript = 'Revenue up 12 percent, margin pressure from input costs, guidance raised for next quarter'
  → BullCase           conf=0.72  «BullCase #6427» Give the strongest positive reading of Revenue up 12 pe
  → BearCase           conf=0.74  «BearCase #2159» Give the strongest negative reading of Revenue up 12 pe
  → RiskFlags          conf=0.92  «RiskFlags #1258» List risk flags in Revenue up 12 percent, margin press
  → MemoWriter         conf=0.72  «MemoWriter #4300» Write a balanced memo from «BullCase #6427» Give the
■ emitted:
    • «MemoWriter #4300» Write a balanced memo from «BullCase #6427» Give the strongest positiv…
■ 4 agent calls · ~453 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P19_earnings_brief.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 453 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	BullCase	sonnet-5-5	0.72	101	«BullCase #6427» Give the strongest positive reading of Revenue u…
2	BearCase	sonnet-5-5	0.74	101	«BearCase #2159» Give the strongest negative reading of Revenue u…
3	RiskFlags	haiku-5-5	0.92	96	«RiskFlags #1258» List risk flags in Revenue up 12 percent, margi…
4	MemoWriter	sonnet-5-5	0.72	155	«MemoWriter #4300» Write a balanced memo from «BullCase #6427» Gi…

Result: `[«MemoWriter #4300» Write a balanced memo from «BullCase #6427» Give the strongest positiv…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "summarise an earnings call from three viewpoints"

AlgoLang: matched **Earnings call summariser** (`markets.earnings_call`), Capital Markets & Investment, pattern fan, match score 0.61. Governance: human review below confidence 0.70. Agents: Sections, Analyst, Sentiment, Brief.

Matched a pack in the same sector. Other matches offered: `realestate.valuation, markets.esg_screening, cyber.soc_triage`.

#	Agent	Model	Conf.	Tok.	Output
1	Sections	haiku-5-5	0.80	102	[item 1 for: Split summarise an earnings call from three viewpoin…
2	Analyst	sonnet-5-5	0.78	107	«Analyst #0671» Summarise key numbers and guidance in item 1 for:…
3	Analyst	sonnet-5-5	0.78	107	«Analyst #6111» Summarise key numbers and guidance in item 3 for:…
4	Analyst	sonnet-5-5	0.72	107	«Analyst #9913» Summarise key numbers and guidance in item 2 for:…
5	Analyst	sonnet-5-5	0.68	107	«Analyst #3586» Summarise key numbers and guidance in item 4 for:…
6	Sentiment	sonnet-5-5	0.65	176	0.84 **human**
7	Brief	sonnet-5-5	0.94	177	«Brief #5478» Write an earnings brief from «Analyst #0671» Summar…

Final answer: `[«Brief #5478» Write an earnings brief from «Analyst #0671» Summarise key numbers and…, 0.84]` (7 agent calls · 883 tokens · 0 retries · 1 escalated · 0 blocked)

20 Trade Compliance Check

Capital Markets router P20_trade_compliance_check.algo

Risky trades are escalated to a human.

Input

`trade`	`text`	Employee buys 5000 shares of a client company two days before an announcement

Output type: `score, text`

AlgoLang code

```
// Trade Compliance Check: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent ComplianceScorer(trade: text) -> score {
   role "Compliance officer"
   goal "Score the policy-breach risk of {trade} from 0 to 1"
   model fast
}

agent EscalationNote(trade: text) -> text {
   role "Compliance officer"
   goal "Write an escalation note for {trade} for the compliance head"
   model smart
}

agent ClearanceNote(trade: text) -> text {
   role "Compliance officer"
   goal "Write a clearance note for {trade}"
   model fast
}

flow main(trade: text) {
   breach = ComplianceScorer(trade)

   if breach >= 0.6 {
      escalation = EscalationNote(trade)
      emit breach, escalation
   } else {
      clearance = ClearanceNote(trade)
      emit breach, clearance
   }
}
```

Output 1 – Command line

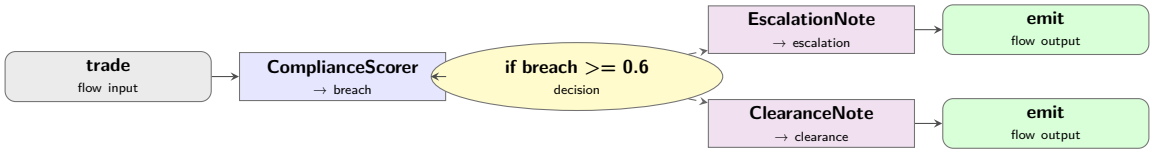
Terminal

```
$ algo run P20_trade_compliance_check.algo --arg "trade=Employee buys 5000 shares of a client company two days before an announcement"

▶ flow main  run=09e72199  backend=mock
    trade = 'Employee buys 5000 shares of a client company two days before an announcement'
  → ComplianceScorer   conf=0.79  0.67  ⚠ ESCALATED to human (conf < 0.80)
  → EscalationNote     conf=0.90  «EscalationNote #5852» Write an escalation note for Employee buys 5000 s
■ emitted:
    • 0.67
    • «EscalationNote #5852» Write an escalation note for Employee buys 5000 shares of a client com…
■ 2 agent calls · ~204 tokens · 0 retries · 1 escalated · 0 blocked · audit → runs/P20_trade_compliance_check.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 204 tokens · 0 retries · 1 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`ComplianceScorer`	haiku-5-5	0.79	101	0.67 **human**
2	`EscalationNote`	sonnet-5-5	0.90	103	«EscalationNote #5852» Write an escalation note for Employee buys…

Result: `[0.67, «EscalationNote #5852» Write an escalation note for Employee buys 5000 shares of a client com…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "check a trade for compliance issues"

AlgoLang: matched **Trade surveillance** (`markets.trade_surveillance`), Capital Markets & Investment, pattern `fan`, match score 0.36. Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: Splitter, Investigator, Concern, Report.

Matched a pack in the same sector. Other matches offered: `legal.compliance_gap`, `pharma.label_compliance`, `manufacturing.sop_compliance`.

#	Agent	Model	Conf.	Tok.	Output
1	`Splitter`	haiku-5-5	0.87	90	[item 1 for: Split check a trade for compliance issues into trade…
2	`Investigator`	sonnet-5-5	0.83	103	«Investigator #1484» Assess item 1 for: Split check a trade for c…
3	`Investigator`	sonnet-5-5	0.66	103	«Investigator #3843» Assess item 4 for: Split check a trade for c…
4	`Investigator`	sonnet-5-5	0.68	103	«Investigator #6365» Assess item 3 for: Split check a trade for c…
5	`Investigator`	sonnet-5-5	0.67	103	«Investigator #7889» Assess item 2 for: Split check a trade for c…
6	`Concern`	sonnet-5-5	0.81	181	0.76 **human**
7	`Report`	sonnet-5-5	0.89	181	«Report #7737» Write a surveillance report from «Investigator #14…

Final answer: `[«Report #7737» Write a surveillance report from «Investigator #1484» Assess item 1 fo…, 0.76]` (7 agent calls · 864 tokens · 0 retries · 1 escalated · 0 blocked)

21 Quiz Generator

Education pipeline P21_quiz_generator.algo

Questions, then distractors, then an answer key.

Input

`lesson`	`text`	Photosynthesis: plants convert light, water and carbon dioxide into glucose and oxygen.

Output type: `text`

AlgoLang code

```
// Quiz Generator: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent QuestionWriter(lesson: text) -> text {
   role "Teacher"
   goal "Write five multiple-choice questions from {lesson}"
   model smart
}

agent DistractorBuilder(questions: text) -> text {
   role "Assessment designer"
   goal "Add plausible wrong options to {questions}"
   model smart
}

agent AnswerKey(options: text) -> text {
   role "Teacher"
   goal "Produce an answer key with a one-line reason for {options}"
   model fast
}

flow main(lesson: text) {
   questions = QuestionWriter(lesson)
   options = DistractorBuilder(questions)
   key = AnswerKey(options)
   emit key
}
```

Output 1 – Command line

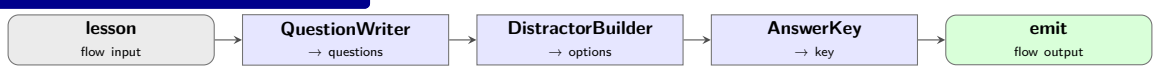
Terminal

```
$ algo run P21_quiz_generator.algo --arg "lesson=Photosynthesis: plants convert light, water and carbon dioxide into glucose and
↪  oxygen."

▶ flow main  run=fccbb0a0  backend=mock
    lesson = 'Photosynthesis: plants convert light, water and carbon dioxide into glucose and oxygen.'
  → QuestionWriter     conf=0.80  «QuestionWriter #4050» Write five multiple-choice questions from Photosy
  → DistractorBuilder  conf=0.70  «DistractorBuilder #8620» Add plausible wrong options to «QuestionWriter
  → AnswerKey          conf=0.62  «AnswerKey #3901» Produce an answer key with a one-line reason for «Dist
■ emitted:
    • «AnswerKey #3901» Produce an answer key with a one-line reason for «DistractorBuilder #8…
■ 3 agent calls · ~306 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P21_quiz_generator.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 306 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	QuestionWriter	sonnet-5-5	0.80	100	«QuestionWriter #4050» Write five multiple-choice questions from …
2	DistractorBuilder	sonnet-5-5	0.70	102	«DistractorBuilder #8620» Add plausible wrong options to «Questio…
3	AnswerKey	haiku-5-5	0.62	104	«AnswerKey #3901» Produce an answer key with a one-line reason fo…

Result: `[«AnswerKey #3901» Produce an answer key with a one-line reason for «DistractorBuilder #8…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "generate exam questions mapped to syllabus units"

AlgoLang: matched **Exam question bank** (`education.question_bank`), Education, pattern fan, match score 1.00. Governance: human review below confidence 0.80. Agents: Units, Setter, Coverage, Paper.

Matched a pack in the same sector. Other matches offered: `education.outcome_mapping`, `education.ai_tutor`, `education.essay_feedback`.

#	Agent	Model	Conf.	Tok.	Output
1	Units	haiku-5-5	0.81	91	[item 1 for: List the units of generate exam questions mapped to …
2	Setter	sonnet-5-5	0.89	108	«Setter #9440» Write 5 exam questions at different Bloom's levels…
3	Setter	sonnet-5-5	0.74	108	«Setter #8346» Write 5 exam questions at different Bloom's levels…
4	Setter	sonnet-5-5	0.79	108	«Setter #2863» Write 5 exam questions at different Bloom's levels…
5	Coverage	sonnet-5-5	0.91	151	0.86
6	Paper	sonnet-5-5	0.64	153	«Paper #9625» Assemble a question paper from «Setter #2863» Write…

Final answer: `[«Paper #9625» Assemble a question paper from «Setter #2863» Write 5 exam questions a…, 0.86]` (6 agent calls · 719 tokens · 0 retries · 0 escalated · 0 blocked)

22 Admissions Panel

Education committee P22_admissions_panel.algo

Merit-only panel; gender, caste and religion are forbidden.

Input

`applicant` `text` Rank 340 in entrance exam, 87 percent in class 12, led a robotics club, strong essay

Output type: `text, score, score, score`

AlgoLang code

```
// Admissions Panel: a scoring committee and a chair.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 4000
   retry 1
   forbid gender, caste, religion
   require human when confidence < 0.8
}

agent AcademicReviewer(applicant: text) -> score {
   role "Admissions reader"
   goal "Score the academic record in {applicant} from 0 to 1"
   model smart
}

agent ActivityReviewer(applicant: text) -> score {
   role "Admissions reader"
   goal "Score the activities in {applicant} from 0 to 1"
   model smart
}
```

```
agent EssayReviewer(applicant: text) -> score {
   role "Admissions reader"
   goal "Score the essay quality in {applicant} from 0 to 1"
   model smart
}

agent AdmissionsChair(applicant: text, academic: score, activity: score, essay: score) -> text {
   role "Admissions chair"
   goal "Recommend admit, waitlist or decline for {applicant} based on the panel scores"
   model smart
}

flow main(applicant: text) {
   [academic, activity, essay] = par {
      AcademicReviewer(applicant),
      ActivityReviewer(applicant),
      EssayReviewer(applicant)
   }
   decision = AdmissionsChair(applicant, academic, activity, essay)
   emit decision, academic, activity, essay
}
```

Output 1 – Command line

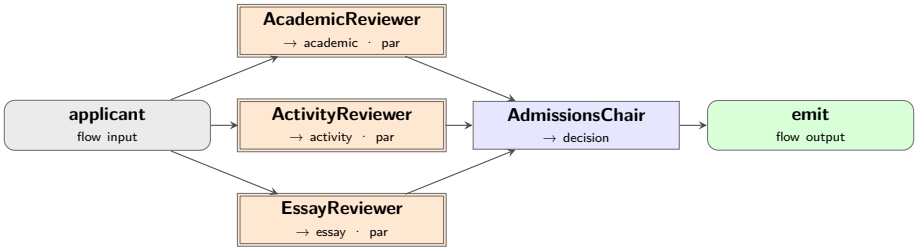
Terminal

```
$ algo run P22_admissions_panel.algo --arg "applicant=Rank 340 in entrance exam, 87 percent in class 12, led a robotics club, strong
↪  essay"

▶ flow main  run=485f3b9b  backend=mock
    applicant = 'Rank 340 in entrance exam, 87 percent in class 12, led a robotics club, strong essay'
  → AcademicReviewer   conf=0.64  0.82  ⚠ ESCALATED to human (conf < 0.80)
  → EssayReviewer      conf=0.72  0.61  ⚠ ESCALATED to human (conf < 0.80)
  → ActivityReviewer   conf=0.74  0.74  ⚠ ESCALATED to human (conf < 0.80)
  → AdmissionsChair    conf=0.82  «AdmissionsChair #8785» Recommend admit, waitlist or decline for Rank 34
■ emitted:
    • «AdmissionsChair #8785» Recommend admit, waitlist or decline for Rank 340 in entrance exam, 87…
    • 0.82
    • 0.74
    • 0.61
■ 4 agent calls · ~410 tokens · 0 retries · 3 escalated · 0 blocked · audit → runs/P22_admissions_panel.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 410 tokens · 0 retries · 3 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	AcademicReviewer	sonnet-5-5	0.64	101	0.82 **human**
2	EssayReviewer	sonnet-5-5	0.72	101	0.61 **human**
3	ActivityReviewer	sonnet-5-5	0.74	100	0.74 **human**
4	AdmissionsChair	sonnet-5-5	0.82	108	«AdmissionsChair #8785» Recommend admit, waitlist or decline for …

Result: [«AdmissionsChair #8785» Recommend admit, waitlist or decline for Rank 340 in entrance exam, 87…, 0.82, 0.74, 0.61]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "run a fair admissions panel for an applicant"

AlgoLang: matched **Admissions screening** (`education.admissions`), Education, pattern `committee`, match score 0.47. Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: AcademicReviewer, StatementReviewer, PotentialReviewer, AdmissionsPanel, Letter.

Matched a pack in the same sector. Other matches offered: `hr.resume_screening`, `retail.return_fraud`, `research.grant_review`.

#	Agent	Model	Conf.	Tok.	Output
1	AcademicReviewer	sonnet-5-5	0.71	96	«AcademicReviewer #1072» Assess academic record in run a fair adm…
2	PotentialReviewer	sonnet-5-5	0.77	99	«PotentialReviewer #7412» Assess potential and achievements in ru…
3	StatementReviewer	sonnet-5-5	0.83	98	«StatementReviewer #1392» Assess the statement of purpose in run …
4	AdmissionsPanel	sonnet-5-5	0.93	162	0.87
5	Letter	sonnet-5-5	0.93	160	«Letter #5909» Write a panel summary from «AcademicReviewer #1072…

Final answer: `[«Letter #5909» Write a panel summary from «AcademicReviewer #1072» Assess academic re…, 0.87]` (5 agent calls · 615 tokens · 0 retries · 0 escalated · 0 blocked)

23 CO-PO Mapper

Education par for P23_co_po_mapper.algo

One mapper per outcome, then a matrix.

Input

`outcomes` `list` Design circuits, Analyse signals, Work in teams

Output type: `text, list`

AlgoLang code

```
// CO-PO Mapper: par for fans one agent out over a list.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent OutcomeMapper(item: text) -> text {
   role "OBE coordinator"
   goal "Map the course outcome {item} to programme outcomes with a strength from 1 to 3"
   model fast
}

agent MatrixBuilder(results: list) -> text {
   role "OBE coordinator"
   goal "Build a CO-PO matrix from {results}"
   model smart
}

flow main(outcomes: list) {
   results = par for it in outcomes { OutcomeMapper(it) }
   report = MatrixBuilder(results)
   emit report, results
}
```

Output 1 – Command line

Terminal

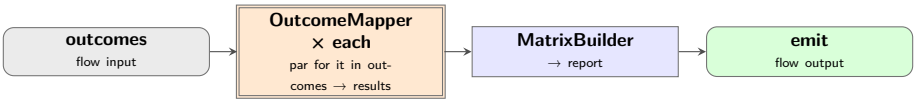
```
$ algo run P23_co_po_mapper.algo --arg "outcomes=Design circuits, Analyse signals, Work in teams"

▶ flow main  run=0c0ceebd  backend=mock
    outcomes = 'Design circuits, Analyse signals, Work in teams'
  → OutcomeMapper      conf=0.84  «OutcomeMapper #7443» Map the course outcome Design circuits, Analyse si
  → MatrixBuilder      conf=0.76  «MatrixBuilder #1654» Build a CO-PO matrix from «OutcomeMapper #7443» Ma
■ emitted:
```

```
    • «MatrixBuilder #1654» Build a CO-PO matrix from «OutcomeMapper #7443» Map the course outcome…
    • «OutcomeMapper #7443» Map the course outcome Design circuits, Analyse signals, Work in teams…
■ 2 agent calls · ~199 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P23_co_po_mapper.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 199 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`OutcomeMapper`	haiku-5-5	0.84	100	«OutcomeMapper #7443» Map the course outcome Design circuits, Ana…
2	`MatrixBuilder`	sonnet-5-5	0.76	99	«MatrixBuilder #1654» Build a CO-PO matrix from «OutcomeMapper #7…

Result: `[«MatrixBuilder #1654» Build a CO-PO matrix from «OutcomeMapper #7443» Map the course outcome…, [«OutcomeMapper #7443» Map the course outcome Design …`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "map course outcomes to programme outcomes"

AlgoLang: matched **Course outcome mapping** (`education.outcome_mapping`), Education, pattern `fan`, match score 0.85. Governance: human review below confidence 0.70. Agents: Outcomes, Mapper, Alignment, Matrix.

Matched a pack in the same sector. Other matches offered: `education.ai_tutor, education.question_bank, education.essay_feedback`.

#	Agent	Model	Conf.	Tok.	Output
1	`Outcomes`	haiku-5-5	0.75	95	[item 1 for: List the course outcomes of map course outcomes to p…
2	`Mapper`	sonnet-5-5	0.87	107	«Mapper #8618» Map item 1 for: List the course outcomes of map co…
3	`Mapper`	sonnet-5-5	0.75	107	«Mapper #8212» Map item 3 for: List the course outcomes of map co…
4	`Mapper`	sonnet-5-5	0.86	107	«Mapper #9089» Map item 2 for: List the course outcomes of map co…
5	`Alignment`	sonnet-5-5	0.69	151	0.60 **human**
6	`Matrix`	sonnet-5-5	0.71	155	«Matrix #6560» Write a CO-PO mapping summary from «Mapper #8618» …

Final answer: `[«Matrix #6560» Write a CO-PO mapping summary from «Mapper #8618» Map item 1 for: List…, 0.60]` (6 agent calls · 722 tokens · 0 retries · 1 escalated · 0 blocked)

24 Literature Review

Research par for P24_literature_review.algo

Papers are read in parallel with a search tool, then synthesised.

Input

`papers` `list` Attention is all you need, ReAct, Toolformer

Output type: `text, list`

AlgoLang code

```
// Literature Review: par for fans one agent out over a list.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"
tool web.search

agent PaperReader(item: text) -> text {
   role "Research assistant"
   goal "Summarise the method and finding of the paper: {item}"
   uses web.search
   model fast
}

agent ReviewWriter(results: list) -> text {
   role "Research lead"
   goal "Write a short literature review from {results}"
   model smart
}

flow main(papers: list) {
   results = par for it in papers { PaperReader(it) }
   report = ReviewWriter(results)
   emit report, results
}
```

Output 1 – Command line

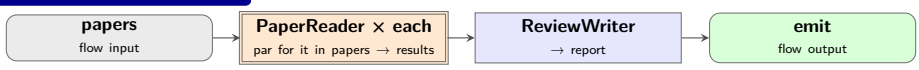
Terminal

```
$ algo run P24_literature_review.algo --arg "papers=Attention is all you need, ReAct, Toolformer"

▶ flow main  run=d5f85a8e  backend=mock
    papers = 'Attention is all you need, ReAct, Toolformer'
  → PaperReader        conf=0.66  «PaperReader #0581» Summarise the method and finding of the paper: Atten
  → ReviewWriter       conf=0.75  «ReviewWriter #9021» Write a short literature review from «PaperReader #
■ emitted:
    • «ReviewWriter #9021» Write a short literature review from «PaperReader #0581» Summarise the…
    • «PaperReader #0581» Summarise the method and finding of the paper: Attention is all you ne…
■ 2 agent calls · ~196 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P24_literature_review.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 196 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	PaperReader	haiku-5-5	0.66	95	«PaperReader #0581» Summarise the method and finding of the paper…
2	ReviewWriter	sonnet-5-5	0.75	101	«ReviewWriter #9021» Write a short literature review from «PaperR…

Result: `[«ReviewWriter #9021» Write a short literature review from «PaperReader #0581» Summarise the…, [«PaperReader #0581» Summarise the method and finding …`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "review a set of research papers"

AlgoLang: matched **Literature review** (`research.literature_review`), Research & Academia, pattern fan, match score 0.54. Governance: human review below confidence 0.80. Agents: Searcher, Reader, Coverage, Writer.

Matched a pack in the same sector. Other matches offered: `research.grant_review`, `research.peer_review`, `research.ethics_check`.

#	Agent	Model	Conf.	Tok.	Output
1	Searcher	haiku-5-5	0.64	92	[item 1 for: Find 6 key recent papers on review a set of research…
2	Reader	sonnet-5-5	0.63	102	«Reader #8577» Summarise method, data and findings of item 1 for:…
3	Reader	sonnet-5-5	0.78	102	«Reader #7160» Summarise method, data and findings of item 4 for:…
4	Reader	sonnet-5-5	0.69	102	«Reader #2231» Summarise method, data and findings of item 3 for:…
5	Reader	sonnet-5-5	0.92	102	«Reader #0372» Summarise method, data and findings of item 2 for:…
6	Coverage	sonnet-5-5	0.79	173	0.82 **human**
7	Writer	sonnet-5-5	0.87	176	«Writer #0468» Write a related-work section from «Reader #8577» S…

Final answer: `[«Writer #0468» Write a related-work section from «Reader #8577» Summarise method, dat…, 0.82]` (7 agent calls · 849 tokens · 0 retries · 1 escalated · 0 blocked)

25 Hypothesis Critic

Research fan-out P25_hypothesis_critic.algo

Three critics in parallel, one editor.

Input

`hypothesis` `text` Daily 20 minute walks reduce exam stress in first-year students

Output type: `text`

AlgoLang code

```
// Hypothesis Critic: workers run in parallel, one agent merges.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent MethodCritic(hypothesis: text) -> text {
   role "Methodologist"
   goal "Critique how {hypothesis} could be tested"
   model smart
}

agent StatsCritic(hypothesis: text) -> text {
   role "Statistician"
   goal "List sample size and bias issues for {hypothesis}"
   model smart
}

agent EthicsCritic(hypothesis: text) -> text {
   role "Ethics reviewer"
   goal "List ethical concerns for studying {hypothesis}"
   model fast
}

agent EditorialSummary(method: text, stats: text, ethics: text) -> text {
   role "Journal editor"
   goal "Combine {method}, {stats} and {ethics} into one review"
   model smart
}

flow main(hypothesis: text) {
   [method, stats, ethics] = par {
      MethodCritic(hypothesis),
      StatsCritic(hypothesis),
      EthicsCritic(hypothesis)
   }
   review = EditorialSummary(method, stats, ethics)
```

```
    emit review
}
```

Output 1 – Command line

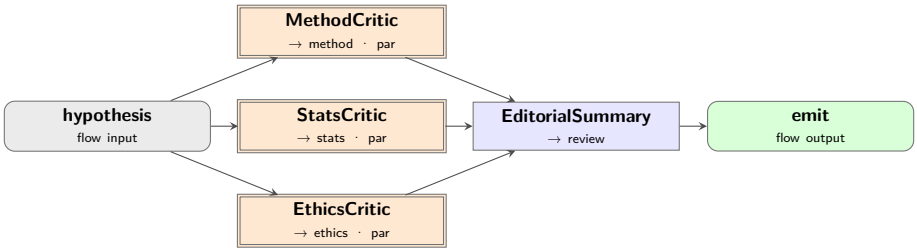
Terminal

```
$ algo run P25_hypothesis_critic.algo --arg "hypothesis=Daily 20 minute walks reduce exam stress in first-year students"

▶ flow main  run=e8e84297  backend=mock
    hypothesis = 'Daily 20 minute walks reduce exam stress in first-year students'
  → MethodCritic       conf=0.63  «MethodCritic #8960» Critique how Daily 20 minute walks reduce exam stre
  → EthicsCritic       conf=0.78  «EthicsCritic #3884» List ethical concerns for studying Daily 20 minute
  → StatsCritic        conf=0.80  «StatsCritic #4781» List sample size and bias issues for Daily 20 minute
  → EditorialSummary   conf=0.94  «EditorialSummary #5161» Combine «MethodCritic #8960» Critique how Daily
■ emitted:
    • «EditorialSummary #5161» Combine «MethodCritic #8960» Critique how Daily 20 minute walks reduce…
■ 4 agent calls · ~426 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P25_hypothesis_critic.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 426 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	MethodCritic	sonnet-5-5	0.63	92	«MethodCritic #8960» Critique how Daily 20 minute walks reduce ex…
2	StatsCritic	sonnet-5-5	0.80	94	«StatsCritic #4781» List sample size and bias issues for Daily 20…
3	EthicsCritic	haiku-5-5	0.78	94	«EthicsCritic #3884» List ethical concerns for studying Daily 20 …
4	EditorialSummary	sonnet-5-5	0.94	146	«EditorialSummary #5161» Combine «MethodCritic #8960» Critique ho…

Result: [«EditorialSummary #5161» Combine «MethodCritic #8960» Critique how Daily 20 minute walks reduce…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "first-round peer review of a research hypothesis"

AlgoLang: matched **Manuscript peer review** (research.peer_review), Research & Academia, pattern fan, match score 0.82. Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: Criteria, Reviewer, Editor, Letter.

Matched a pack in the same sector. Other matches offered: research.literature_review, research.grant_review, research.ethics_check.

#	Agent	Model	Conf.	Tok.	Output
1	Criteria	haiku-5-5	0.62	101	[item 1 for: List the review criteria that matter most for first-…
2	Reviewer	sonnet-5-5	0.91	116	«Reviewer #7245» Review item 1 for: List the review criteria that…
3	Reviewer	sonnet-5-5	0.78	116	«Reviewer #7594» Review item 2 for: List the review criteria that…
4	Reviewer	sonnet-5-5	0.72	116	«Reviewer #6864» Review item 3 for: List the review criteria that…
5	Editor	sonnet-5-5	0.89	153	0.70 **human**
6	Letter	sonnet-5-5	0.81	154	«Letter #7929» Draft a decision letter from «Reviewer #7245» Revi…

Final answer: [«Letter #7929» Draft a decision letter from «Reviewer #7245» Review item 1 for: List …, 0.70] (6 agent calls · 756 tokens · 0 retries · 1 escalated · 0 blocked)

26 Grievance Router

Government router P26_grievance_router.algo

Urgent grievances get a priority desk and service-level target.

Input

`grievance`	`text`	No water supply in our ward for six days, elderly residents are suffering

Output type: `score, text`

AlgoLang code

```
// Grievance Router: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent UrgencyScorer(grievance: text) -> score {
   role "Grievance officer"
   goal "Score the urgency of {grievance} from 0 to 1"
   model fast
}

agent PriorityDesk(grievance: text) -> text {
   role "Department officer"
   goal "Write a priority action note with a 24-hour target for {grievance}"
   model smart
}

agent StandardDesk(grievance: text) -> text {
   role "Department officer"
   goal "Write a standard acknowledgement and a 7-day target for {grievance}"
   model fast
}

flow main(grievance: text) {
   urgency = UrgencyScorer(grievance)

   if urgency >= 0.7 {
      priority = PriorityDesk(grievance)
      emit urgency, priority
   } else {
      standard = StandardDesk(grievance)
      emit urgency, standard
   }
}
```

Output 1 – Command line

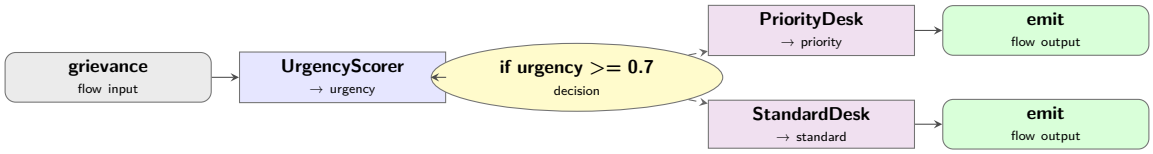
Terminal

```
$ algo run P26_grievance_router.algo --arg "grievance=No water supply in our ward for six days, elderly residents are suffering"

▶ flow main  run=2978510e  backend=mock
    grievance = 'No water supply in our ward for six days, elderly residents are suffering'
  → UrgencyScorer      conf=0.86  0.7
  → PriorityDesk       conf=0.68  «PriorityDesk #0537» Write a priority action note with a 24-hour target
■ emitted:
    • 0.7
    • «PriorityDesk #0537» Write a priority action note with a 24-hour target for No water supply…
■ 2 agent calls · ~199 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P26_grievance_router.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 199 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`UrgencyScorer`	haiku-5-5	0.86	97	0.70
2	`PriorityDesk`	sonnet-5-5	0.68	102	«PriorityDesk #0537» Write a priority action note with a 24-hour …

Result: `[0.70, «PriorityDesk #0537» Write a priority action note with a 24-hour target for No water supply…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "route a citizen grievance to the right desk"

AlgoLang: matched **Citizen grievance routing** (`government.grievance_routing`), Government & Public Services, pattern `router`, match score 0.75. Governance: human review below confidence 0.80. Agents: Urgency, PriorityCell, Department, Clarity.

Matched a pack in the same sector. Other matches offered: `government.public_consultation`, `government.rti_response`, `banking.complaints`.

#	Agent	Model	Conf.	Tok.	Output
1	`Urgency`	haiku-5-5	0.64	99	0.74
2	`PriorityCell`	sonnet-5-5	0.77	103	«PriorityCell #3733» Assign to the priority cell with an action p…
3	`Clarity`	sonnet-5-5	0.75	120	0.64 **human**
4	`PriorityCell`	sonnet-5-5	0.77	103	«PriorityCell #3733» Assign to the priority cell with an action p…

Final answer: `[«PriorityCell #3733» Assign to the priority cell with an action plan: route a citizen griev… [kb.search: knowledge base 'kb' is empty], 0.64]` (4 agent calls · 425 tokens · 0 retries · 1 escalated · 0 blocked)

27 Welfare Pre-check

Government pipeline `P27_welfare_pre_check.algo`

Rules, documents, and a citizen letter. Religion and caste are forbidden inputs.

Input

`household` `text` Family of five, monthly income 9000, owns no land, two school-age children

Output type: `text`

AlgoLang code

```
// Welfare Pre-check: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 4000
   retry 1
   forbid caste, religion
   require human when confidence < 0.8
}

agent EligibilityRules(household: text) -> text {
   role "Scheme officer"
   goal "Check {household} against the income and household rules of the scheme"
   model smart
}

agent DocumentChecklist(rules_result: text) -> text {
   role "Scheme officer"
```

```
   goal "List documents still needed given {rules_result}"
   model fast
}

agent CitizenLetter(checklist: text) -> text {
   role "Citizen services writer"
   goal "Write a polite letter explaining {checklist} in simple language"
   model fast
}

flow main(household: text) {
   rules_result = EligibilityRules(household)
   checklist = DocumentChecklist(rules_result)
   letter = CitizenLetter(checklist)
   emit letter
}
```

Output 1 – Command line

Terminal

```
$ algo run P27_welfare_pre_check.algo --arg "household=Family of five, monthly income 9000, owns no land, two school-age children"

▶ flow main  run=ab973e36  backend=mock
    household = 'Family of five, monthly income 9000, owns no land, two school-age children'
  → EligibilityRules   conf=0.92  «EligibilityRules #2347» Check Family of five, monthly income 9000, owns
  → DocumentChecklist  conf=0.63  «DocumentChecklist #1085» List documents still needed given «Eligibility  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → CitizenLetter      conf=0.79  «CitizenLetter #0564» Write a polite letter explaining «DocumentChecklis  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
■ emitted:
    • «CitizenLetter #0564» Write a polite letter explaining «DocumentChecklist #1085» List docume…
■ 3 agent calls · ~314 tokens · 0 retries · 2 escalated · 0 blocked · audit → runs/P27_welfare_pre_check.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 314 tokens · 0 retries · 2 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	EligibilityRules	sonnet-5-5	0.92	103	«EligibilityRules #2347» Check Family of five, monthly income 900…
2	DocumentChecklist	haiku-5-5	0.63	102	«DocumentChecklist #1085» List documents still needed given «Elig… human
3	CitizenLetter	haiku-5-5	0.79	109	«CitizenLetter #0564» Write a polite letter explaining «DocumentC… human

Result: [«CitizenLetter #0564» Write a polite letter explaining «DocumentChecklist #1085» List docume…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "pre-check a household for a welfare scheme"

AlgoLang: matched **Welfare scheme eligibility** (`government.welfare_eligibility`), Government & Public Services, pattern `pipe`, match score 0.56.

Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: Extractor, Rules, Eligibility, Notice.

Matched a pack in the same sector. Other matches offered: `agriculture.scheme_eligibility`, `markets.research_review`, `supplychain.customs_documents`.

#	Agent	Model	Conf.	Tok.	Output
1	Extractor	sonnet-5-5	0.68	100	«Extractor #4984» Extract household and income details from pre-c…
2	Rules	sonnet-5-5	0.80	120	«Rules #5680» Check «Extractor #4984» Extract household and incom…
3	Eligibility	sonnet-5-5	0.75	107	0.93 human
4	Notice	sonnet-5-5	0.66	111	«Notice #0060» Draft an eligibility notice from «Rules #5680» Che…

Final answer: [«Notice #0060» Draft an eligibility notice from «Rules #5680» Check «Extractor #4984»…, 0.93] (4 agent calls · 438

tokens · 0 retries · 1 escalated · 0 blocked)

28 NDA Triage

Legal router P28_nda_triage.algo

Low-risk NDAs are approved; risky ones go to a lawyer.

Input

nda	text	Mutual NDA, 5 year term, unlimited liability, governed by foreign law

Output type: `score, text`

AlgoLang code

```
// NDA Triage: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent NDARiskScorer(nda: text) -> score {
   role "Contracts counsel"
   goal "Score the risk of {nda} against a standard playbook from 0 to 1"
   model fast
}

agent LawyerReview(nda: text) -> text {
   role "Contracts counsel"
   goal "List the clauses in {nda} that need a lawyer and why"
   model smart
}

agent StandardApproval(nda: text) -> text {
   role "Contracts counsel"
   goal "Write approval notes for {nda}, with any minor edits"
   model fast
}

flow main(nda: text) {
   risk = NDARiskScorer(nda)

   if risk >= 0.5 {
      review = LawyerReview(nda)
      emit risk, review
   } else {
      approval = StandardApproval(nda)
      emit risk, approval
   }
}
```

Output 1 – Command line

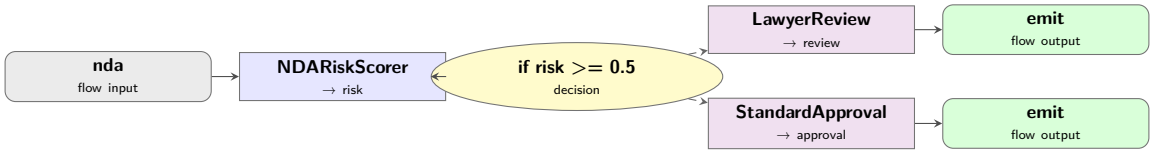
Terminal

```
$ algo run P28_nda_triage.algo --arg "nda=Mutual NDA, 5 year term, unlimited liability, governed by foreign law"

▶ flow main  run=fffa5e8f  backend=mock
    nda = 'Mutual NDA, 5 year term, unlimited liability, governed by foreign law'
  → NDARiskScorer      conf=0.82  0.58
  → LawyerReview       conf=0.88  «LawyerReview #6537» List the clauses in Mutual NDA, 5 year term, unlimi
■ emitted:
    • 0.58
    • «LawyerReview #6537» List the clauses in Mutual NDA, 5 year term, unlimited liability, gove…
■ 2 agent calls · ~201 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P28_nda_triage.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 201 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`NDARiskScorer`	haiku-5-5	0.82	102	0.58
2	`LawyerReview`	sonnet-5-5	0.88	99	«LawyerReview #6537» List the clauses in Mutual NDA, 5 year term,…

Result: `[0.58, «LawyerReview #6537» List the clauses in Mutual NDA, 5 year term, unlimited liability, gove…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "triage an NDA against the legal playbook"

AlgoLang: matched **NDA triage** (`legal.nda_triage`), Legal & Compliance, pattern `pipe`, match score 0.94. Governance: human review below confidence 0.80. Agents: Terms, Playbook, Deviation, Redlines.

Matched a pack in the same sector. Other matches offered: `legal.case_research`, `legal.compliance_gap`, `legal.regulatory_change`.

#	Agent	Model	Conf.	Tok.	Output
1	`Terms`	sonnet-5-5	0.64	90	«Terms #2278» Extract key terms from triage an NDA against the le…
2	`Playbook`	sonnet-5-5	0.90	112	«Playbook #3608» Compare «Terms #2278» Extract key terms from tri…
3	`Deviation`	sonnet-5-5	0.77	106	0.92 **human**
4	`Redlines`	sonnet-5-5	0.69	106	«Redlines #9029» Suggest redlines for «Playbook #3608» Compare «T…

Final answer: `[«Redlines #9029» Suggest redlines for «Playbook #3608» Compare «Terms #2278» Extract ke…, 0.92]` (4 agent calls · 414 tokens · 0 retries · 1 escalated · 0 blocked)

29 Clause Reviewer

Legal par for P29_clause_reviewer.algo

Each clause is checked in parallel, then summarised.

Input

`clauses` `list` Indemnity, Termination, Limitation of liability

Output type: `text, list`

AlgoLang code

```
// Clause Reviewer: par for fans one agent out over a list.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 5000
   retry 1
   require human when confidence < 0.8
}

agent ClauseChecker(item: text) -> text {
   role "Contracts counsel"
   goal "Review the clause {item} for risk and suggest safer wording"
   model fast
}

agent ContractSummary(results: list) -> text {
   role "Senior counsel"
   goal "Summarise the clause reviews in {results} into a risk list"
   model smart
}

flow main(clauses: list) {
   results = par for it in clauses { ClauseChecker(it) }
   report = ContractSummary(results)
   emit report, results
}
```

Output 1 – Command line

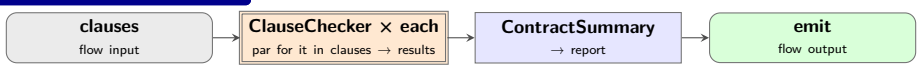
Terminal

```
$ algo run P29_clause_reviewer.algo --arg "clauses=Indemnity, Termination, Limitation of liability"

▶ flow main  run=ad9a877b  backend=mock
    clauses = 'Indemnity, Termination, Limitation of liability'
  → ClauseChecker      conf=0.64  «ClauseChecker #8740» Review the clause Indemnity, Termination, Limitati  ⚠ ESCALATED to human (conf
  ↪  < 0.80)
  → ContractSummary    conf=0.90  «ContractSummary #3110» Summarise the clause reviews in «ClauseChecker #
■ emitted:
    • «ContractSummary #3110» Summarise the clause reviews in «ClauseChecker #8740» Review the claus…
    • «ClauseChecker #8740» Review the clause Indemnity, Termination, Limitation of liability for …
■ 2 agent calls · ~200 tokens · 0 retries · 1 escalated · 0 blocked · audit → runs/P29_clause_reviewer.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 2 agent calls · 200 tokens · 0 retries · 1 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	ClauseChecker	haiku-5-5	0.64	95	«ClauseChecker #8740» Review the clause Indemnity, Termination, L… human
2	ContractSummary	sonnet-5-5	0.90	105	«ContractSummary #3110» Summarise the clause reviews in «ClauseCh…

Result: [«ContractSummary #3110» Summarise the clause reviews in «ClauseChecker #8740» Review the claus…, [«ClauseChecker #8740» Review the clause Indemnity,…

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "review each clause of a contract"

AlgoLang: matched **Contract clause review** (`legal.contract_review`), Legal & Compliance, pattern fan, match score 1.01. Governance: human review below confidence 0.90. Agents: Clauses, Analyst, Risk, Summary.

Matched a pack in the same sector. Other matches offered: `legal.case_research`, `legal.litigation_risk`, `insurance.underwriting`.

#	Agent	Model	Conf.	Tok.	Output
1	Clauses	haiku-5-5	0.66	91	[item 1 for: List every clause in review each clause of a contrac…
2	Analyst	sonnet-5-5	0.83	102	«Analyst #8710» Flag risks and missing protections in item 1 for:…
3	Analyst	sonnet-5-5	0.74	102	«Analyst #5459» Flag risks and missing protections in item 3 for:…
4	Analyst	sonnet-5-5	0.81	102	«Analyst #2957» Flag risks and missing protections in item 2 for:…
5	Analyst	sonnet-5-5	0.91	102	«Analyst #3283» Flag risks and missing protections in item 4 for:…
6	Risk	sonnet-5-5	0.70	174	0.82 **human**
7	Summary	sonnet-5-5	0.89	177	«Summary #3489» Write a contract risk summary from «Analyst #8710…

Final answer: [«Summary #3489» Write a contract risk summary from «Analyst #8710» Flag risks and miss…, 0.82] (7 agent calls · 850 tokens · 0 retries · 1 escalated · 0 blocked)

30 Fair CV Screening

HR committee P30_fair_cv_screening.algo

Three scorers and a chair; protected attributes are forbidden.

Input

`cv`	`text`	5 years Python and SQL, built churn model, MSc statistics
`role_title`	`text`	Data Analyst

Output type: `text, score, score, score`

AlgoLang code

```
// Fair CV Screening: a scoring committee and a chair.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

policy {
   budget tokens 4000
   retry 1
   forbid gender, caste, religion
   require human when confidence < 0.8
}

agent SkillsScorer(cv: text, role_title: text) -> score {
   role "Technical recruiter"
   goal "Score the skills in {cv} against {role_title} from 0 to 1"
   model smart
}

agent ExperienceScorer(cv: text, role_title: text) -> score {
   role "Technical recruiter"
   goal "Score the experience in {cv} for {role_title} from 0 to 1"
   model smart
}

agent ProjectScorer(cv: text, role_title: text) -> score {
   role "Technical recruiter"
   goal "Score the projects in {cv} for {role_title} from 0 to 1"
   model smart
```

```
}

agent HiringChair(cv: text, role_title: text, skills: score, experience: score, projects: score) -> text {
   role "Hiring manager"
   goal "Recommend shortlist, hold or reject for {cv} and {role_title} using only job-related evidence"
   model smart
}

flow main(cv: text, role_title: text) {
   [skills, experience, projects] = par {
      SkillsScorer(cv, role_title),
      ExperienceScorer(cv, role_title),
      ProjectScorer(cv, role_title)
   }
   recommendation = HiringChair(cv, role_title, skills, experience, projects)
   emit recommendation, skills, experience, projects
}
```

Output 1 – Command line

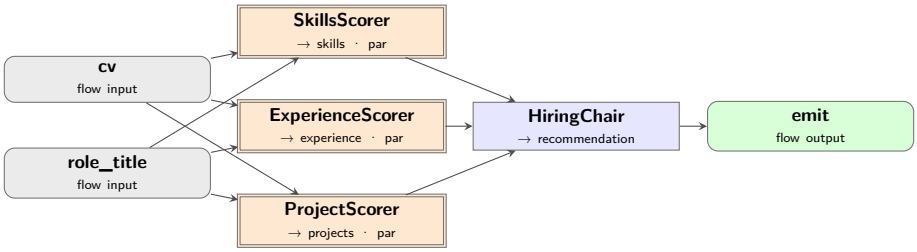
Terminal

```
$ algo run P30_fair_cv_screening.algo --arg "cv=5 years Python and SQL, built churn model, MSc statistics" --arg "role_title=Data
↪  Analyst"

▶ flow main  run=b3786a89  backend=mock
    cv = '5 years Python and SQL, built churn model, MSc statistics'
    role_title = 'Data Analyst'
  → SkillsScorer       conf=0.68  0.9  ⚠ ESCALATED to human (conf < 0.80)
  → ProjectScorer      conf=0.76  0.56  ⚠ ESCALATED to human (conf < 0.80)
  → ExperienceScorer   conf=0.78  0.87  ⚠ ESCALATED to human (conf < 0.80)
  → HiringChair        conf=0.86  «HiringChair #9054» Recommend shortlist, hold or reject for 5 years Pyth
■ emitted:
    • «HiringChair #9054» Recommend shortlist, hold or reject for 5 years Python and SQL, built …
    • 0.9
    • 0.87
    • 0.56
■ 4 agent calls · ~400 tokens · 0 retries · 3 escalated · 0 blocked · audit → runs/P30_fair_cv_screening.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 400 tokens · 0 retries · 3 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	SkillsScorer	sonnet-5-5	0.68	98	0.90 **human**
2	ProjectScorer	sonnet-5-5	0.76	98	0.56 **human**
3	ExperienceScorer	sonnet-5-5	0.78	98	0.87 **human**
4	HiringChair	sonnet-5-5	0.86	106	«HiringChair #9054» Recommend shortlist, hold or reject for 5 yea…

Result: [«HiringChair #9054» Recommend shortlist, hold or reject for 5 years Python and SQL, built …, 0.90, 0.87, 0.56]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "screen a CV fairly for a job"

AlgoLang: matched **Resume screening** (`hr.resume_screening`), HR & Workforce, pattern `committee`, match score 0.68. Governance: human review below confidence 0.90, forbids gender, caste, religion, nationality. Agents: SkillsReviewer, ExperienceReviewer, PotentialReviewer, HiringPanel, Note.

Matched a pack in the same sector. Other matches offered: `hr.job_description`, `realestate.bid_comparison`, `hr.interview_kit`.

#	Agent	Model	Conf.	Tok.	Output
1	SkillsReviewer	sonnet-5-5	0.95	92	«SkillsReviewer #5087» Assess skills against the job in screen a …
2	ExperienceReviewer	sonnet-5-5	0.90	92	«ExperienceReviewer #6562» Assess relevant experience in screen a…
3	PotentialReviewer	sonnet-5-5	0.70	91	«PotentialReviewer #6134» Assess growth potential in screen a CV …
4	HiringPanel	sonnet-5-5	0.69	150	0.63 **human**
5	Note	sonnet-5-5	0.78	149	«Note #4927» Write a shortlist note from «SkillsReviewer #5087» A…

Final answer: `[«Note #4927» Write a shortlist note from «SkillsReviewer #5087» Assess skills again…, 0.63]` (5 agent calls · 574 tokens · 0 retries · 1 escalated · 0 blocked)

31 Interview Builder

HR pipeline P31_interview_builder.algo

Competencies, questions, then a scoring rubric.

Input

`job_title` `text` Backend Engineer

Output type: `text`

AlgoLang code

```
// Interview Builder: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent CompetencyMap(job_title: text) -> text {
   role "HR business partner"
   goal "List the five core competencies for a {job_title}"
   model smart
}

agent QuestionWriter(competencies: text) -> text {
   role "Interviewer"
   goal "Write one behavioural question for each of {competencies}"
   model smart
}

agent RubricBuilder(questions: text) -> text {
   role "Interviewer"
   goal "Write a 1-to-5 scoring rubric for {questions}"
   model fast
}

flow main(job_title: text) {
   competencies = CompetencyMap(job_title)
   questions = QuestionWriter(competencies)
   rubric = RubricBuilder(questions)
   emit rubric
}
```

Output 1 – Command line

Terminal

```
$ algo run P31_interview_builder.algo --arg "job_title=Backend Engineer"

▶ flow main  run=67b02eb9  backend=mock
    job_title = 'Backend Engineer'
  → CompetencyMap      conf=0.63  «CompetencyMap #6442» List the five core competencies for a Backend Engi
  → QuestionWriter     conf=0.89  «QuestionWriter #6419» Write one behavioural question for each of «Compe
  → RubricBuilder      conf=0.64  «RubricBuilder #0361» Write a 1-to-5 scoring rubric for «QuestionWriter
■ emitted:
    • «RubricBuilder #0361» Write a 1-to-5 scoring rubric for «QuestionWriter #6419» Write one beh…
■ 3 agent calls · ~283 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P31_interview_builder.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 283 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	CompetencyMap	sonnet-5-5	0.63	84	«CompetencyMap #6442» List the five core competencies for a Backe…
2	QuestionWriter	sonnet-5-5	0.89	98	«QuestionWriter #6419» Write one behavioural question for each of…
3	RubricBuilder	haiku-5-5	0.64	101	«RubricBuilder #0361» Write a 1-to-5 scoring rubric for «Question…

Result: [«RubricBuilder #0361» Write a 1-to-5 scoring rubric for «QuestionWriter #6419» Write one beh…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "build an interview kit for a role"

AlgoLang: matched **Interview kit builder** (hr.interview_kit), HR & Workforce, pattern fan, match score 0.69. Governance: human review below confidence 0.70. Agents: Competencies, QuestionWriter, Coverage, Kit.

Matched a pack in the same sector. Other matches offered: hr.exit_interviews, energy.energy_audit, realestate.building_code.

#	Agent	Model	Conf.	Tok.	Output
1	Competencies	haiku-5-5	0.76	93	[item 1 for: List the key competencies for build an interview kit…
2	QuestionWriter	sonnet-5-5	0.63	113	«QuestionWriter #6323» Write 3 structured interview questions and…
3	QuestionWriter	sonnet-5-5	0.87	113	«QuestionWriter #3740» Write 3 structured interview questions and…
4	QuestionWriter	sonnet-5-5	0.69	113	«QuestionWriter #1841» Write 3 structured interview questions and…
5	Coverage	sonnet-5-5	0.69	156	0.90 **human**
6	Kit	sonnet-5-5	0.90	159	«Kit #9805» Assemble an interview kit from «QuestionWriter #6323»…

Final answer: [«Kit #9805» Assemble an interview kit from «QuestionWriter #6323» Write 3 structur…, 0.90] (6 agent calls · 747 tokens · 0 retries · 1 escalated · 0 blocked)

32 Exit Interview Insights

HR fan-out P32_exit_interview_insights.algo

Themes, tone and actions run in parallel.

Input

`transcript`	`text`	Left for a better role; liked the team but saw no clear growth path and long hours

Output type: `text`

AlgoLang code

```
// Exit Interview Insights: workers run in parallel, one agent merges.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent ThemeFinder(transcript: text) -> text {
   role "People analyst"
   goal "List the main themes in {transcript}"
   model smart
}

agent SentimentReader(transcript: text) -> text {
   role "People analyst"
   goal "Describe the emotional tone of {transcript}"
   model fast
}

agent ActionIdeas(transcript: text) -> text {
   role "HR business partner"
   goal "Suggest retention actions based on {transcript}"
   model smart
}

agent HRBrief(themes: text, tone: text, actions: text) -> text {
   role "HR lead"
   goal "Write a short brief from {themes}, {tone} and {actions}"
   model smart
}

flow main(transcript: text) {
   [themes, tone, actions] = par {
      ThemeFinder(transcript),
      SentimentReader(transcript),
      ActionIdeas(transcript)
   }
   brief = HRBrief(themes, tone, actions)
   emit brief
}
```

Output 1 – Command line

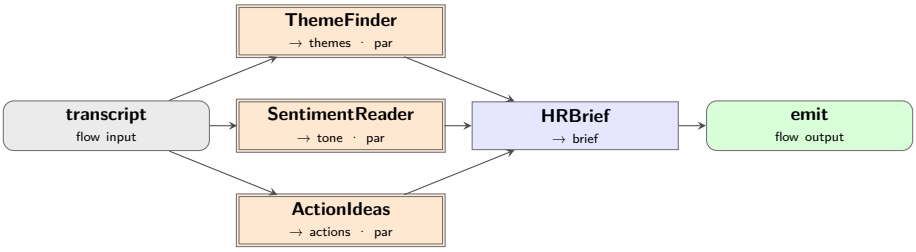
Terminal

```
$ algo run P32_exit_interview_insights.algo --arg "transcript=Left for a better role; liked the team but saw no clear growth path and
↪  long hours"

▶ flow main  run=772c0a1a  backend=mock
    transcript = 'Left for a better role; liked the team but saw no clear growth path and long hours'
  → ThemeFinder        conf=0.68  «ThemeFinder #2351» List the main themes in Left for a better role; like
  → ActionIdeas        conf=0.92  «ActionIdeas #6014» Suggest retention actions based on Left for a better
  → SentimentReader    conf=0.73  «SentimentReader #8646» Describe the emotional tone of Left for a better
  → HRBrief            conf=0.90  «HRBrief #1860» Write a short brief from «ThemeFinder #2351» List the ma
■ emitted:
    • «HRBrief #1860» Write a short brief from «ThemeFinder #2351» List the main themes in L…
■ 4 agent calls · ~439 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P32_exit_interview_insights.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 439 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	ThemeFinder	sonnet-5-5	0.68	96	«ThemeFinder #2351» List the main themes in Left for a better rol…
2	SentimentReader	haiku-5-5	0.73	98	«SentimentReader #8646» Describe the emotional tone of Left for a…
3	ActionIdeas	sonnet-5-5	0.92	100	«ActionIdeas #6014» Suggest retention actions based on Left for a…
4	HRBrief	sonnet-5-5	0.90	145	«HRBrief #1860» Write a short brief from «ThemeFinder #2351» List…

Result: `[«HRBrief #1860» Write a short brief from «ThemeFinder #2351» List the main themes in L…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "analyse an exit interview"

AlgoLang: matched **Exit interview analysis** (`hr.exit_interviews`), HR & Workforce, pattern `fan`, match score 0.74. Governance: human review below confidence 0.70. Agents: Splitter, Analyst, Attrition, Report.

Matched a pack in the same sector. Other matches offered: `hr.interview_kit, energy.grid_fault, cyber.phishing`.

#	Agent	Model	Conf.	Tok.	Output
1	Splitter	haiku-5-5	0.80	93	[item 1 for: Split analyse an exit interview into individual exit…
2	Analyst	sonnet-5-5	0.65	102	«Analyst #7230» Find reasons for leaving in item 1 for: Split ana…
3	Analyst	sonnet-5-5	0.76	102	«Analyst #3926» Find reasons for leaving in item 2 for: Split ana…
4	Analyst	sonnet-5-5	0.79	102	«Analyst #3511» Find reasons for leaving in item 3 for: Split ana…
5	Attrition	sonnet-5-5	0.71	157	0.63
6	Report	sonnet-5-5	0.65	154	«Report #2797» Write a retention report from «Analyst #7230» Find…

Final answer: `[«Report #2797» Write a retention report from «Analyst #7230» Find reasons for leaving…, 0.63]` (6 agent calls · 710 tokens · 0 retries · 0 escalated · 0 blocked)

33 Defect Root Cause

Manufacturing pipeline P33_defect_root_cause.algo

Symptoms, ranked causes, corrective actions.

Input

`defect_report`	`text`	Scratches on 3 percent of aluminium panels from line 2 since the tool change

Output type: `text`

AlgoLang code

```
// Defect Root Cause: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent SymptomParser(defect_report: text) -> text {
   role "Quality engineer"
   goal "Extract the symptoms, timing and location from {defect_report}"
   model fast
}

agent CauseRanker(symptoms: text) -> text {
   role "Process engineer"
   goal "Rank likely root causes for {symptoms} using the 5-why method"
   model smart
}

agent CorrectiveAction(causes: text) -> text {
   role "Quality manager"
   goal "Propose corrective and preventive actions for {causes}"
   model smart
}

flow main(defect_report: text) {
   symptoms = SymptomParser(defect_report)
   causes = CauseRanker(symptoms)
   actions = CorrectiveAction(causes)
   emit actions
}
```

Output 1 – Command line

Terminal

```
$ algo run P33_defect_root_cause.algo --arg "defect_report=Scratches on 3 percent of aluminium panels from line 2 since the tool
↪  change"

▶ flow main  run=1bc8d9d9  backend=mock
    defect_report = 'Scratches on 3 percent of aluminium panels from line 2 since the tool change'
  → SymptomParser      conf=0.88  «SymptomParser #7143» Extract the symptoms, timing and location from Scr
  → CauseRanker        conf=0.84  «CauseRanker #9513» Rank likely root causes for «SymptomParser #7143» Ex
  → CorrectiveAction   conf=0.75  «CorrectiveAction #9148» Propose corrective and preventive actions for «
■ emitted:
    • «CorrectiveAction #9148» Propose corrective and preventive actions for «CauseRanker #9513» Rank…
■ 3 agent calls · ~311 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P33_defect_root_cause.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 311 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	SymptomParser	haiku-5-5	0.88	101	«SymptomParser #7143» Extract the symptoms, timing and location f…
2	CauseRanker	sonnet-5-5	0.84	106	«CauseRanker #9513» Rank likely root causes for «SymptomParser #7…
3	CorrectiveAction	sonnet-5-5	0.75	104	«CorrectiveAction #9148» Propose corrective and preventive action…

Result: `[«CorrectiveAction #9148» Propose corrective and preventive actions for «CauseRanker #9513» Rank…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "find the root cause of a manufacturing defect"

AlgoLang: matched **Root cause analysis** (`manufacturing.root_cause`), Manufacturing, pattern `committee`, match score 0.57. Governance: human review below confidence 0.90. Agents: ProcessEngineer, EquipmentEngineer, QualityEngineer, RCAPanel, Report.

Matched a pack in the same sector. Other matches offered: `manufacturing.quality_inspection`, `manufacturing.supplier_quality`, `manufacturing.sop_compliance`.

#	Agent	Model	Conf.	Tok.	Output
1	ProcessEngineer	sonnet-5-5	0.67	95	«ProcessEngineer #9819» Find process causes of find the root caus…
2	QualityEngineer	sonnet-5-5	0.92	98	«QualityEngineer #1986» Find material and method causes of find t…
3	EquipmentEngineer	sonnet-5-5	0.66	96	«EquipmentEngineer #0642» Find equipment causes of find the root …
4	RCAPanel	sonnet-5-5	0.92	160	0.63
5	Report	sonnet-5-5	0.93	160	«Report #0439» Write a 5-Whys RCA report from «ProcessEngineer #9…

Final answer: `[«Report #0439» Write a 5-Whys RCA report from «ProcessEngineer #9819» Find process ca…, 0.63]` (5 agent calls · 609 tokens · 0 retries · 0 escalated · 0 blocked)

34 Maintenance Planner

Manufacturing router P34_maintenance_planner.algo

High failure risk produces an urgent work order.

Input

`sensor_log`	`text`	Pump P-7 vibration rising 18 percent in 5 days, bearing temperature 82 C

Output type: `score, text`

AlgoLang code

```
// Maintenance Planner: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent FailureRiskScorer(sensor_log: text) -> score {
   role "Reliability engineer"
   goal "Score the failure risk in {sensor_log} from 0 to 1"
   model fast
}

agent UrgentWorkOrder(sensor_log: text) -> text {
   role "Maintenance planner"
   goal "Write an urgent work order for the issue in {sensor_log}"
   model smart
}

agent RoutineSchedule(sensor_log: text) -> text {
   role "Maintenance planner"
   goal "Add the issue in {sensor_log} to the routine schedule with a check date"
   model fast
}

flow main(sensor_log: text) {
```

```
   failure_risk = FailureRiskScorer(sensor_log)

   if failure_risk >= 0.7 {
      urgent = UrgentWorkOrder(sensor_log)
      emit failure_risk, urgent
   } else {
      routine = RoutineSchedule(sensor_log)
      emit failure_risk, routine
   }
}
```

Output 1 – Command line

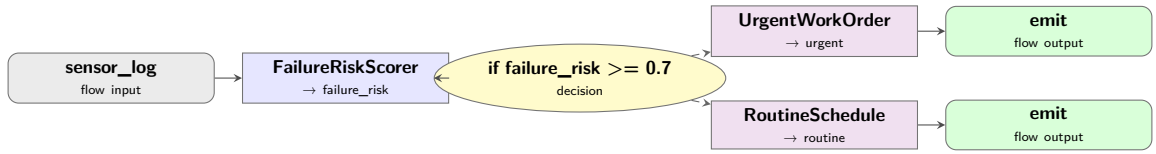
Terminal

```
$ algo run P34_maintenance_planner.algo --arg "sensor_log=Pump P-7 vibration rising 18 percent in 5 days, bearing temperature 82 C"

▶ flow main  run=f3a1341a  backend=mock
    sensor_log = 'Pump P-7 vibration rising 18 percent in 5 days, bearing temperature 82 C'
  → FailureRiskScorer  conf=0.67  0.82
  → UrgentWorkOrder    conf=0.94  «UrgentWorkOrder #0106» Write an urgent work order for the issue in Pump
■ emitted:
    • 0.82
    • «UrgentWorkOrder #0106» Write an urgent work order for the issue in Pump P-7 vibration rising …
■ 2 agent calls · ~198 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P34_maintenance_planner.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 2 agent calls · 198 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	FailureRiskScorer	haiku-5-5	0.67	98	0.82
2	UrgentWorkOrder	sonnet-5-5	0.94	100	«UrgentWorkOrder #0106» Write an urgent work order for the issue …

Result: [0.82, «UrgentWorkOrder #0106» Write an urgent work order for the issue in Pump P-7 vibration rising …]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "plan maintenance from machine sensor data"

AlgoLang: matched **Predictive maintenance** (manufacturing.predictive_maintenance), Manufacturing, pattern fan, match score 0.56. Governance: human review below confidence 0.80. Agents: Assets, Engineer, Urgency, WorkOrders.

Matched a pack in the same sector. Other matches offered: realestate.tenant_queries, retail.pricing_brief, energy.emissions_report.

#	Agent	Model	Conf.	Tok.	Output
1	Assets	haiku-5-5	0.77	92	[item 1 for: List the assets in plan maintenance from machine sen…
2	Engineer	sonnet-5-5	0.76	101	«Engineer #0679» Assess failure risk for item 1 for: List the ass…
3	Engineer	sonnet-5-5	0.74	101	«Engineer #7143» Assess failure risk for item 2 for: List the ass…
4	Engineer	sonnet-5-5	0.89	101	«Engineer #9772» Assess failure risk for item 4 for: List the ass…
5	Engineer	sonnet-5-5	0.72	101	«Engineer #8322» Assess failure risk for item 3 for: List the ass…
6	Urgency	sonnet-5-5	0.73	175	0.75 **human**
7	WorkOrders	sonnet-5-5	0.64	180	«WorkOrders #2173» Draft prioritised work orders from «Engineer #…

Final answer: [«WorkOrders #2173» Draft prioritised work orders from «Engineer #0679» Assess failure ris…, 0.75] (7 agent calls · 851 tokens · 0 retries · 1 escalated · 0 blocked)

35 Shipment Exception

Supply Chain | pipeline | P35_shipment_exception.algo

Diagnose, find recovery options, tell the customer.

Input

exception	text	Container stuck at port for 5 days, customs hold, delivery due in 3 days

Output type: text

AlgoLang code

```
// Shipment Exception: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent Diagnoser(exception: text) -> text {
   role "Logistics analyst"
   goal "Diagnose the cause and impact of {exception}"
   model smart
}

agent RecoveryOptions(diagnosis: text) -> text {
   role "Supply planner"
   goal "List recovery options with cost and time for {diagnosis}"
   model smart
}

agent CustomerNotice(options: text) -> text {
   role "Customer success lead"
   goal "Write a short, honest customer notice using {options}"
   model fast
}

flow main(exception: text) {
   diagnosis = Diagnoser(exception)
   options = RecoveryOptions(diagnosis)
   notice = CustomerNotice(options)
   emit notice
}
```

Output 1 – Command line

Terminal

```
$ algo run P35_shipment_exception.algo --arg "exception=Container stuck at port for 5 days, customs hold, delivery due in 3 days"

▶ flow main  run=eca0b14f  backend=mock
    exception = 'Container stuck at port for 5 days, customs hold, delivery due in 3 days'
  → Diagnoser          conf=0.73  «Diagnoser #8409» Diagnose the cause and impact of Container stuck at po
  → RecoveryOptions    conf=0.82  «RecoveryOptions #1733» List recovery options with cost and time for «Di
  → CustomerNotice     conf=0.76  «CustomerNotice #7790» Write a short, honest customer notice using «Reco
■ emitted:
    • «CustomerNotice #7790» Write a short, honest customer notice using «RecoveryOptions #1733» Li…
■ 3 agent calls · ~305 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P35_shipment_exception.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 305 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	Diagnoser	sonnet-5-5	0.73	96	«Diagnoser #8409» Diagnose the cause and impact of Container stuc…
2	RecoveryOptions	sonnet-5-5	0.82	103	«RecoveryOptions #1733» List recovery options with cost and time …
3	CustomerNotice	haiku-5-5	0.76	106	«CustomerNotice #7790» Write a short, honest customer notice usin…

Result: `[«CustomerNotice #7790» Write a short, honest customer notice using «RecoveryOptions #1733» Li…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "route shipment exceptions by criticality"

AlgoLang: matched **Shipment exception router** (`supplychain.shipment_exceptions`), Supply Chain & Logistics, pattern `router`, match score 0.78. Governance: human review below confidence 0.80. Agents: Critical, ControlTower, Carrier, Clarity.

Matched a pack in the same sector. Other matches offered: `supplychain.route_disruption`, `supplychain.customs_documents`, `banking.complaints`.

#	Agent	Model	Conf.	Tok.	Output
1	Critical	haiku-5-5	0.91	101	0.67
2	ControlTower	sonnet-5-5	0.63	96	«ControlTower #7414» Draft an expedite plan for: route shipment e…
3	Clarity	sonnet-5-5	0.82	110	0.62
4	ControlTower	sonnet-5-5	0.63	96	«ControlTower #7414» Draft an expedite plan for: route shipment e…

Final answer: `[«ControlTower #7414» Draft an expedite plan for: route shipment exceptions by criticality, 0.62]` (4 agent calls · 403 tokens · 0 retries · 0 escalated · 0 blocked)

36 Supplier Risk Panel

Supply Chain committee P36_supplier_risk_panel.algo

Three risk scores and a procurement decision.

Input

`supplier`	`text`	Single-source resin supplier, 3 late deliveries last quarter, ISO certified

Output type: `text, score, score, score`

AlgoLang code

```
// Supplier Risk Panel: a scoring committee and a chair.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent FinancialRisk(supplier: text) -> score {
   role "Supply risk analyst"
   goal "Score the financial health risk of {supplier} from 0 to 1"
   model smart
}

agent DeliveryRisk(supplier: text) -> score {
   role "Supply risk analyst"
   goal "Score the delivery risk of {supplier} from 0 to 1"
   model smart
}

agent ComplianceRisk(supplier: text) -> score {
   role "Supply risk analyst"
   goal "Score the compliance risk of {supplier} from 0 to 1"
   model smart
}

agent RiskChair(supplier: text, financial: score, delivery: score, compliance: score) -> text {
   role "Procurement head"
   goal "Recommend keep, monitor or replace for {supplier} based on the panel scores"
   model smart
```

```
}

flow main(supplier: text) {
   [financial, delivery, compliance] = par {
      FinancialRisk(supplier),
      DeliveryRisk(supplier),
      ComplianceRisk(supplier)
   }
   decision = RiskChair(supplier, financial, delivery, compliance)
   emit decision, financial, delivery, compliance
}
```

Output 1 – Command line

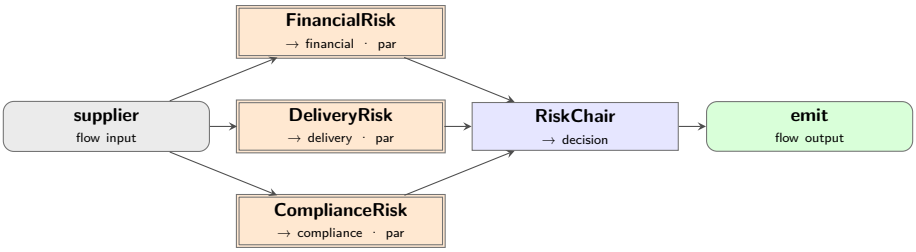
Terminal

```
$ algo run P36_supplier_risk_panel.algo --arg "supplier=Single-source resin supplier, 3 late deliveries last quarter, ISO certified"

▶ flow main  run=eecbaeb4  backend=mock
    supplier = 'Single-source resin supplier, 3 late deliveries last quarter, ISO certified'
  → FinancialRisk      conf=0.83  0.64
  → DeliveryRisk       conf=0.87  0.81
  → ComplianceRisk     conf=0.68  0.65
  → RiskChair          conf=0.72  «RiskChair #2972» Recommend keep, monitor or replace for Single-source r
■ emitted:
    • «RiskChair #2972» Recommend keep, monitor or replace for Single-source resin supplier, 3…
    • 0.64
    • 0.81
    • 0.65
■ 4 agent calls · ~405 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P36_supplier_risk_panel.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 405 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	FinancialRisk	sonnet-5-5	0.83	101	0.64
2	DeliveryRisk	sonnet-5-5	0.87	99	0.81
3	ComplianceRisk	sonnet-5-5	0.68	100	0.65
4	RiskChair	sonnet-5-5	0.72	105	«RiskChair #2972» Recommend keep, monitor or replace for Single-s…

Result: `[«RiskChair #2972» Recommend keep, monitor or replace for Single-source resin supplier, 3…, 0.64, 0.81, 0.65]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "assess the risk of a supplier"

AlgoLang: matched **Supplier risk monitoring** (`supplychain.supplier_risk`), Supply Chain & Logistics, pattern `fan`, match score 0.69. Governance: human review below confidence 0.80. Agents: Splitter, Analyst, Risk, Report.

Matched a pack in the same sector. Other matches offered: `manufacturing.supplier_quality`, `itops.change_review`, `insurance.underwriting`.

#	Agent	Model	Conf.	Tok.	Output
1	Splitter	haiku-5-5	0.95	91	[item 1 for: List the suppliers in assess the risk of a supplier,...
2	Analyst	sonnet-5-5	0.86	106	«Analyst #2094» Assess financial, geographic and compliance risk ...
3	Analyst	sonnet-5-5	0.84	106	«Analyst #6713» Assess financial, geographic and compliance risk ...
4	Analyst	sonnet-5-5	0.88	106	«Analyst #4766» Assess financial, geographic and compliance risk ...
5	Analyst	sonnet-5-5	0.68	106	«Analyst #3510» Assess financial, geographic and compliance risk ...
6	Risk	sonnet-5-5	0.69	175	0.76 **human**
7	Report	sonnet-5-5	0.66	177	«Report #2832» Write a supplier risk report from «Analyst #2094» ...

Final answer: `[«Report #2832» Write a supplier risk report from «Analyst #2094» Assess financial, ge…, 0.76]` (7 agent calls · 867 tokens · 0 retries · 1 escalated · 0 blocked)

37 Product Copy Studio

Retail fan-out P37_product_copy_studio.algo

Three channels write in parallel; a brand checker merges.

Input

`product` `text` Stainless steel insulated bottle, 750 ml, keeps drinks cold 24 hours
Output type: `text`

AlgoLang code

```
// Product Copy Studio: workers run in parallel, one agent merges.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent WebCopy(product: text) -> text {
   role "E-commerce copywriter"
   goal "Write a 50-word web description for {product}"
   model smart
}

agent SocialPost(product: text) -> text {
   role "Social media manager"
   goal "Write a short social post for {product}"
   model fast
}

agent EmailBlurb(product: text) -> text {
   role "CRM marketer"
   goal "Write a two-line email promo for {product}"
   model fast
}

agent BrandChecker(web: text, social: text, email: text) -> text {
   role "Brand manager"
   goal "Check {web}, {social} and {email} for consistent tone and fix any claims"
   model smart
}

flow main(product: text) {
   [web, social, email] = par {
      WebCopy(product),
      SocialPost(product),
      EmailBlurb(product)
   }
   approved = BrandChecker(web, social, email)
   emit approved
}
```

Output 1 – Command line

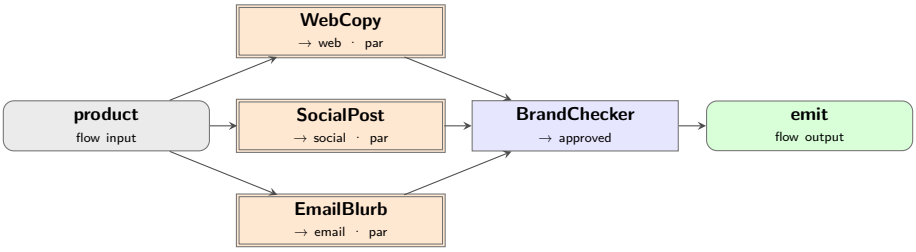
Terminal

```
$ algo run P37_product_copy_studio.algo --arg "product=Stainless steel insulated bottle, 750 ml, keeps drinks cold 24 hours"

▶ flow main  run=85a9163d  backend=mock
    product = 'Stainless steel insulated bottle, 750 ml, keeps drinks cold 24 hours'
  → WebCopy            conf=0.74  «WebCopy #1847» Write a 50-word web description for Stainless steel insu
  → SocialPost         conf=0.85  «SocialPost #6514» Write a short social post for Stainless steel insulat
  → EmailBlurb         conf=0.95  «EmailBlurb #7968» Write a two-line email promo for Stainless steel insu
  → BrandChecker       conf=0.66  «BrandChecker #1893» Check «WebCopy #1847» Write a 50-word web descripti
■ emitted:
    • «BrandChecker #1893» Check «WebCopy #1847» Write a 50-word web description for Stainless st…
■ 4 agent calls · ~435 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P37_product_copy_studio.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 4 agent calls · 435 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`WebCopy`	sonnet-5-5	0.74	97	«WebCopy #1847» Write a 50-word web description for Stainless ste…
2	`EmailBlurb`	haiku-5-5	0.95	94	«EmailBlurb #7968» Write a two-line email promo for Stainless ste…
3	`SocialPost`	haiku-5-5	0.85	95	«SocialPost #6514» Write a short social post for Stainless steel …
4	`BrandChecker`	sonnet-5-5	0.66	149	«BrandChecker #1893» Check «WebCopy #1847» Write a 50-word web de…

Result: `[«BrandChecker #1893» Check «WebCopy #1847» Write a 50-word web description for Stainless st…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "write accurate product descriptions"

AlgoLang: matched **Product description writer** (`retail.product_description`), Retail & E-commerce, pattern `pipe`, match score 0.88. Governance: human review below confidence 0.70. Agents: Features, Writer, Accuracy, Final.

Matched a pack in the same sector. Other matches offered: `hr.job_description`, `retail.review_insights`, `retail.pricing_brief`.

#	Agent	Model	Conf.	Tok.	Output
1	`Features`	sonnet-5-5	0.70	94	«Features #4247» List features and benefits of write accurate pro…
2	`Writer`	sonnet-5-5	0.71	108	«Writer #6383» Write an engaging product description from «Featur…
3	`Accuracy`	sonnet-5-5	0.81	108	0.68
4	`Final`	sonnet-5-5	0.66	109	«Final #1617» Finalise the description «Writer #6383» Write an en…

Final answer: `[«Final #1617» Finalise the description «Writer #6383» Write an engaging product desc…, 0.68]` (4 agent calls · 419 tokens · 0 retries · 0 escalated · 0 blocked)

38 Complaint Reply

Retail router P38_complaint_reply.algo

Very upset customers get a manager-level apology.

Input

`complaint`	`text`	Third time my order arrived damaged. I want a refund and an explanation today!

Output type: `score, text`

AlgoLang code

```
// Complaint Reply: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent AngerScorer(complaint: text) -> score {
   role "Customer care lead"
   goal "Score how upset the customer is in {complaint} from 0 to 1"
   model fast
}

agent ApologyManager(complaint: text) -> text {
   role "Senior care manager"
   goal "Write an empathetic apology with a concrete remedy for {complaint}"
   model smart
}

agent StandardReply(complaint: text) -> text {
   role "Care agent"
   goal "Write a friendly standard reply to {complaint}"
   model fast
}

flow main(complaint: text) {
   anger = AngerScorer(complaint)

   if anger >= 0.7 {
      apology = ApologyManager(complaint)
      emit anger, apology
   } else {
      reply = StandardReply(complaint)
      emit anger, reply
   }
}
```

Output 1 – Command line

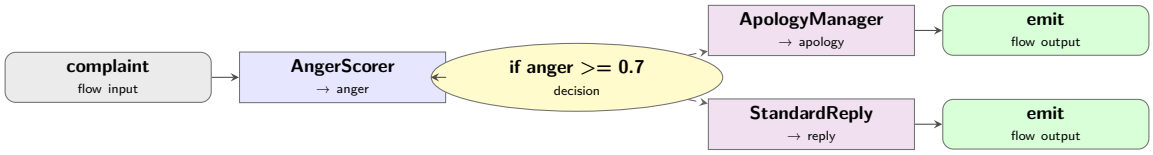
Terminal

```
$ algo run P38_complaint_reply.algo --arg "complaint=Third time my order arrived damaged. I want a refund and an explanation today!"

▶ flow main  run=bde0a73e  backend=mock
    complaint = 'Third time my order arrived damaged. I want a refund and an explanation today!'
  → AngerScorer        conf=0.68  0.63
  → StandardReply      conf=0.68  «StandardReply #1883» Write a friendly standard reply to Third time my o
■ emitted:
    • 0.63
    • «StandardReply #1883» Write a friendly standard reply to Third time my order arrived damaged…
■ 2 agent calls · ~199 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P38_complaint_reply.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 2 agent calls · 199 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	AngerScorer	haiku-5-5	0.68	102	0.63
2	StandardReply	haiku-5-5	0.68	97	«StandardReply #1883» Write a friendly standard reply to Third ti…

Result: `[0.63, «StandardReply #1883» Write a friendly standard reply to Third time my order arrived damaged…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "route retail customer messages and reply to complaints"

AlgoLang: matched **Customer service router** (`retail.customer_service`), Retail & E-commerce, pattern `router`, match score 0.49. Governance: human review below confidence 0.70. Agents: IsOrder, OrderDesk, HelpDesk, Tone.

Matched a pack in the same sector. Other matches offered: `banking.complaints`, `retail.review_insights`, `banking.kyc_review`.

#	Agent	Model	Conf.	Tok.	Output
1	IsOrder	haiku-5-5	0.72	104	0.69
2	OrderDesk	sonnet-5-5	0.76	102	«OrderDesk #8504» Resolve this order or delivery issue: route ret…
3	Tone	sonnet-5-5	0.72	122	0.81

Final answer: `[«OrderDesk #8504» Resolve this order or delivery issue: route retail customer messages a… [kb.search: knowledge base 'kb' is empty], 0.81]` (3 agent calls · 328 tokens · 0 retries · 0 escalated · 0 blocked)

39 Incident Triage

IT Operations pipeline P39_incident_triage.algo

Classify, assess impact, choose the runbook.

Input

`incident`	`text`	Login API returning 500 for all users since 09:40, database connections maxed out

Output type: `text`

AlgoLang code

```
// Incident Triage: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent Classifier(incident: text) -> text {
   role "On-call engineer"
   goal "Classify the type and severity of {incident}"
   model fast
}

agent ImpactAssessor(classification: text) -> text {
   role "Incident commander"
   goal "Estimate the user and business impact given {classification}"
   model smart
}

agent RunbookSelector(impact: text) -> text {
   role "SRE"
   goal "Pick the runbook and first three actions for {impact}"
   model smart
}
```

```
flow main(incident: text) {
   classification = Classifier(incident)
   impact = ImpactAssessor(classification)
   actions = RunbookSelector(impact)
   emit actions
}
```

Output 1 – Command line

Terminal

```
$ algo run P39_incident_triage.algo --arg "incident=Login API returning 500 for all users since 09:40, database connections maxed out"

▶ flow main  run=8d6d1fa9  backend=mock
    incident = 'Login API returning 500 for all users since 09:40, database connections maxed out'
  → Classifier         conf=0.89  «Classifier #5061» Classify the type and severity of Login API returning
  → ImpactAssessor     conf=0.84  «ImpactAssessor #8312» Estimate the user and business impact given «Clas
  → RunbookSelector    conf=0.84  «RunbookSelector #8819» Pick the runbook and first three actions for «Im
■ emitted:
    • «RunbookSelector #8819» Pick the runbook and first three actions for «ImpactAssessor #8312» Es…
■ 3 agent calls · ~304 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P39_incident_triage.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 304 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	Classifier	haiku-5-5	0.89	99	«Classifier #5061» Classify the type and severity of Login API re…
2	ImpactAssessor	sonnet-5-5	0.84	104	«ImpactAssessor #8312» Estimate the user and business impact give…
3	RunbookSelector	sonnet-5-5	0.84	101	«RunbookSelector #8819» Pick the runbook and first three actions …

Result: [«RunbookSelector #8819» Pick the runbook and first three actions for «ImpactAssessor #8312» Es…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "triage a production incident"

AlgoLang: matched **IT incident triage** (itops.incident_triage), Telecom & IT Operations, pattern pipe, match score 0.51. Governance: human review below confidence 0.90. Agents: Classifier, Diagnosis, Priority, Runbook.

Matched a pack in the same sector. Other matches offered: cyber.soc_triage, insurance.claims_triage, manufacturing.safety_incident.

#	Agent	Model	Conf.	Tok.	Output
1	Classifier	sonnet-5-5	0.72	94	«Classifier #2876» Classify service, impact and urgency of triage…
2	Diagnosis	sonnet-5-5	0.80	118	«Diagnosis #3232» Suggest likely causes and checks for «Classifie…
3	Priority	sonnet-5-5	0.94	107	0.55
4	Runbook	sonnet-5-5	0.77	110	«Runbook #9487» Write first-response steps for «Diagnosis #3232» …

Final answer: [«Runbook #9487» Write first-response steps for «Diagnosis #3232» Suggest likely causes…, 0.55] (4 agent calls · 429 tokens · 0 retries · 0 escalated · 0 blocked)

40 Postmortem Writer

IT Operations pipeline P40_postmortem_writer.algo

Timeline, root cause, and a blameless report.

Input

`timeline`	`text`	09:40 alerts fire; 09:55 rollback; 10:10 recovery; cause: connection pool misconfiguration

Output type: `text`

AlgoLang code

```
// Postmortem Writer: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent TimelineCleaner(timeline: text) -> text {
   role "SRE"
   goal "Turn {timeline} into a clean, timestamped timeline"
   model fast
}

agent RootCauseWriter(clean_timeline: text) -> text {
   role "SRE"
   goal "Explain the root cause and contributing factors from {clean_timeline}"
   model smart
}

agent PostmortemWriter(analysis: text) -> text {
   role "Engineering manager"
   goal "Write a blameless postmortem with action items from {analysis}"
   model smart
}

flow main(timeline: text) {
   clean_timeline = TimelineCleaner(timeline)
   analysis = RootCauseWriter(clean_timeline)
   postmortem = PostmortemWriter(analysis)
   emit postmortem
}
```

Output 1 – Command line

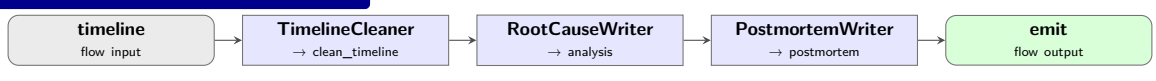
Terminal

```
$ algo run P40_postmortem_writer.algo --arg "timeline=09:40 alerts fire; 09:55 rollback; 10:10 recovery; cause: connection pool
↪  misconfiguration"

▶ flow main  run=e1583cb6  backend=mock
    timeline = '09:40 alerts fire; 09:55 rollback; 10:10 recovery; cause: connection pool misconfiguration'
  → TimelineCleaner    conf=0.63  «TimelineCleaner #2487» Turn 09:40 alerts fire; 09:55 rollback; 10:10 re
  → RootCauseWriter    conf=0.78  «RootCauseWriter #9705» Explain the root cause and contributing factors
  → PostmortemWriter   conf=0.79  «PostmortemWriter #8205» Write a blameless postmortem with action items
■ emitted:
    • «PostmortemWriter #8205» Write a blameless postmortem with action items from «RootCauseWriter #…
■ 3 agent calls · ~310 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P40_postmortem_writer.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 310 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	TimelineCleaner	haiku-5-5	0.63	99	«TimelineCleaner #2487» Turn 09:40 alerts fire; 09:55 rollback; 1…
2	RootCauseWriter	sonnet-5-5	0.78	104	«RootCauseWriter #9705» Explain the root cause and contributing f…
3	PostmortemWriter	sonnet-5-5	0.79	107	«PostmortemWriter #8205» Write a blameless postmortem with action…

Result: `[«PostmortemWriter #8205» Write a blameless postmortem with action items from «RootCauseWriter #…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "write a blameless postmortem"

AlgoLang: matched **Incident postmortem writer** (`itops.postmortem`), Telecom & IT Operations, pattern fan, match score 0.65. Governance: human review below confidence 0.70. Agents: Phases, Analyst, Learning, Postmortem.

Matched a pack in the same sector. Other matches offered: `hr.job_description`, `retail.product_description`, `media.campaign_brief`.

#	Agent	Model	Conf.	Tok.	Output
1	Phases	haiku-5-5	0.63	97	[item 1 for: Split write a blameless postmortem into detection, r…
2	Analyst	sonnet-5-5	0.76	109	«Analyst #2241» Find what went well and what to improve in item 1…
3	Analyst	sonnet-5-5	0.65	109	«Analyst #2281» Find what went well and what to improve in item 2…
4	Analyst	sonnet-5-5	0.86	109	«Analyst #9106» Find what went well and what to improve in item 3…
5	Learning	sonnet-5-5	0.66	152	0.71 **human**
6	Postmortem	sonnet-5-5	0.71	157	«Postmortem #8854» Write a blameless postmortem from «Analyst #22…

Final answer: `[«Postmortem #8854» Write a blameless postmortem from «Analyst #2241» Find what went well …, 0.71]` (6 agent calls · 733 tokens · 0 retries · 1 escalated · 0 blocked)

41 SOC Alert Enrichment

Cybersecurity pipeline P41_soc_alert_enrichment.algo

An enrichment agent with a declared search tool, then severity and a plan.

Input

`alert` `text` Impossible travel login for the finance admin account, then a new mailbox rule

Output type: `text`

AlgoLang code

```
// SOC Alert Enrichment: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"
tool web.search

agent Enricher(alert: text) -> text {
   role "SOC analyst"
   goal "Look up context and threat intelligence for the alert: {alert}"
   uses web.search
   model smart
}

agent SeverityRater(context: text) -> text {
   role "SOC lead"
   goal "Rate the severity and likely attack stage from {context}"
   model smart
}

agent ResponsePlan(severity: text) -> text {
   role "Incident responder"
   goal "Write a containment and response plan for {severity}"
   model smart
```

```
}

flow main(alert: text) {
   context = Enricher(alert)
   severity = SeverityRater(context)
   plan = ResponsePlan(severity)
   emit plan
}
```

Output 1 – Command line

Terminal

```
$ algo run P41_soc_alert_enrichment.algo --arg "alert=Impossible travel login for the finance admin account, then a new mailbox rule"

▶ flow main  run=3cfcb023  backend=mock
    alert = 'Impossible travel login for the finance admin account, then a new mailbox rule'
  → Enricher           conf=0.89  «Enricher #0857» Look up context and threat intelligence for the alert:
  → SeverityRater      conf=0.78  «SeverityRater #7404» Rate the severity and likely attack stage from «En
  → ResponsePlan       conf=0.90  «ResponsePlan #0327» Write a containment and response plan for «Severity
■ emitted:
    • «ResponsePlan #0327» Write a containment and response plan for «SeverityRater #7404» Rate t…
■ 3 agent calls · ~309 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P41_soc_alert_enrichment.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 309 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	Enricher	sonnet-5-5	0.89	103	«Enricher #0857» Look up context and threat intelligence for the …
2	SeverityRater	sonnet-5-5	0.78	102	«SeverityRater #7404» Rate the severity and likely attack stage f…
3	ResponsePlan	sonnet-5-5	0.90	104	«ResponsePlan #0327» Write a containment and response plan for «S…

Result: [«ResponsePlan #0327» Write a containment and response plan for «SeverityRater #7404» Rate t…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "enrich and respond to a security alert"

AlgoLang: matched **SOC alert triage** (cyber.soc_triage), Cybersecurity, pattern fan, match score 0.41. Governance: human review below confidence 0.90. Agents: Batcher, Analyst, Severity, Incident.

Matched a pack in the same sector. Other matches offered: banking.aml_alerts, cyber.policy_qa, supplychain.route_disruption.

#	Agent	Model	Conf.	Tok.	Output
1	Batcher	haiku-5-5	0.94	93	[item 1 for: Split enrich and respond to a security alert into in…
2	Analyst	sonnet-5-5	0.86	105	«Analyst #0784» Investigate item 1 for: Split enrich and respond …
3	Analyst	sonnet-5-5	0.65	105	«Analyst #8720» Investigate item 2 for: Split enrich and respond …
4	Analyst	sonnet-5-5	0.67	105	«Analyst #2607» Investigate item 3 for: Split enrich and respond …
5	Severity	sonnet-5-5	0.94	150	0.83
6	Incident	sonnet-5-5	0.68	154	«Incident #4060» Write an incident summary from «Analyst #0784» I…

Final answer: [«Incident #4060» Write an incident summary from «Analyst #0784» Investigate item 1 for:…, 0.83] (6 agent calls · 712 tokens · 0 retries · 0 escalated · 0 blocked)

42 Phishing Email Check

Cybersecurity router P42_phishing_email_check.algo

Suspicious mail is quarantined; the rest is delivered with a note.

Input

`email`	`text`	Urgent: your payroll account is locked. Click http://secure-payroll-login.example to verify

Output type: `score, text`

AlgoLang code

```
// Phishing Email Check: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent PhishScorer(email: text) -> score {
   role "Email security analyst"
   goal "Score the phishing likelihood of {email} from 0 to 1"
   model fast
}

agent QuarantineNotice(email: text) -> text {
   role "Security analyst"
   goal "Write a quarantine notice and user guidance for {email}"
   model smart
}

agent DeliverNote(email: text) -> text {
   role "Security analyst"
   goal "Write a short note that {email} looks safe with a caution"
   model fast
}

flow main(email: text) {
   phish = PhishScorer(email)

   if phish >= 0.7 {
      quarantine = QuarantineNotice(email)
      emit phish, quarantine
   } else {
      deliver = DeliverNote(email)
      emit phish, deliver
   }
}
```

Output 1 – Command line

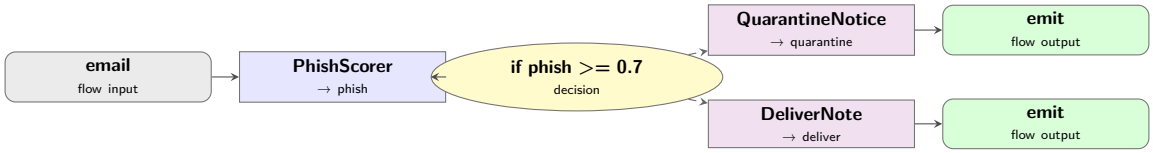
Terminal

```
$ algo run P42_phishing_email_check.algo --arg "email=Urgent: your payroll account is locked. Click
↪  http://secure-payroll-login.example to verify"

▶ flow main  run=ac0e17d9  backend=mock
    email = 'Urgent: your payroll account is locked. Click http://secure-payroll-login.example to verify'
  → PhishScorer        conf=0.81  0.91
  → QuarantineNotice   conf=0.66  «QuarantineNotice #3034» Write a quarantine notice and user guidance for
■ emitted:
    • 0.91
    • «QuarantineNotice #3034» Write a quarantine notice and user guidance for Urgent: your payroll a…
■ 2 agent calls · ~210 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P42_phishing_email_check.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 210 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`PhishScorer`	haiku-5-5	0.81	105	0.91
2	`QuarantineNotice`	sonnet-5-5	0.66	105	«QuarantineNotice #3034» Write a quarantine notice and user guida…

Result: `[0.91, «QuarantineNotice #3034» Write a quarantine notice and user guidance for Urgent: your payroll a…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "check whether an email is phishing"

AlgoLang: matched **Phishing email analysis** (`cyber.phishing`), Cybersecurity, pattern `pipe`, match score 0.89. Governance: human review below confidence 0.80. Agents: Indicators, Assessment, Likelihood, Advice.

Matched a pack in the same sector. Other matches offered: `cyber.policy_qa`, `cyber.soc_triage`, `cyber.access_review`.

#	Agent	Model	Conf.	Tok.	Output
1	`Indicators`	sonnet-5-5	0.65	93	«Indicators #4033» List phishing indicators in check whether an e…
2	`Assessment`	sonnet-5-5	0.67	106	«Assessment #5076» Assess sender, links and intent in «Indicators…
3	`Likelihood`	sonnet-5-5	0.83	109	0.56
4	`Advice`	sonnet-5-5	0.63	112	«Advice #0966» Write user and SOC advice for «Assessment #5076» A…

Final answer: `[«Advice #0966» Write user and SOC advice for «Assessment #5076» Assess sender, links …, 0.56]` (4 agent calls · 420 tokens · 0 retries · 0 escalated · 0 blocked)

43 Grid Anomaly Router

Energy router P43_grid_anomaly_router.algo

Abnormal readings alert a human operator.

Input

`readings`	`text`	Feeder 14 voltage drops 9 percent at 18:00 for 12 minutes, three nights this week

Output type: `score, text`

AlgoLang code

```
// Grid Anomaly Router: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent AnomalyScorer(readings: text) -> score {
   role "Grid analyst"
   goal "Score how abnormal {readings} are from 0 to 1"
   model fast
}

agent OperatorAlert(readings: text) -> text {
   role "Control room lead"
   goal "Write an operator alert with checks for {readings}"
   model smart
}

agent LogOnly(readings: text) -> text {
   role "Control room lead"
   goal "Write a log entry for {readings}"
   model fast
}

flow main(readings: text) {
   anomaly = AnomalyScorer(readings)

   if anomaly >= 0.7 {
      alert = OperatorAlert(readings)
      emit anomaly, alert
   } else {
      log = LogOnly(readings)
      emit anomaly, log
   }
}
```

Output 1 – Command line

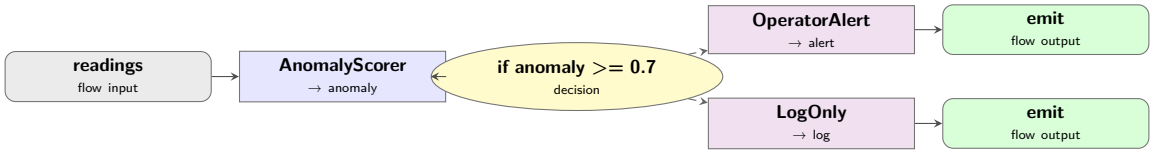
Terminal

```
$ algo run P43_grid_anomaly_router.algo --arg "readings=Feeder 14 voltage drops 9 percent at 18:00 for 12 minutes, three nights this
↪  week"

▶ flow main  run=206f0fc9  backend=mock
    readings = 'Feeder 14 voltage drops 9 percent at 18:00 for 12 minutes, three nights this week'
  → AnomalyScorer      conf=0.73  0.78
  → OperatorAlert      conf=0.90  «OperatorAlert #7141» Write an operator alert with checks for Feeder 14
■ emitted:
    • 0.78
    • «OperatorAlert #7141» Write an operator alert with checks for Feeder 14 voltage drops 9 perc…
■ 2 agent calls · ~198 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P43_grid_anomaly_router.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 198 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	`AnomalyScorer`	haiku-5-5	0.73	98	0.78
2	`OperatorAlert`	sonnet-5-5	0.90	100	«OperatorAlert #7141» Write an operator alert with checks for Fee…

Result: `[0.78, «OperatorAlert #7141» Write an operator alert with checks for Feeder 14 voltage drops 9 perc…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "decide whether a power grid anomaly needs an operator"

AlgoLang: matched **Grid fault analysis** (`energy.grid_fault`), Energy & Utilities, pattern `pipe`, match score 0.52. Governance: human review below confidence 0.90. Agents: Events, Cause, Confidence, Report.

Matched a pack in the same sector. Other matches offered: `energy.outage_response`, `energy.meter_anomaly`, `energy.renewable_site`.

#	Agent	Model	Conf.	Tok.	Output
1	`Events`	sonnet-5-5	0.89	96	«Events #9223» Order the protection events in decide whether a po…
2	`Cause`	sonnet-5-5	0.71	106	«Cause #5437» Identify the likely fault cause from «Events #9223»…
3	`Confidence`	sonnet-5-5	0.81	107	0.88 **human**
4	`Report`	sonnet-5-5	0.72	108	«Report #5420» Write a fault report from «Cause #5437» Identify t…

Final answer: `[«Report #5420» Write a fault report from «Cause #5437» Identify the likely fault caus…, 0.88]` (4 agent calls · 417 tokens · 0 retries · 1 escalated · 0 blocked)

44 Solar Site Assessment

Energy fan-out P44_solar_site_assessment.algo

Sun, grid and cost views are merged into a brief.

Input

`site` `text` 10 acre barren plot near Nashik, 3 km from a 33 kV substation
Output type: `text`

AlgoLang code

```
// Solar Site Assessment: workers run in parallel, one agent merges.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent IrradianceAnalyst(site: text) -> text {
   role "Solar engineer"
   goal "Estimate the sunlight potential and shading risks for {site}"
   model smart
}

agent GridAnalyst(site: text) -> text {
   role "Grid engineer"
   goal "Assess grid connection options for {site}"
   model smart
}

agent CostAnalyst(site: text) -> text {
   role "Project finance analyst"
   goal "Estimate capital cost and payback range for {site}"
   model fast
}

agent FeasibilityBrief(sun: text, grid: text, cost: text) -> text {
   role "Project lead"
   goal "Write a feasibility brief from {sun}, {grid} and {cost}"
   model smart
}

flow main(site: text) {
   [sun, grid, cost] = par {
      IrradianceAnalyst(site),
      GridAnalyst(site),
      CostAnalyst(site)
   }
   brief = FeasibilityBrief(sun, grid, cost)
   emit brief
}
```

Output 1 – Command line

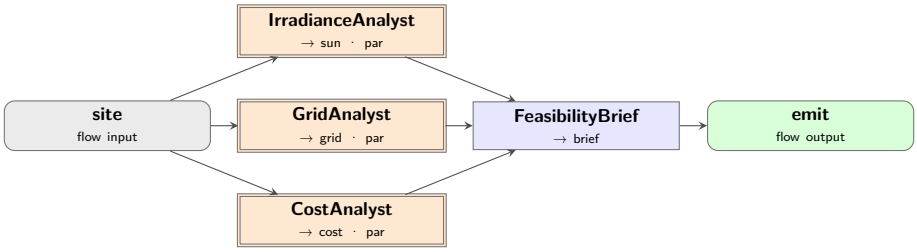
Terminal

```
$ algo run P44_solar_site_assessment.algo --arg "site=10 acre barren plot near Nashik, 3 km from a 33 kV substation"

▶ flow main  run=c2860207  backend=mock
    site = '10 acre barren plot near Nashik, 3 km from a 33 kV substation'
  → IrradianceAnalyst  conf=0.75  «IrradianceAnalyst #0106» Estimate the sunlight potential and shading ri
  → GridAnalyst        conf=0.95  «GridAnalyst #6451» Assess grid connection options for 10 acre barren pl
  → CostAnalyst        conf=0.95  «CostAnalyst #8487» Estimate capital cost and payback range for 10 acre
  → FeasibilityBrief   conf=0.93  «FeasibilityBrief #8916» Write a feasibility brief from «IrradianceAnaly
■ emitted:
    • «FeasibilityBrief #8916» Write a feasibility brief from «IrradianceAnalyst #0106» Estimate the …
■ 4 agent calls · ~437 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P44_solar_site_assessment.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 437 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	IrradianceAnalyst	sonnet-5-5	0.75	98	«IrradianceAnalyst #0106» Estimate the sunlight potential and sha…
2	CostAnalyst	haiku-5-5	0.95	98	«CostAnalyst #8487» Estimate capital cost and payback range for 1…
3	GridAnalyst	sonnet-5-5	0.95	93	«GridAnalyst #6451» Assess grid connection options for 10 acre ba…
4	FeasibilityBrief	sonnet-5-5	0.93	148	«FeasibilityBrief #8916» Write a feasibility brief from «Irradian…

Result: [«FeasibilityBrief #8916» Write a feasibility brief from «IrradianceAnalyst #0106» Estimate the …]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "assess a site for a solar plant"

AlgoLang: matched **Renewable site assessment** (energy.renewable_site), Energy & Utilities, pattern committee, match score 0.59. Governance: human review below confidence 0.80. Agents: ResourceAnalyst, GridAnalyst, EnvironmentAnalyst, Panel, Report.

Matched a pack in the same sector. Other matches offered: agriculture.crop_advisory, realestate.site_safety, agriculture.scheme_eligibility.

#	Agent	Model	Conf.	Tok.	Output
1	ResourceAnalyst	sonnet-5-5	0.64	97	«ResourceAnalyst #8042» Assess solar and wind resource at assess …
2	EnvironmentAnalyst	sonnet-5-5	0.94	98	«EnvironmentAnalyst #7507» Assess environmental and land issues f…
3	GridAnalyst	sonnet-5-5	0.92	93	«GridAnalyst #0079» Assess grid connection for assess a site for …
4	Panel	sonnet-5-5	0.72	153	0.90 **human**
5	Report	sonnet-5-5	0.68	157	«Report #2459» Write a site assessment from «ResourceAnalyst #804…

Final answer: [«Report #2459» Write a site assessment from «ResourceAnalyst #8042» Assess solar and …, 0.90] (5 agent calls · 598 tokens · 0 retries · 1 escalated · 0 blocked)

45 Crop Advisory

Agriculture pipeline P45_crop_advisory.algo

Soil reading, action steps, and a simple advisory.

Input

field_report	text	Black soil, 2 acres, onion nursery, leaves yellowing, last rain 10 days ago

Output type: text

AlgoLang code

```
// Crop Advisory: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent SoilReader(field_report: text) -> text {
   role "Agronomist"
   goal "Describe likely soil and water issues in {field_report}"
   model smart
}

agent CropPlanner(soil_view: text) -> text {
   role "Agronomist"
   goal "Suggest fertiliser and irrigation steps for {soil_view}"
   model smart
}

agent FarmerAdvisory(steps: text) -> text {
   role "Extension officer"
   goal "Rewrite {steps} as a short advisory in very simple language"
   model fast
}

flow main(field_report: text) {
   soil_view = SoilReader(field_report)
   steps = CropPlanner(soil_view)
   advisory = FarmerAdvisory(steps)
   emit advisory
}
```

Output 1 – Command line

Terminal

```
$ algo run P45_crop_advisory.algo --arg "field_report=Black soil, 2 acres, onion nursery, leaves yellowing, last rain 10 days ago"

▶ flow main  run=6cf8e6cc  backend=mock
    field_report = 'Black soil, 2 acres, onion nursery, leaves yellowing, last rain 10 days ago'
  → SoilReader         conf=0.95  «SoilReader #4296» Describe likely soil and water issues in Black soil,
  → CropPlanner        conf=0.66  «CropPlanner #5413» Suggest fertiliser and irrigation steps for «SoilRea
  → FarmerAdvisory     conf=0.70  «FarmerAdvisory #6590» Rewrite «CropPlanner #5413» Suggest fertiliser an
■ emitted:
    • «FarmerAdvisory #6590» Rewrite «CropPlanner #5413» Suggest fertiliser and irrigation steps fo…
■ 3 agent calls · ~305 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P45_crop_advisory.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 305 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	SoilReader	sonnet-5-5	0.95	97	«SoilReader #4296» Describe likely soil and water issues in Black…
2	CropPlanner	sonnet-5-5	0.66	102	«CropPlanner #5413» Suggest fertiliser and irrigation steps for «…
3	FarmerAdvisory	haiku-5-5	0.70	106	«FarmerAdvisory #6590» Rewrite «CropPlanner #5413» Suggest fertil…

Result: `[«FarmerAdvisory #6590» Rewrite «CropPlanner #5413» Suggest fertiliser and irrigation steps fo…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "give crop advice to a farmer in local language"

AlgoLang: matched **Crop advisory** (`agriculture.crop_advisory`), Agriculture, pattern `pipe`, match score 0.68. Governance: human review below confidence 0.70. Agents: Advisor, Translator, Safety, Final.

Matched a pack in the same sector. Other matches offered: `agriculture.scheme_eligibility`, `agriculture.farmer_queries`, `agriculture.market_prices`.

#	Agent	Model	Conf.	Tok.	Output
1	Advisor	sonnet-5-5	0.70	99	«Advisor #2514» Give weather-aware sowing and care advice for giv…
2	Translator	sonnet-5-5	0.82	107	«Translator #2513» Translate «Advisor #2514» Give weather-aware s…
3	Safety	sonnet-5-5	0.91	109	0.89
4	Final	sonnet-5-5	0.76	106	«Final #0286» Finalise the advisory «Translator #2513» Translate …

Final answer: `[«Final #0286» Finalise the advisory «Translator #2513» Translate «Advisor #2514» Giv…, 0.89]` (4 agent calls · 421 tokens · 0 retries · 0 escalated · 0 blocked)

46 Pest Alert Triage

Agriculture router P46_pest_alert_triage.algo

Likely outbreaks alert the district officer.

Input

`observation` `text` Armyworm larvae found on 20 percent of maize plants across three neighbouring farms

Output type: `score, text`

AlgoLang code

```
// Pest Alert Triage: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent OutbreakScorer(observation: text) -> score {
   role "Plant protection officer"
   goal "Score the outbreak risk of {observation} from 0 to 1"
   model fast
}

agent OfficerAlert(observation: text) -> text {
   role "Extension officer"
   goal "Write an alert to the district officer for {observation}"
   model smart
}

agent FarmerTip(observation: text) -> text {
   role "Extension officer"
   goal "Write a simple scouting tip for farmers about {observation}"
   model fast
}

flow main(observation: text) {
   outbreak = OutbreakScorer(observation)
```

```
   if outbreak >= 0.6 {
      officer_alert = OfficerAlert(observation)
      emit outbreak, officer_alert
   } else {
      tip = FarmerTip(observation)
      emit outbreak, tip
   }
}
```

Output 1 – Command line

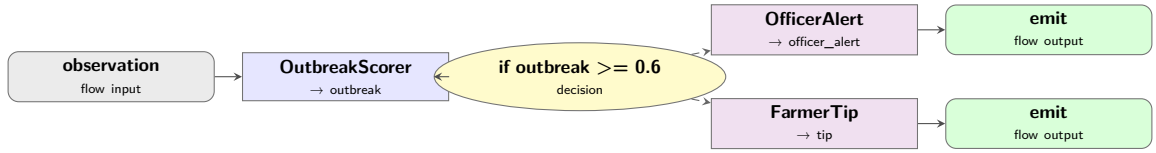
Terminal

```
$ algo run P46_pest_alert_triage.algo --arg "observation=Armyworm larvae found on 20 percent of maize plants across three neighbouring
↪  farms"

▶ flow main  run=176bb6b5  backend=mock
    observation = 'Armyworm larvae found on 20 percent of maize plants across three neighbouring farms'
  → OutbreakScorer     conf=0.70  0.93
  → OfficerAlert       conf=0.89  «OfficerAlert #8282» Write an alert to the district officer for Armyworm
■ emitted:
    • 0.93
    • «OfficerAlert #8282» Write an alert to the district officer for Armyworm larvae found on 20…
■ 2 agent calls · ~204 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P46_pest_alert_triage.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 2 agent calls · 204 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	OutbreakScorer	haiku-5-5	0.70	102	0.93
2	OfficerAlert	sonnet-5-5	0.89	102	«OfficerAlert #8282» Write an alert to the district officer for A…

Result: `[0.93, «OfficerAlert #8282» Write an alert to the district officer for Armyworm larvae found on 20…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "triage a crop pest sighting"

AlgoLang: matched **Pest and disease diagnosis** (`agriculture.pest_diagnosis`), Agriculture, pattern `committee`, match score 0.54. Governance: human review below confidence 0.80. Agents: Entomologist, Pathologist, Agronomist, Panel, Advice.

Matched a pack in the same sector. Other matches offered: `agriculture.crop_advisory`, `agriculture.scheme_eligibility`, `agriculture.farmer_queries`.

#	Agent	Model	Conf.	Tok.	Output
1	Entomologist	sonnet-5-5	0.70	93	«Entomologist #1363» Identify likely pests from triage a crop pes…
2	Pathologist	sonnet-5-5	0.78	93	«Pathologist #7644» Identify likely diseases from triage a crop p…
3	Agronomist	sonnet-5-5	0.75	95	«Agronomist #8764» Identify nutrient or water stress from triage …
4	Panel	sonnet-5-5	0.78	158	0.90 **human**
5	Advice	sonnet-5-5	0.76	160	«Advice #3562» Write treatment advice from «Entomologist #1363» I…

Final answer: `[«Advice #3562» Write treatment advice from «Entomologist #1363» Identify likely pests…, 0.90]` (5 agent calls · 599 tokens · 0 retries · 1 escalated · 0 blocked)

47 Claim Fact-Check

Media par for P47_claim_fact_check.algo

Each claim is checked with a search tool, then reported.

Input

`claims` `list` City built 50 new schools in 2025, Rainfall was highest in 30 years

Output type: `text, list`

AlgoLang code

```
// Claim Fact-Check: par for fans one agent out over a list.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"
tool web.search

agent FactChecker(item: text) -> text {
   role "Fact-checker"
   goal "Check the claim {item} against reliable sources and give a verdict with evidence"
   uses web.search
   model fast
}

agent VerdictReport(results: list) -> text {
   role "Standards editor"
   goal "Write a verdict report from {results}"
   model smart
}

flow main(claims: list) {
   results = par for it in claims { FactChecker(it) }
   report = VerdictReport(results)
   emit report, results
}
```

Output 1 – Command line

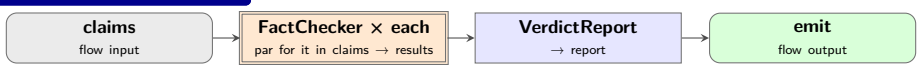
Terminal

```
$ algo run P47_claim_fact_check.algo --arg "claims=City built 50 new schools in 2025, Rainfall was highest in 30 years"

▶ flow main  run=f1e5d9f4  backend=mock
    claims = 'City built 50 new schools in 2025, Rainfall was highest in 30 years'
  → FactChecker        conf=0.76  «FactChecker #8561» Check the claim City built 50 new schools in 2025, R
  → VerdictReport      conf=0.62  «VerdictReport #1434» Write a verdict report from «FactChecker #8561» Ch
■ emitted:
    • «VerdictReport #1434» Write a verdict report from «FactChecker #8561» Check the claim City b…
    • «FactChecker #8561» Check the claim City built 50 new schools in 2025, Rainfall was highes…
■ 2 agent calls · ~206 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P47_claim_fact_check.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: `mock` 2 agent calls · 206 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	FactChecker	haiku-5-5	0.76	106	«FactChecker #8561» Check the claim City built 50 new schools in …
2	VerdictReport	sonnet-5-5	0.62	100	«VerdictReport #1434» Write a verdict report from «FactChecker #8…

Result: `[«VerdictReport #1434» Write a verdict report from «FactChecker #8561» Check the claim City b…, [«FactChecker #8561» Check the claim City built 50 ne…`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "fact-check claims for a press release"

AlgoLang: matched **Press release writer** (`media.press_release`), Media & Marketing, pattern `pipe`, match score 0.74. Governance: human review below confidence 0.80. Agents: Facts, Writer, Accuracy, Final.

Matched a pack in the same sector. Other matches offered: `insurance.claims_triage`, `insurance.fraud_screening`, `insurance.subrogation`.

#	Agent	Model	Conf.	Tok.	Output
1	Facts	sonnet-5-5	0.67	90	«Facts #6162» List the key facts in fact-check claims for a press…
2	Writer	sonnet-5-5	0.89	101	«Writer #0782» Write a press release from «Facts #6162» List the …
3	Accuracy	sonnet-5-5	0.93	106	0.65
4	Final	sonnet-5-5	0.63	107	«Final #1708» Finalise the release «Writer #0782» Write a press r…

Final answer: `[«Final #1708» Finalise the release «Writer #0782» Write a press release from «Facts …, 0.65]` (4 agent calls · 404 tokens · 0 retries · 0 escalated · 0 blocked)

48 Headline Variants

Media fan-out P48_headline_variants.algo

Three styles in parallel; an editor picks.

Input

`story` `text` City council approves new metro line connecting airport to the old town by 2029

Output type: `text`

AlgoLang code

```
// Headline Variants: workers run in parallel, one agent merges.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent StraightHeadline(story: text) -> text {
   role "News editor"
   goal "Write a straight news headline for {story}"
   model fast
}

agent CuriosityHeadline(story: text) -> text {
   role "Digital editor"
   goal "Write a curiosity-driven headline for {story}"
   model fast
}

agent SocialHeadline(story: text) -> text {
   role "Social editor"
   goal "Write a social-media headline for {story}"
   model fast
}

agent EditorPick(straight: text, curious: text, social: text) -> text {
   role "Chief editor"
   goal "Pick the best of {straight}, {curious} and {social} and explain why"
   model smart
}

flow main(story: text) {
   [straight, curious, social] = par {
      StraightHeadline(story),
      CuriosityHeadline(story),
      SocialHeadline(story)
   }
   pick = EditorPick(straight, curious, social)
   emit pick
}
```

Output 1 – Command line

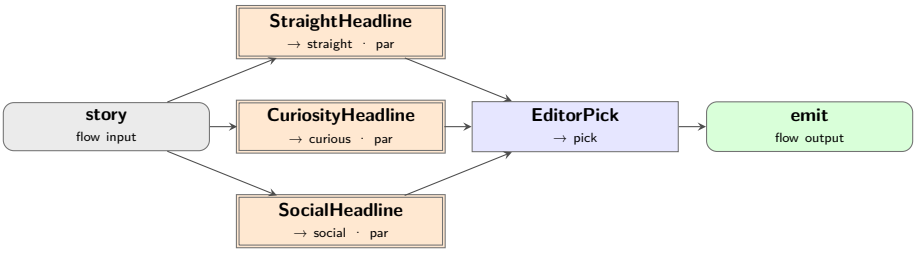
Terminal

```
$ algo run P48_headline_variants.algo --arg "story=City council approves new metro line connecting airport to the old town by 2029"

▶ flow main  run=3e67cc49  backend=mock
    story = 'City council approves new metro line connecting airport to the old town by 2029'
  → StraightHeadline   conf=0.72  «StraightHeadline #4876» Write a straight news headline for City council
  → SocialHeadline     conf=0.75  «SocialHeadline #0061» Write a social-media headline for City council ap
  → CuriosityHeadline  conf=0.91  «CuriosityHeadline #5387» Write a curiosity-driven headline for City cou
  → EditorPick         conf=0.86  «EditorPick #8708» Pick the best of «StraightHeadline #4876» Write a str
■ emitted:
    • «EditorPick #8708» Pick the best of «StraightHeadline #4876» Write a straight news headli…
■ 4 agent calls · ~444 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P48_headline_variants.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 4 agent calls · 444 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	StraightHeadline	haiku-5-5	0.72	97	«StraightHeadline #4876» Write a straight news headline for City …
2	SocialHeadline	haiku-5-5	0.75	97	«SocialHeadline #0061» Write a social-media headline for City cou…
3	CuriosityHeadline	haiku-5-5	0.91	99	«CuriosityHeadline #5387» Write a curiosity-driven headline for C…
4	EditorPick	sonnet-5-5	0.86	151	«EditorPick #8708» Pick the best of «StraightHeadline #4876» Writ…

Result: [«EditorPick #8708» Pick the best of «StraightHeadline #4876» Write a straight news headli…]

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "write a campaign brief with headline options"

AlgoLang: matched **Campaign brief** (media.campaign_brief), Media & Marketing, pattern pipe, match score 0.58. Governance: human review below confidence 0.70. Agents: Strategist, Creative, Fit, Brief.

Matched a pack in the same sector. Other matches offered: legal.litigation_risk, government.policy_brief, markets.portfolio_risk.

#	Agent	Model	Conf.	Tok.	Output
1	Strategist	sonnet-5-5	0.88	94	«Strategist #5208» Define audience and message for write a campai…
2	Creative	sonnet-5-5	0.94	106	«Creative #2939» Write three campaign concepts from «Strategist #…
3	Fit	sonnet-5-5	0.76	102	0.88
4	Brief	sonnet-5-5	0.78	106	«Brief #0476» Write a campaign brief from «Creative #2939» Write …

Final answer: [«Brief #0476» Write a campaign brief from «Creative #2939» Write three campaign conc…, 0.88] (4 agent calls · 408 tokens · 0 retries · 0 escalated · 0 blocked)

49 Listing Writer

Real Estate pipeline P49_listing_writer.algo

Features, copy, and a fair-housing language check.

Input

property_details	text	3 BHK, 1450 sq ft, 9th floor, east facing, near metro, covered parking, 1.2 crore

Output type: text

AlgoLang code

```
// Listing Writer: a pipeline of 3 agents.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent FeatureExtractor(property_details: text) -> text {
   role "Property consultant"
   goal "List the top selling features in {property_details}"
   model fast
}

agent ListingCopywriter(features: text) -> text {
   role "Real estate copywriter"
   goal "Write an engaging 80-word listing from {features}"
   model smart
}

agent FairHousingCheck(copy: text) -> text {
   role "Compliance reviewer"
   goal "Check {copy} for discriminatory language and fix it"
   model smart
}

flow main(property_details: text) {
   features = FeatureExtractor(property_details)
   copy = ListingCopywriter(features)
   final_copy = FairHousingCheck(copy)
   emit final_copy
}
```

Output 1 – Command line

Terminal

```
$ algo run P49_listing_writer.algo --arg "property_details=3 BHK, 1450 sq ft, 9th floor, east facing, near metro, covered parking, 1.2
↪  crore"

▶ flow main  run=ffcccc13  backend=mock
    property_details = '3 BHK, 1450 sq ft, 9th floor, east facing, near metro, covered parking, 1.2 crore'
  → FeatureExtractor   conf=0.82  «FeatureExtractor #2716» List the top selling features in 3 BHK, 1450 sq
  → ListingCopywriter  conf=0.93  «ListingCopywriter #9090» Write an engaging 80-word listing from «Featur
  → FairHousingCheck   conf=0.76  «FairHousingCheck #0456» Check «ListingCopywriter #9090» Write an engagi
■ emitted:
    • «FairHousingCheck #0456» Check «ListingCopywriter #9090» Write an engaging 80-word listing from…
■ 3 agent calls · ~310 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P49_listing_writer.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 3 agent calls · 310 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	FeatureExtractor	haiku-5-5	0.82	99	«FeatureExtractor #2716» List the top selling features in 3 BHK, …
2	ListingCopywriter	sonnet-5-5	0.93	105	«ListingCopywriter #9090» Write an engaging 80-word listing from …
3	FairHousingCheck	sonnet-5-5	0.76	106	«FairHousingCheck #0456» Check «ListingCopywriter #9090» Write an…

Result: `[«FairHousingCheck #0456» Check «ListingCopywriter #9090» Write an engaging 80-word listing from…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "write a property listing"

AlgoLang: matched **Property valuation brief** (`realestate.valuation`), Real Estate & Construction, pattern `committee`, match score 0.49.

Governance: human review below confidence 0.80. Agents: MarketAnalyst, IncomeAnalyst, CostAnalyst, Panel, Report.

Matched a pack in the same sector. Other matches offered: `cyber.access_review`, `supplychain.customs_documents`, `retail.catalog_quality`.

#	Agent	Model	Conf.	Tok.	Output
1	MarketAnalyst	sonnet-5-5	0.85	95	«MarketAnalyst #5713» Estimate value from comparables for write a…
2	IncomeAnalyst	sonnet-5-5	0.81	92	«IncomeAnalyst #8284» Estimate value from income for write a prop…
3	CostAnalyst	sonnet-5-5	0.82	94	«CostAnalyst #0912» Estimate value from replacement cost for writ…
4	Panel	sonnet-5-5	0.88	166	0.61
5	Report	sonnet-5-5	0.80	169	«Report #0525» Write a valuation brief from «MarketAnalyst #5713»…

Final answer: `[«Report #0525» Write a valuation brief from «MarketAnalyst #5713» Estimate value from…, 0.61]` (5 agent calls · 616 tokens · 0 retries · 0 escalated · 0 blocked)

50 Hotel Review Responder

Hospitality router `P50_hotel_review_responder.algo`

Happy guests get thanks; unhappy ones get a service-recovery reply.

Input

`guest_review`	`text`	Lovely staff and view, but the room was not cleaned on day two and breakfast was cold.

Output type: `score, text`

AlgoLang code

```
// Hotel Review Responder: a router that branches on a score.
model fast = "claude-haiku-5-5"
model smart = "claude-sonnet-5-5"

agent ReviewSentiment(guest_review: text) -> score {
   role "Guest relations manager"
   goal "Score the overall satisfaction in {guest_review} from 0 to 1"
   model fast
}

agent ThankYouReply(guest_review: text) -> text {
   role "Guest relations manager"
   goal "Write a warm thank-you reply to {guest_review}"
   model smart
}

agent ServiceRecovery(guest_review: text) -> text {
   role "Guest relations manager"
   goal "Write an apology and service-recovery reply to {guest_review}"
   model fast
}

flow main(guest_review: text) {
   satisfaction = ReviewSentiment(guest_review)
```

```
   if satisfaction >= 0.5 {
      thanks = ThankYouReply(guest_review)
      emit satisfaction, thanks
   } else {
      recovery = ServiceRecovery(guest_review)
      emit satisfaction, recovery
   }
}
```

Output 1 – Command line

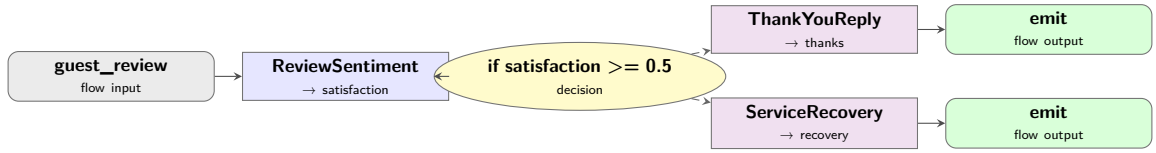
Terminal

```
$ algo run P50_hotel_review_responder.algo --arg "guest_review=Lovely staff and view, but the room was not cleaned on day two and
↪  breakfast was cold."

▶ flow main  run=c8b35583  backend=mock
    guest_review = 'Lovely staff and view, but the room was not cleaned on day two and breakfast was cold.'
  → ReviewSentiment    conf=0.69  0.67
  → ThankYouReply      conf=0.79  «ThankYouReply #5983» Write a warm thank-you reply to Lovely staff and v
■ emitted:
    • 0.67
    • «ThankYouReply #5983» Write a warm thank-you reply to Lovely staff and view, but the room wa…
■ 2 agent calls · ~206 tokens · 0 retries · 0 escalated · 0 blocked · audit → runs/P50_hotel_review_responder.audit.jsonl
```

Output 2 – Graphical UI (AlgoLang Studio)

Studio: agent graph and run panel



Run panel backend: mock 2 agent calls · 206 tokens · 0 retries · 0 escalated · 0 blocked

#	Agent	Model	Conf.	Tok.	Output
1	ReviewSentiment	haiku-5-5	0.69	105	0.67
2	ThankYouReply	sonnet-5-5	0.79	101	«ThankYouReply #5983» Write a warm thank-you reply to Lovely staf…

Result: `[0.67, «ThankYouReply #5983» Write a warm thank-you reply to Lovely staff and view, but the room wa…]`

Output 3 – Prompt UI

Prompt UI: describe the problem in plain English

You: "turn guest reviews into actions"

AlgoLang: matched **Guest review insights** (`hospitality.guest_reviews`), Hospitality & Travel, pattern fan, match score 0.84. Governance: human review below confidence 0.70. Agents: Splitter, Analyst, Satisfaction, Report.

Matched a pack in the same sector. Other matches offered: `hospitality.booking_router`, `retail.review_insights`, `hospitality.loyalty_offers`.

#	Agent	Model	Conf.	Tok.	Output
1	Splitter	haiku-5-5	0.82	92	[item 1 for: Split turn guest reviews into actions into individua…
2	Analyst	sonnet-5-5	0.66	104	«Analyst #5063» Extract service issues and praise from item 1 for…
3	Analyst	sonnet-5-5	0.92	104	«Analyst #2589» Extract service issues and praise from item 4 for…
4	Analyst	sonnet-5-5	0.65	104	«Analyst #8935» Extract service issues and praise from item 2 for…
5	Analyst	sonnet-5-5	0.65	104	«Analyst #6557» Extract service issues and praise from item 3 for…
6	Satisfaction	sonnet-5-5	0.73	175	0.64
7	Report	sonnet-5-5	0.87	179	«Report #6436» Write a guest experience report from «Analyst #506…

Final answer: `[«Report #6436» Write a guest experience report from «Analyst #5063» Extract service i…, 0.64]` (7 agent calls · 862 tokens · 0 retries · 0 escalated · 0 blocked)

Appendix: running the programs yourself

```
pip install git+https://github.com/drssnaiml1-cell/algolang
algo doctor

# 1. Command line  (add --live to call real Claude models)
algo check  P14_credit_committee.algo
algo run    P14_credit_committee.algo --arg "application=Salaried, income 85000 per month"

# 2. Graphical UI
algo studio                 # open the .algo file in the editor, then press Run

# 3. Prompt UI
algo ui                     # type the plain-English request in the chat box
```

The 50 `.algo` source files and this document's `.tex` source are supplied alongside the PDF.